

VITALs

Visual Intelligent Tactical Assistant for Low-power Operations

Authors:

Diego Aguirre
Computer Science

Aniket Bansal
Computer Science

Brody Moorman
Computer Science

Edward Rosales
Computer Science

Ricardo Sanchez-Gordon
Computer Science

James Trent
Computer Science

Reviewer Committee:

Mark Heinrich

Tanvir Ahmed

Name

Mentor:

Dr. Richard Leinecker



Table of Contents

Introduction - Original Design Proposal

- 1) Objective
- 2) Key System Components
- 3) Key Phases of Operation
- 4) Emergency Procedures

Diego

- 1) Research
- 2) Hardware
- 3) Swarm Movement

James

- 1) Deterministic Swarm Movement Introduction
- 2) Swarm Intelligence
- 3) Swarm Movement Algorithms
- 4) Deep Reinforcement Learning
- 5) Drone Distance and Measurement
- 6) Drone Swarm AI Operating Information
- 7) Cybersecurity of the Drone Swarm
- 8) Drone Charging and Replenishment

- 9) Drone Swarms in Wild and Urban Areas
- 10) Dual-Use Cases for an AI Drone Swarm
- 11) Impact in the Search-and-Rescue Community
- 12) Stretch Goals
- 13) Ollama
- 14) Ollama – Llama 3.2 Vision
- 15) LLaVa and Ollama
- 16) First Prototype Installation
- 17) Model Improvements
- 18) Native Drone Computation
- 19) Training and Reinforcement Learning
- 20) Transitioning from Single Drone to Drone Swarm Model
- 21) Search Algorithm Implementation
- 22) Expected End Results

Ricardo

- 1) Updated General Requirements
- 2) Project Sequence
- 3) Expanding LLM Memory
- 4) LlamaIndex with Llava
- 5) ChromaDB
- 6) Accurate object detection and tracking at long distances
- 7) Front-End
- 8) Drone malfunction Anomaly detection
 - a. Dataset
 - b. Anomaly Detection Toolkit (ADTK)
 - c. Potential complications of fault anomaly detection
- 9) Scenery Drone Anomaly detection
 - a. Dataset
 - b. Challenges in acquiring dataset

- c. Attention-based Residual Autoencoder for Video Anomaly Detection (ASTNet)
- d. Potential complications of scenery Anomaly detection

Edward

- 1) Computer Vision
 - e. Introduction
 - f. YOLO Models
 - g. Object Detection
 - h. Challenges with Computer Vision
 - i. Integration of AI and Decision-Making
 - j. YOLO in the Drone Swarm
 - k. Results
- 2) Computer Vision to LLM
 - a. Introduction
 - b. Data Format Design
 - c. Data Transfer Mechanisms
 - d. Prompt Engineering
 - e. Workflow Summary
- 3) Swarm Communication
 - a. Introduction
 - b. Communication Protocols
 - c. Data Types to Exchange and Prioritization
 - d. Communication Summary

Aniket

- 1. Drone Testing Regulations and Legal Framework near UCF, Florida
- 2. Federal Regulations: FAA Part 107
- 3. State Regulations (Florida)

4. Local Regulations (Orlando)
5. Special Considerations for Drone Testing
6. Drone Swarms for Rescue Missions
7. Technical Challenges and Solutions in Drone Swarm Operations
8. Drones in Search and Rescue (SAR) Missions
9. Safety and Reliability in Drone Swarm Rescue Operations
10. Real-World Applications
11. Future Trends in Drone Swarm Technology
12. Ethical Implications of Using Drones in Rescue Missions
13. Data Management in Drone Swarm Rescue Missions
14. Regulatory Challenges in Civilian Use
15. Humanitarian and Environmental Impact
16. Scalability of Drone Swarms in Search-and-Rescue Missions
17. Benefits of Scalability in Drone Swarms
18. Challenges of Scaling Drone Swarms
19. Workforce Development for Scalable Drone Swarms
20. Training Operators for Complex Systems
21. Training Maintenance Teams for Technical Excellence
22. Ensuring Safety and Compliance
23. Ongoing Education and Certification
24. Broader Implications of Scalability
25. Advanced Algorithms for Drone Swarm Operations

Brody

- 1) Testing
 - a. Python Interface
 - b. Test Script
 - c. Web Interface
 - d. Front-End
 - e. Back-End

- 2) Coordinate Calculations
 - a. Caveats
 - b. Code Implementation
- 3) Tool Calling
 - a. Overview
 - b. Tool Calling in Ollama
 - c. Groq
 - d. Shortcomings of tool-calling in Ollama
 - e. LangChain
- 4) LangGraph
 - a. Implementation
 - i. Graph Definition
 - ii. Graph Construction
 - b. LangGraph for Drone Agent
- 5) Application Backend

Final Implementation

- 1) GitHub
- 2) Application Structure
 - a. Mission State
 - b. Dispatcher
 - c. GUI
 - d. LLM Interactions
 - e. Terrain Pre-Processing
 - f. Path Planning
- 3) Ideas for further improvement

Original Concept of Operations:

Objective:

To design, develop, and integrate an AI agent capable of autonomous operation on low-power hardware, such as Nvidia Jetson, for use in physical drone teams. The autonomous operation will include identifying victims from drone swarm cameras using computer vision and controlling the drone swarm flight paths with waypoints sent through Mission Planner.

The system will be applied in real-world scenarios like search and rescue and disaster response missions, where drones can assist in locating survivors and providing real-time data.

Mission Overview:

The AI agent will be deployed on a swarm of drones equipped with sensors like IR/EO cameras and LiDAR. These drones will autonomously navigate hazardous environments, collect critical data, and provide it to rescue teams in real time.

The AI system will operate optimized for low power consumption to ensure extended mission durations. The drones are designed for SAR (Search & Rescue) missions, to search a specified search area, using cameras and enhanced computer vision analysis to identify victims within sight autonomously. Upon a potential find, the drones alert humans at a Ground Station or mobile control device.

Key System Components:

AI Agent:

The AI agent in the system's core, responsible for analyzing sensor data and making real-time decisions for autonomous navigation and search operations. A large language model will be used to accomplish a majority or all of these functions. Our sponsor has specified we should use an open-source model, so our current model of choice is Llama 3.1 from Meta, and

we will use LlamaOS as a testbed to set up AI agents and different versions of the Llama 3.1 model.

It will process inputs from sensors creating points of interest and adjusting flight paths based on these environmental factors, enabling the drone to carry out complex tasks without direct human control. The AI Agent will communicate directly with the drone through MissionPlanner and adjust flight paths by sending waypoint coordinates from the agent output through MissionPlanner to the drone.

Low-Power Hardware/Sensors

Hardware platforms like Nvidia Jetson will allow the use of a large language model for the AI agent to perform complex tasks like real-time data analysis and autonomous navigation while consuming minimal energy. The Jetson will incorporate sensors like IR(Infrared), EO (electro-optical), LiDAR (light detection and ranging), and GPS (global positioning system). IR will be used for detecting heat signatures and visual insights and the LiDAR will generate real-time terrain maps and detecting obstacles.

Our sponsor has an NVIDIA Orin for the hardware teams and our team to test on, allowing us to test higher power versions of the agent, and at a minimum our agents need to perform well on an Orin attached to a ground station connected to the low-power hardware drones, however we hope to have the agent run on-board the drones with cheaper, less powerful GPUs if possible.

Key Phases of Operation:

Mission Planning, Deployment and Initialization

The mission begins with human operators defining high-level objectives, such as the area to search or specific targets to locate (e.g., missing people, infrastructure). These parameters will be inputted through Mission Planner –

ArduPilot. Upon deployment, the AI agent establishes initial communications between drones for coordinated operations such as syncing of sensor data and positions. After being given a boundary to conduct the search the agent will deploy the drones. Initially the drone will fly up to a high altitude to get an overview of the terrain that is being searched. Depending on the size of the search area this drone may have to move to several locations at that altitude to get imagery of the entire area. While this data is being collected the agent will be analyzing images to determine irregularities from the natural terrain.

Autonomous Operation

The AI agent will command each drone to follow a path-planning algorithm based on data gathered from the sensors and initial parameters. The drones will be capable of avoiding obstacles, mapping terrain, flagging points of interest, and detecting heat signatures/object recognition. The drones will continuously share data amongst each other and with the rescue team. The agent should dynamically adapt to the environment when new data arises, reducing human oversight.

The drones will also be able to readjust their search paths based on the input from the ground-based or onboard AI Agent. The inputs will be coordinate-based waypoints input through the MissionFinder software. For example, if one drone identifies a potential area of interest (e.g., a heat signature, structural anomaly), then the AI Agent will send new waypoints to the entire drone swarm, and instructed drones nearby would adjust their routes to gather onto that location.

The AI agent will process the data locally which in turn allows for immediate insight. User Interface There are two potential set-ups for the user: A mobile interface to interact with drones that possess on-board AI

Agent computing, or a stationary laptop Ground Station that contains the AI Agent computing.

A user will use a UI that initially includes activation of the drone swarm. The activation process has the user provide coordinate-based boundaries of a search perimeter, prompt the AI what it is searching for, select a return-on-find or land-on find exit condition, and select which drones and the number of drones that will be used.

Upon launch, an operations UI will launch. During an operation, the user will see a dynamic map displaying current drone locations, projected pathing of the individual drones and points of interest as they are detected by the drones. While the agent will determine its own perception of an optimal order to further investigate POI's a user will be able to select a marked POI on the map and request to prioritize its investigation.

The user will be able to manually view on-board cameras from the drones, and switch between camera views. The user can also, depending on the success of the SOAR and SPARROW interdisciplinary teams, toggle LIDAR, Thermal imaging, and other sensors developed by those teams. Should the thermal imaging be successfully integrated, a stretch-goal is to include an AI Agent trained in computer vision of thermal imaging to identify targets.

The user can view drone power levels and see if the drones are online or offline. The AI Agent will automatically flag issues and identification to users, using a "human-on-the-loop" model. If the AI encounters obstacles to its flight, believes it has identified a target, or for any other reason that requires human intervention, the drone will pause in space and await a confirmation from the human operator to proceed with an exit condition or to change behavior.

Failure Conditions

If all POIs are investigated, and no positive identifications are made from flags sent to human operators, the drones will do a low-level sweep of the entire search area which may possibly discover more than previously possible with the high-altitude scan. If no target is found, during this final low-level sweep, the agent will command the drones to return to their starting positions and using data from the search provide a likely direction the target may have moved and suggest a new search perimeter for human operator approval.

Success Conditions If a POI is investigated and a positive identification is made on the target from a human confirmation of real-time footage, the drones can have exit operations. The agent will operate depending on how the agent is configured by a human operator. The first configuration would simply mark the coordinates of the target on the map and enter an exit sequence where all drones will return home, and the agent will deactivate.

The second configuration would mark the location of the target and command all drones to that location. From here the agent would span the drones out from the target and identify possible entry points for a rescue team. These could be in the form of a nearby road, large clearing accessible by helicopter, or a path leading into the search perimeter. From here the drones would scan for possible hazards or impassable terrain.

It should utilize the drone's onboard LIDAR sensor to determine the grade of the terrain. The agent will use this data to determine an optimal path for a rescue team to traverse from each possible entry point to the target. Once the scanning has concluded and the paths have been generated the agent will initiate the exit condition and the drones will return home. We intend to achieve this stretch-goal of identifying routes to the target by incorporating outside research on SAR Drone Path Scanning, such as the work done by Jan-Hendrick Ewers on "Deep Reinforcement Learning for

Time-Critical Wilderness Search And Rescue Using Drones”, who we are already in contact with for questions and potential collaboration.

Following a search operation the program will generate a mission report with a log of all the points of interest discovered, what order they were investigated, as well as what type of POI the target was found in. The mission log will also include drone flight paths, all AI decisions with timestamps, and mission time, location, and length. This data will be saved in a readable format for After-Action analysis by the drone users and has a dual-use for improving the drone swarm AI through better tuning for the agent to find the target in a timelier manner.

Emergency Procedures

Low battery, Weather, Loss of Signal, Collisions, Sensor Failures, Overheating.

Should hazardous conditions arise such that the drone would be put in great peril, the AI agent will take measures to prevent catastrophic damage. If the AI agent detects that an individual drone’s battery would not be enough to return to its initial launch destination if the drone was to continue the search, the AI agent will instruct that drone to return back to the launch destination.

To maximize the chances of a safe return, the AI agent will disable any unnecessary sensors. The AI agent will continue its current search with all other available drones until a majority are no longer able to continue. In that event, all drones will return to the launch destination. Depending on the severity of the weather, the AI agent can take 1 of two measures. In mild weather conditions, the AI agent will instruct the drones to shut off all drone sensors and instruct them to return to the launch destination as quickly as possible.

In severe weather conditions, the AI agent will instruct the drones to land as soon as the drone detects a viable landing location. The AI agent will turn off the drones' sensors, and leave GPS enabled so that the drones can be retrieved by human personnel later. In the event of a drone losing signal, the AI agent will try and use other drones to locate the missing drone.

If the AI agent is not able to locate the drone this way, it will alert human operators to the missing drone's last known location and keep this alert until the drone reestablishes connection. If a drone collides with an object, or another drone, the AI agent will instruct the involved drones to land immediately. The AI agent will then verify that the drone is still functional and run a short flight test to ensure it can still fly effectively.

Should the drone fail this flight test, the AI agent will alert human operators to the downed drone's location. Depending on which sensors have failed, the AI will decide whether to continue the search. If a crucial sensor is found to be damaged, the AI agent will direct the drone to return back to the launch location. In situations that could cause the drone to overheat, such as searching in an area with an active fire, the AI agent will instruct the drones to retreat to the launch location.

If the drone's temperature drops to a safe level during this retreat, the AI agent will instruct the drone to return to the search. Should a human operator deem it necessary, they can instruct the AI agent to ignore emergency procedures. For example, if a drone does not have enough battery to return but has spotted the rescue target, the operator can instruct the AI agent to ignore standard protocols. The AI agent will follow its standard search procedures and permit a drone to fail if it can no longer operate.

Diego

Research

Motors

Brushless DC Motor

A brushless DC motor is a motor that does not rely on brushes to generate torque, but rather an electronic controller used to switch currents to produce a magnetic field. These types of motors carry benefits over brushed motors, namely the higher efficiency and lower wear on components. Brushless motors are able to generate precise speeds, making them incredibly useful when considering maneuverability and stability for high-altitude operations. These Brushless motors are typically more expensive than brushed motors, and more complex, but the cost is negligible. Brushless motors are a common use for RC quadcopter operations due to these factors,

Coreless Motors

To Do

Hardware

Drone Parts

In order to facilitate the testing of our search and rescue AI operations, we must first have a drone to test on. Lockheed Martin has graciously given us access to a couple of parts, namely:

1. NVIDIA Jetson AGX Orin Developer Kit
2. Teledyne FLIR Boson Thermal Camera module

Motors

While we still do not have access to the SPARROW/SOAR student developed drone swarms, we still have access to a previously designed student-built drone. The motor is the main component of the quadcopter that allows for flight. Given that our drone is to be surveilling a large patch of terrain from high up in the sky, we need to make sure that our motors are powerful.

There exists an important trade-off between power, stability, size, and efficiency.

- We want small enough motors to be able to fit on our drone unencumbered, not weighing it down too much.
- We want a motor that remains stable in flight, with enough reactivity to stabilize itself when winds or other challenge terrain move the drone.
- We want a motor that is able to generate enough thrust to move the drone to high altitudes and stay there.
- We want a motor that isn't too energy inefficient. We want to have long flights, and if our battery runs out too fast, we will have to spend a long time swapping out batteries.

The current drone we are using utilizes the RS2212 920KV Brushless Motor; DC 7-12V, 53 grams, and only 22x12 mm. These motors have worked very well, and were relatively cheap (retailing at about \$50 for a set of 4). We have yet to hear about the motors that the SPARROW and SOAR teams are using, but a successful flight is contingent on finding a good balance between all of these factors.

Drone Chassis

When choosing a drone chassis, we want to consider what type of operation we are conducting when deciding on weight. If weight is no issue, we can get a large chassis made of durable material (metal or some composite). We can load up the drone with as much equipment and still be fine. In our case we want to conduct search and rescue operations, so speed and finesse are key. We want to be able to move fast while also supporting vital systems used for identification. The chassis we are currently using on our test drone is comprised of carbon fiber landing gear, a lightweight Polyamide-Nylon composite, reinforced with a carbon-fiber rod. The bottom plate of the drone

comes with a built in Power Distribution Board (PDB). The entire frame weighs in at about 800 grams, making it both lightweight and durable. The drone is 390x390x240mm so it can remain small while also being able to fit a camera and sensors underneath and an embedded processor on top.

Battery

Our battery needs to be large enough to support extended flights at high altitudes while also being small enough to fit on our drone and light enough to not weigh it down too much. Our battery is the HRB 6s 5000mah 22.2v 50c-100c LiPo battery. This battery has good output but the lifetime of the battery is not suitable for long flights. The battery time is typically about 40 minutes, making it suitable for short range flights. Our purposes call for long flights extending over miles, so we want to find a battery that lasts for about 2-3 hours rather than 40 minutes. A 30000 or even 50000mAh battery would be more sufficient for our purposes, although these can get quite bulky and heavy.

Propellers

To Do

GPS

[GPS module](#)

For successful localization and mapping, we need to be able to keep track of where our drone is located in space. Our testing drone is using an M8N GPS / WiFi Telemetry module to interface with the PixHawk 4. Our module is accurate to about 0.9 meters, which is crucial for search and rescue ops where location is crucial. These GPS modules are also low power making it good for long flights. This GPS is only compatible with PixHawk and APM flight controllers however, so custom firmware may be challenging to implement with this specific type of GPS.

Altimeter

To Do

Transmitter

For our test drone we have decided to go with the Taranis Q X7 brand of transmitters. Ultimately we plan to have our drone be autonomous, where flight movement is completely controlled by the onboard edge AI.

[Transmitter - Taranis Q X7](#)

This specific transmitter operates on 16 channels utilizing an ACCESS protocol. Very low latency,

Receiver

The receiver used for the drone determines how we transmit data from the transmitter to the drone. When we want to remotely send a command to the drone from the ground, we need to send individual inputs on separate channels to control what part on the drone does what. We want a receiver that supports multiple channels due to the large number of sensors we will have on board. This might not be necessary as part of our final design due to the integrated nature of the AI agent that controls the drone movement.

GoPro + Gimble

Autopilot

For the drone to stay in the sky, it needs to control the speed of it's motors, as well as stabilize itself and keep itself upright. If we were to individually adjust the speed of each motor we would quickly crash. The autopilot allows the computer to control most of the flight systems autonomously to keep the drone stable. When we send a command from the transmitter to the receiver, it tells the autopilot to adjust the speed of the motors, bringing it

up, down, left, or right, without the human operator having to manually adjust each motor. Our testing drone utilizes the PixHawk 4, a standard autopilot with a lot of modularity. The PixHawk allows us to control sensor and motor data, allowing for smooth and easy flight while keeping the drone stable.

Swarm Movement

A big part of a search and rescue operation is the search. No matter how sophisticated the drone is, if we don't look in the right places at the right times, we will never find our target. As part of our research, we want to develop efficient movement algorithms for swarm-specific movement, utilizing the large area coverage that comes with multiple drones. One drone is able to cover a large area effectively, especially from high in the sky, it can speed up coordinated search and rescue efforts. Multiple drones can almost 50x the search effort, with each drone having a part in the overall coordinated effort.

Search and Rescue Movement

In order to understand efficient drone search and rescue missions, we need to understand how search and rescue works. In a typical search and rescue (SAR) operation, our efficiency is measured by three key factors: Speed, Coverage, and coordination.

- In a typical SAR, speed is key. We want to find our target as soon as possible; the sooner we locate the target, the faster we get them away from danger, the better their chances are of survival are. Time-sensitive scenarios, such as lost hikers or blizzard survivors, demand immediate action.
- Coverage is also an important factor to consider. If we make our scope too small / limited, we risk not searching in the right areas. A hiker

missing on one side of a mountain will never be found if we don't look in the mountain range, but rather a field. If we make our scope too large we risk wasting too much time and resources, as well as accidentally skipping over the hiker. If we search the entire state park, we will never find a hiker who is lost near the entrance.

- Coordination is also a big problem in search and rescue operations. Being able to round up searchers and EMT's is a challenge, and coordinating where each team is supposed to go is an even bigger challenge. Some groups may get lost, veering off path and potentially missing the target. Some groups may fall behind, increasing the time taken to locate the target and wasting resources. Efficient coordination is key when trying to find a target in a time sensitive scenario.

The Role of Drones in Speeding Up SAR Operations

Drones are increasingly being utilized in SAR missions because of their ability to operate in conditions where human search teams are either too slow or put at risk. Drones can:

- Reach inaccessible areas quickly: In rugged or hazardous terrain where rescue teams may take hours or even days to reach, drones can be deployed almost immediately, reducing the time it takes to locate the individual.
- Work day and night: Equipped with infrared cameras and other sensors, drones can continue the search during nighttime or in low-visibility conditions, ensuring that the search doesn't lose momentum.
- Provide rapid situational awareness: Drones equipped with high-resolution cameras and sensors provide real-time data to rescue teams, enabling them to adjust their strategies immediately based on

new information. This adaptability improves the speed at which rescuers can locate and assist victims.

The use of swarm movements can dramatically increase efficiency in search by allowing multiple drones to work in parallel, each covering different sections of the search area. Unlike a single drone, which has to scan a large area by itself, a swarm can split up, with each drone assigned to a different point of interest. This division of labor enables the swarm to cover more ground in less time, improving the overall speed of the search operation. Additionally, swarm algorithms ensure that the drones can communicate and adjust their flight paths dynamically, avoiding overlap and ensuring no area is left unchecked.

Another key advantage of swarm movements is the ability for individual drones to investigate points of interest independently without holding back the entire swarm. When one drone detects something (a backpack, car, clothes, tracks, person) it can break off from the main group to investigate, while the rest of the swarm continues on. This flexibility prevents the entire swarm from being delayed by false positives or distractions, ensuring that the search remains efficient and focused. By utilizing both coordinated movement patterns and independent action, swarm drones can speed up SAR missions, increasing the chances of a successful rescue.

Probability Heat Maps

To create an efficient search path in a rescue mission, it's important to use as much information as possible before we even begin the search. The more data we gather about the area, the conditions, and even how people typically behave in that environment, the better our search will be. By studying this data beforehand, we can make smarter decisions about where to look first, instead of just randomly searching everywhere. For example, if

we know a person is likely to follow a certain trail or avoid a steep area, we can focus our efforts there, saving time and resources.

One powerful tool we can use in this planning stage is a **probabilistic heat map**. This map shows us the areas where the missing person is most likely to be based on previous search and rescue operations, terrain features, and patterns of human behavior. For instance, people often move toward lower elevations, stick to open spaces, or follow paths when they're lost. By using these patterns, we can prioritize certain areas on the map, giving us a clearer picture of where to search first. This approach is much faster and more efficient than just dividing the area into a grid and searching blindly.

By focusing on the areas where the person is most likely to be, we can make the search up to 20 times faster. Instead of wasting time searching low-probability areas, we can direct drones and rescue teams to the spots where they're most likely to find the missing person. This not only saves valuable time but also increases the chances of a successful rescue, especially in time-sensitive situations where every minute counts.

We can get valuable data to create these probabilistic maps by collaborating with local authorities, state parks, and even fitness apps like Strava, Nike Run Club, or Google Maps. These sources provide insights into where people tend to walk, such as popular hiking trails, paths near rivers, or landmarks. By analyzing this data, we can prioritize where to send our drones during a search, focusing on areas where the missing person is most likely to be. This collaboration helps us make the search more efficient, covering high-probability areas first and increasing the chances of a quicker rescue.



Commented [DA1]: Strava Heatmap

Figure 1: Strave-Generated Heat Map

Here we can see an example of a Strave generated heat map. A Search and Rescue volunteer David Webster utilizes these maps to concentrate search efforts along the most populated routes.

Pathing

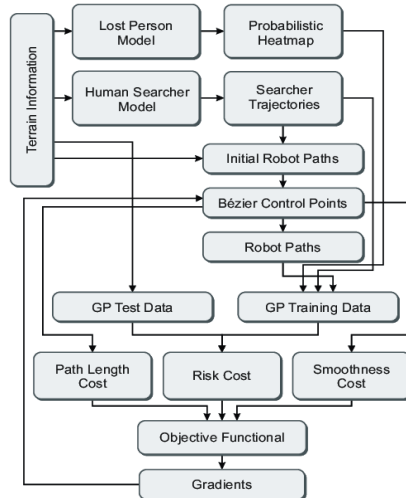


Figure 2: Generation Flow for Drone Swarm Search Map

Here we can see a generalized flow of planning for creating a drone swarm search map

[https://www.researchgate.net/publication/344180309_Anticipatory_Human-Robot_Path_Planning_for_Search_and_Rescue]

Using a generalized lost persons model, we can predict where a lost person is most likely to wander based on landmarks and terrain features like trails, rivers, or open areas. This data helps us create a probabilistic heatmap that highlights the areas where the person is most likely located. By combining this with a human searcher model, which estimates the most efficient paths a human rescuer would take, we can design a series of control points for the drone swarm to follow. These control points guide the swarm to cover the most likely areas first, maximizing search efficiency.

As drones gather real-time information by scanning the terrain, we can update and adjust the search grid dynamically. This process, known as recalibration, allows the drones to shift their focus as new data comes in, ensuring the search is always adapting to the latest findings. This dynamic approach speeds up the search and increases the chances of finding the target quickly and efficiently, even as conditions change.

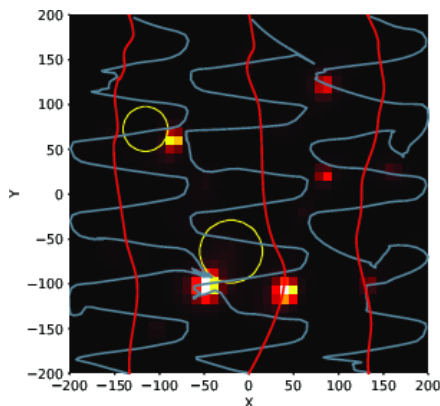


Figure 3: Proposed Separated Drones Search Grid

A finished search grid for our drone swarm would look like the image above, divided into sections that reflect the most likely areas for the missing person to be found based on our probabilistic heatmap. Each section is assigned to specific drones in the swarm, ensuring comprehensive coverage while minimizing overlap. This grid continuously adjusts as drones gather new information, enabling the search to stay flexible and responsive to real-time data.

This algorithm, which combines the lost persons model, human searcher model, and dynamic recalibration, has been shown to improve search and rescue times by nearly 160%. This is a substantial improvement, allowing rescue teams to locate missing individuals much faster and more efficiently than traditional methods. The success of this system demonstrates the potential of drone swarms to revolutionize search and rescue operations.

The blue path is the human searchers, while the overhead drone pathing is shown in red.

Mavlink Encoding

MAVLink, short for Micro Air Vehicle Link, is a communication protocol widely used for interactions between unmanned vehicles and ground control stations. Its lightweight design makes it ideal for efficiently transmitting coordinates, altitude data, and commands to the PX4 autopilot. Given its versatility and reliability, MAVLink will serve as the primary means of controlling our drone swarm.

How MavLink Works

MavLink is able to tell the drone to change flight modes (Stabilize, go standby, begin landing), send sensor data, update paths, and start/stop different preplanned missions. It can do this by encoding each command into

a series of bytes arranged in a packet. This byte stream is then picked up by the receiver, streams the bytes to the hardware interface on the PX4, and the autopilot then decodes the message into coordinates to move to.

MavLink Packet = (6 byte header + 9 byte payload + 2 byte checksum)

Header

The header is comprised of the **System ID, Component ID, and Message ID**. The system ID helps to keep track of which system is sending the message, the component ID keeps track of what component of the system or subsystem is sending the message, and the message ID holds information on what type of message is being sent. This ID can include a heartbeat, command, set mode, or any type of command. Along with these is the **message length** and the **sequence number**, both are vital in ensuring that the drone receives the right command in the right order. The component ID is especially important as it is used to differentiate copters in a swarm.

Payload

The payload is used to communicate the specifics of the command specified in the message ID. If we want to move the drone, we need to specify where we want the drone to move to. The payload can be anywhere between 0 and 255 bytes.

PX4 Interface

The PX4 is an autopilot used to fly the drone without any human input. It utilizes onboard flight systems and telemetry tools to keep itself upright and automatically generate flight plans when given coordinates.

The PX4 begins by receiving a packet from either the ground station or from an onboard computer. The PX4 has several serial ports with which to interface with: UART ports to interface with radio and ground stations and USB ports to directly connect to onboard systems. It is important to note that USB ports take priority over UART ports on the PX4, so an onboard computer will always be faster than sending/receiving signals from a ground station.

The PX4 utilizes MAVLink encoding to receive messages. Once received and validated, the payload is extracted from the packet and automatically processed for either coordinates or commands (e.g. start, stop, auto, take picture). These commands are put into a queue and then executed serially. The autopilot will then automatically calculate the heading, rotation, motor speeds, etc... to bring the drone to the desired point in space relative to the drone. The autopilot will continuously check for any deviations and course-correct while maintaining it's path.

Generating Dynamic Movement

The most important part of an autonomous drone swarm is it's ability to be autonomous. This means that instead of predetermined sweeping patterns we need to be able to generate new coordinates in the case that our drones see anything of interest. If we were to only use ArduPilot MissionPlanner we would have to have a human create new routing points for each drone. Not only would this be laborious, this would also completely negate the point of an autonomous drone swarm. Instead, we can have an AI agent automatically generate coordinates.

DroneKit API

When our AI agent creates new coordinates we need to have a method of communicating with the drone. DroneKit is a python API created specifically for interfacing with the ArduPilot flight controller (It just so happens that our PX4 is compatible) using a “Low-Latency link”. This is an open-source project that provides a good jumping off point for our project. This API allows us to “programmatically access a vehicle’s telemetry, state, vehicle movement, and operations.” The GitHub page can be found here:

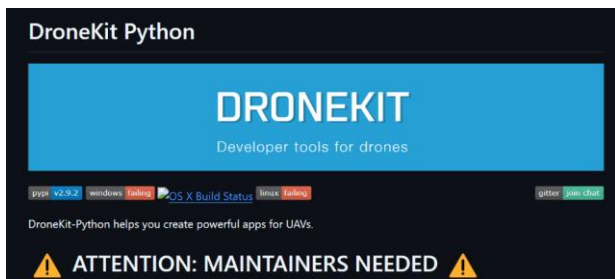


Figure 4: DroneKit Python <https://github.com/dronekit/dronekit-python>

The DroneKit API enables seamless communication with the various AI systems integrated into our project. When our LLM generates a set of coordinates, these serve as the central point for our drone’s search operations. Rather than having the drones navigate directly to the specified point, we can apply pathfinding and search algorithms to optimize the search area. Using Python, we can define a dynamic "search area" around the central coordinates and generate a path of waypoints within this region. These waypoints, created via the DroneKit API, allow the drones to efficiently explore the area relative to the designated center.

Swarm Movement

In order to facilitate Search-and-Rescue operations we need to define how our drones are going to be moving in our environment. To be effective we need to keep a list of requirements in mind while we create an algorithm:

- Our algorithm needs to be scalable; The algorithm must handle a wide range of search areas, from compact regions like a single square mile to expansive zones covering thousands of square miles. It should adapt seamlessly to the scope and scale of the operation.
- Our algorithm needs to be efficient; Not all patterns yield the same results. We need to strike a balance between speed and search, optimizing our coverage while minimizing the risk of missing our target. Time is crucial, but it should not come at the cost of missing our target completely.
- Our algorithm needs to be dynamic; It should be able to handle any kind of terrain. Our approach must adapt to any type of environment in order to remain practical in a real-world scenario.

General Movement Principles

Our drone swarm's movement strategy emphasizes cohesion and coordination, treating the swarm as a single entity with individual drones operating small tasks within the larger group. The idea is to ensure efficient area coverage while maintaining structured, collaborative motion. This approach minimizes gaps in coverage while leveraging the individual capabilities of each drone.

Individual Movement Principles

Each drone operates autonomously within the swarm, performing specific tasks such as scanning, mapping, or communication. Despite this independence, every drone will still follow the swarm's overall movement strategy. This distributed workload ensures efficient data collection and processing.

- **Scanning Tasks:** Certain drones will be fitted with specialized sensors, such as LiDAR, thermal imaging, or video cameras. These drone's will be monitoring specific sub-areas throughout the swarm's path.
- **Data Sharing:** Drones communicate findings in real-time, enabling the swarm to make informed, collective decisions.
- **Obstacle Avoidance:** Each drone independently navigates around obstacles while maintaining its relative position in the swarm.

Developing Path Finding Algorithms

Our drone swarm is not going to have a set environment it will be operating in. Urban, suburban, and rural areas are all equal candidates for deployment, and as such, we need to have an algorithm that will fit to it's environment. The challenges we see in an urban environment may not necesarilly be the same as a suburban or rural environment; Dense trees may not be an issue in a city.

Shrinking POMCP – Probabilistic modeling [1]

Markov Decision Processes (MDPs) provide a robust framework for modeling decision-making under uncertainty, utilizing a probabilistic state diagram to predict outcomes. A more generalized and practical extension of MDPs is the Partially Observable Markov Decision Process (POMDP), which accounts for scenarios where the decision-maker lacks complete visibility of all states or actions.

In the context of UAV path planning, this approach enables drones to make decisions based on partial observations of their environment, approximating optimal actions through probabilistic inference. The POMDP model of searching offers a significant improvement in areas with high uncertainty, such as search and rescue, where data may not always be reliable or non-existent. In the paper by Zhang et al., by using a Monte Carlo simulation (random sampling to produce a probability distribution) we

are able to best pick where to move to next, rather than a fixed non-dynamic search pattern.

An MDP is essentially a state diagram where actions are predicated by probabilities. These are important when our environment is unknown. In our case, we utilize the POMCP to guide the drone towards the most information gain, moving gradually towards the next best region which maximizes information gain.

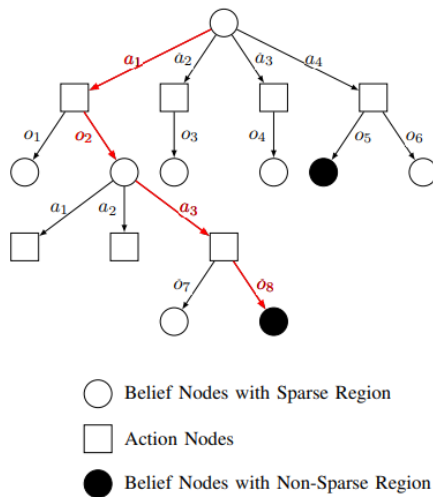


Figure 5: Decision Tree

In the figure we see the decision tree that dictates swarm movement. The arrows show the action sequence decided on by the agent, and each square indicates an action taken by the agent. The circles are belief states, with the black circles being areas with the most information. We want to choose actions that lead us towards more black circles rather than white circles, as the probability that the target is in that area is higher than the white circles.

By incorporating techniques like Shrinking POMCP, which efficiently narrows down the decision space, UAVs can balance computational cost and decision quality, resulting in smarter, more adaptive operations.

Reinforcement Learning with A Priori Knowledge [2]

Deep reinforcement learning (DRL) introduces a transformative approach to UAV path planning by integrating a priori data, such as probability maps of the search area, to optimize flight paths and minimize search time. This method not only dynamically adjusts search strategies in real-time but also significantly outperforms traditional search patterns. For example, the proposed approach demonstrated a 160% performance improvement over common algorithms like the lawnmower search pattern and up to 250% in other optimized search patterns. By continuously refining its trajectory using real-time observations and probability data, DRL enables UAVs to efficiently cover search areas, reducing operational time and improving success rates in critical missions.

In David Anderson's paper "Deep Reinforcement Learning for Time-Critical Wilderness Search And Rescue Using Drones", he highlights a problem with certain optimization algorithms (namely LFC_GW_CONV – Local hill climb, global warming, convolution). Areas of high probability may not be considered until current operations are finished in the current cell. Deep reinforcement learning is able to sidestep these issues by prioritizing long-term search over short-term rewards. By training a model to recognize optimal paths that may be low-information in the short term, but highly rewarding in the long term, we can see massive improvements in search times.

Optimal Path with Dynamic Programming [3]

Dynamic programming offers an efficient solution for UAV path planning, producing near-optimal flight paths in polynomial time. This method is particularly effective for connecting pre-determined points of interest, making it ideal for structured missions such as post-disaster surveys or infrastructure inspections. By breaking the search space into manageable sub-problems, dynamic programming enables UAVs to compute paths that are computationally feasible while retaining high-quality results. This approach complements probabilistic and reinforcement learning methods, providing a hybrid framework for UAV operations that balances precision with scalability.

This type of dynamic programming approach can also be useful when generating sample training data for training an algorithm. The paper by Wang et al. Outlines an algorithm for efficient coverage of Points of Interest (POI) with a certain quality. Quality is defined as an efficiently observed object, sufficiently in the range of the camera, and in focus. This constraint adds another layer of complexity that traditional path algorithms are not equipped to deal with. This algorithm considers both constraints, efficiency and quality, to create a path.

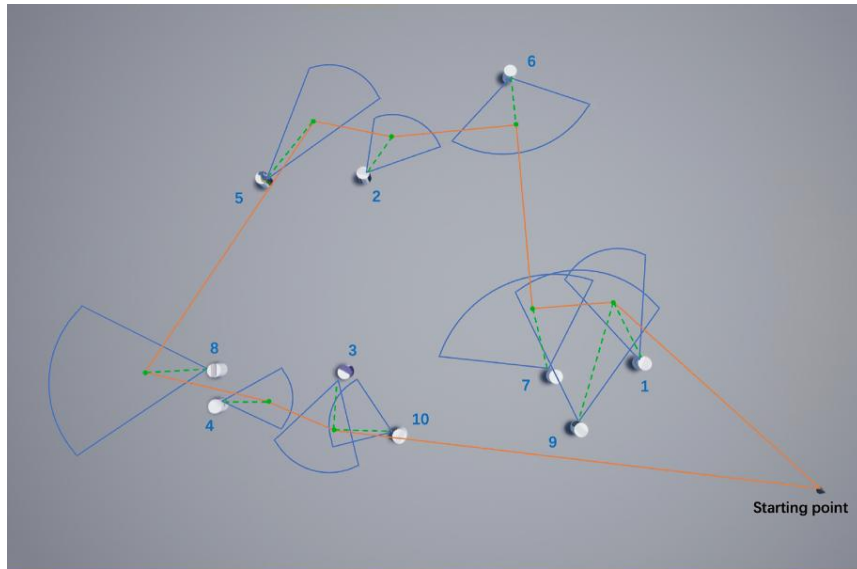


Figure 6: Efficient Pathing

Here, a path in green is calculated for maximum efficiency. The blue cones represent the area where the drone can take a good quality photo of the numbered object (NOTE: The numbers do not correspond with order, they are simply there to categorize the objects). The flight path is able to fall within each objects quality range in an efficient manner.

In the case of search and rescue, we not only want to search as much area as possible, but we need to be able to search in a way that allows us to get as much coverage as possible. If we are searching in an area with dense trees, planning a route with sufficient coverage is more important than simply flying over the trees and not being able to see anything but cover much more ground. In combination with an LLM generating coordinates, this algorithm could prove to be extremely useful.

Active Search Strategy with Multiple UAV's and Targets [5]

Creating an efficient algorithm for multiple drones is just as important as planning the general path the swarm will take. With each drone acting as an individual searcher in a large group, we are able to expand the area the swarm can cover and speed up our search. In order to realize these efficiencies, we must optimize our algorithm to include multiple searching agents.

Using both local and global information gathered from the drone and the rest of the swarm is crucial when making a decision on what to search next. A proper partitioning technique is crucial to reduce overlap in search, so a Voronoi partitioning algorithm is used to divide up the search area.

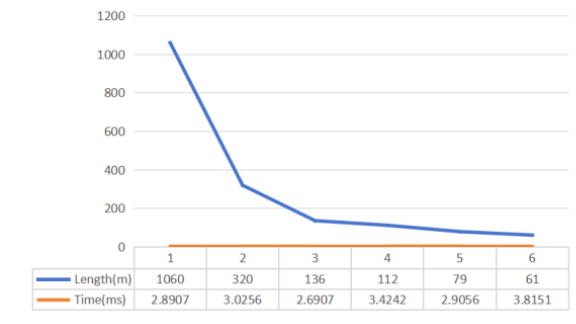


Fig. 6. Scalability analysis about the different number of UAVs. The blue line represents the maximum path length in UAVs. The yellow line represents the computation time.

Figure 7: Scalability

Increasing the amount of drones in the swarm also decreases the path length per drone with only a small amount of compute time added. Scaling up the amount of drones used per search can lead to exponentially faster rescue times. Implementing a good divide-and-conquer strategy is crucial.

Task Allocation in Unkown Environments [6]

Task allocation is a critical consideration when designing pathfinding algorithms for drone swarms, especially in unknown environments. To fully utilize the capabilities of a swarm, it's essential to ensure each drone is effectively utilized to maximize data collection and mission efficiency. Effective task allocation not only enhances individual drone productivity but also ensures the swarm operates towards a shared objective.

In a paper by Grace Cai, Noble Harasha, Nancy Lynch, "A Comparison of New Swarm Task Allocation Algorithms in Unknown Environments with Varying Task Density", they propose a "pheromone" based information transfer. Like how ants leave pheromones to lead other ants to food, drones could leave behind or transmit a signal to the rest of the swarm if any points of interest have been found.

This task allocation also helps to distribute demand efficiently. Areas marked to be important will draw more drones towards it, effectively creating a priority queue for the drones to navigate. The more attention a spot receives, the faster it gets searched, the more information we gain.

Optimizing Connectivity Through Multi Q-Learning [4]

To ensure reliable communication between the ground station and the drone swarm, it's crucial to establish a strong and consistent connection. Challenging terrains, such as mountainous regions, pose obstacles not only for navigation but also for maintaining connectivity. Satellite channels, while an option, are costly and suffer from latency, and many remote areas lack any coverage altogether.

This approach involves training a model to prioritize paths that maintain uninterrupted connectivity between the drones and the ground station. By rewarding routes that preserve communication and penalizing crashes or disconnections, we can develop a robust pathfinding agent capable of handling difficult environments. Additionally, training across

multiple frequencies enhances the model's adaptability to varying operational conditions, ensuring reliable performance even in complex scenarios.

Building a Path Planning Framework

By integrating elements from various advanced algorithms, a comprehensive drone swarm path planning framework can be developed. This model would aim to efficiently combine real-time adaptability, decentralized coordination, and optimal coverage strategies to meet the demands of dynamic and unpredictable environments.

Dynamic Task Allocation - This approach allows drones to prioritize areas of interest, minimize redundancy, and respond effectively to changing conditions without requiring centralized coordination.

Global and Local Information Integration - Merging global and local data enhances the swarm's ability to make informed decisions. A coarse global map provides initial guidance, while real-time local observations refine navigation and task allocation.

Markov Decision Processes (MDP) and Reinforcement Learning - Leveraging MDPs and reinforcement learning techniques enables drones to make optimal decisions under uncertainty. By formulating the problem as an MDP, drones can evaluate current states and predict future outcomes, continuously improving their paths and task allocation through feedback loops.

Coverage Path Planning (CPP) - Incorporating elements of CPP ensures thorough exploration of the designated area. Advanced adaptations, such as dynamic programming or mutual information metrics, improve traditional CPP by focusing on high-priority zones and gathering relevant data.

Voronoi Partitioning for Decentralized Coordination - To manage multi-UAV operations, Voronoi partitioning divides the environment into non-overlapping regions, assigning each drone a specific area to minimize redundancy. This technique enhances swarm efficiency while maintaining robust coverage and safety.

Robust Connectivity - Ensuring uninterrupted communication between drones and the ground station is vital. Implementing strategies like multi-Q learning helps drones prioritize paths that maintain connectivity while adapting to terrain-induced challenges, such as mountainous regions or areas with limited satellite coverage.

Object Detection and Data Collection – Ideally, this stage would be efficiently covered by the other VITALs group, but the data passed on from the image processing stage can be used. High-quality data collection ensures the swarm gathers actionable insights to support decision-making and mission goals.

James

Swarm Movement Introduction

For our project, we are tasked with utilizing and enabling an LLM to successfully command a Drone Swarm to maneuver synchronously in real-time based off of Computer Vision results and text input from a user. This is a complex system of Moving parts, so to speak, that requires the use of existing Drone Movement knowledge to the best of our ability.

For both simplicity, and more importantly, system efficiency and success at locating victims, we should default Drone Swarm Movement to optimal search algorithms and Movement patterns as understood by the Drone Swarm community. The LLM we create will work with existing algorithms in order to maximize search efficiency, utilizing the natural

advantages of the drone's high ground, the breadth of camera angles and area coverage a drone swarm provides, and automatic Computer Vision-based image/video scanning in order to make Search-and-Rescue efforts quicker and safer, potentially saving lives.

As a core tenet of the MVP agreed upon by Joe Rivera from Lockheed Martin, our sponsor, we must have the LLM direct this Movement. Therefore, we must train the LLM to identify under which conditions different search algorithms should be used, and correctly implement a function for the LLM to command the drones to follow that Movement by inputting a series of coordinates in the MissionPlanner software interface.

This identifies 4 main steps in implementing Swarm Movement in Search and Rescue: First, Identifying optimal pre-determined Search algorithms and patterns. Second, Training the LLM using Reinforcement Learning to respond with the correct search algorithm based upon different user prompts. This entails rewarding the system when it responds with a correct search path, and punishing it when it doesn't, to skew the results towards the most efficient search patterns. Third, learning how to allow objects flagged by Computer Vision to be input into the Swarm Movement LLM and place objective coordinates in MissionPlanner accordingly. Fourth, researching how to most rapidly send coordinates based upon the search pattern from the Ground Station LLM with minimal lag to MissionPlanner and then the Drone Swarm.

The added complexity for the Search and Rescue use case comes from having to communicate with several real endpoint devices transmitting real-time data, transmitting and receiving real-time locator signals like GPS, and processing initial computer vision-scanned images on the drone to flag to the more powerful Ground Station. The time element is not insignificant either, as human lives are often endangered when Search-and-Rescue

operations are being conducted. An efficient and timely search pattern, effective drone hardware like cameras and processors, as well as a precise Ground Station that can confirm flagged objects and accurately use new data to help the search.

The movement of swarms in existing research

The Movement of Swarms in technical fields can be referred to as Swarm Intelligence, the collective operation of many simpler agents to work together and achieve a larger objective, such as Group or multiple-agent specialized Movement, engaging an objective, or releasing many payloads.

In this case, our application of Swarm Intelligence as directed by an LLM, requires a comprehensive array of modern Swarm search-and-detect algorithms. Swarm Intelligence has a basis in simple Artificial Intelligence like Decision Trees in the world of Video Games with early games such as Starcraft or World of Warcraft using many simple computer agents to achieve tasks within game simulations. Now the modern and cutting-edge Swarm AI architectures have direct applications in Search-and-Rescue and are the basis for real-world technology-based swarms.

As described in the article “How can you design and implement drone swarm algorithms?”, there are two kinds of swarm algorithms, consensus-based and bio-inspired. The consensus-based type is based upon exchanging information with a set of drones between the “peers”, influenced by visual range and how much information the swarm can gather together. This type of algorithm fits our exact need as a project, and we do not need a swarm inspired by how fish move when we have drones unobstructed in the air.

Similar to origins of Swarm Intelligence in video games, and as described by Consensus-based algorithms, we must make sure each Drone is aware of its location and the location of other drones, avoid collision, and know the location and direction of both objectives and next coordinates in

correlation with other drone movement within the swarm. They must also be adaptable to the environment and obstacles in a fluid situation, dealing with wind, water, and visual impairments to the Computer Vision-aiding camera gimbal.

Swarm Movement Algorithms

Now we need to determine the best search algorithms. Our MVP use case assumes a “normal” Search-and-Rescue scenario, searching for a person in the wilderness, with other areas and types of Search=and=Rescue as stretch goals. However according to the research paper out of West Point, “An Investigation of Search Algorithms for Aerial Reconnaissance of an Area Target”, the size of target greatly affects which search pattern should be used by drones. The researchers modified the existing search patterns to optimize for multiple target sizes. Taken from that research is the below figure, which shows Normal and Modified search patterns for the Horizontal Sweep (also called a Lawnmower pattern), Expanding Spiral, and Lissajous Search.

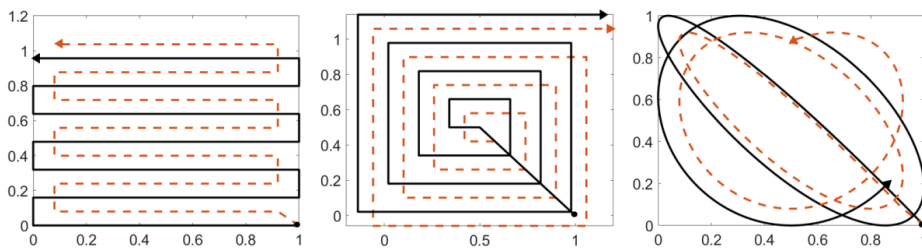


Figure 7: Standard and Modified Deterministic Search Paths

The research determined that for a large search area with a small target, such as a human, a Lissajous Search was the most effective for an active Search-and-Rescue operation utilizing aerial drones. Given an unlimited time, the Horizontal Sweep would cover the entire area in the shortest

amount of time, but Lissajous is able to cover more ground early in the search.

However, these are deterministic search paths, which do not account for changing a search path mid-flight. Therefore, we must utilize calculations based upon smaller search areas centered on points of interest in addition to the larger search pattern. Since we are working with a swarm, leaving 1 or 2 drones to perform the smaller search while having the rest continue the larger search is our current plan, however that is open to change should research indicate otherwise.

The Lissajous deterministic path acts as a good default for a large wilderness search, likely using the drones to expand the width of the line in an optimal drone formation to maximize visibility and minimize overlap between drone camera coverage. However, for a condensed search area, which occurs after finding an input such as a drone finding an abandoned vehicle, it would be more effective at that time to switch to using the Expanding Spiral outward from the point of contact, in this case the vehicle. At any time a human operator can request the LLM to change a decision or use a different known search pattern, in the case that the operator believes the LLM has made a faulty decision, allowing for redundancy in the case of system malfunction or unanticipated edge cases.

Should the drones cover the smaller area without finding anything, the LLM should have those drones rejoin and continue the larger search path, with the larger search path now skirting the heavily searched spiral area. Should more traces be found during the small search, or paths that match a heat map, those should always be followed before defaulting back to the larger search. In the case of a successful identification, the drones continuing the larger search would either rejoin the other drones, or return to the Ground Station.

Jan Hendrick's Search & Rescue landscape research

We have also found research conducted earlier this year on Search Paths of autonomous drones being used in the Scottish Highlands by their police forces, a paper called “Deep Reinforcement Learning for Time-Critical Wilderness Search and Rescue Using Drones”, led by Jan-Hendrick Ewers of the University of Glasgow. We’ve reached out and gotten in contact with Jan-Hendrik, sharing the MVP and prospective plan for our project, and inquiring about the details of their related research.

Their research combined Deep Reinforcement Learning and Probability Distribution Maps, similar to heat maps used in some current drone search methods, and developed specific training algorithms for the drones. They calculate the seen area from the drone by a fixed downward camera, similar to how our drones will be configured. They used an existing reward algorithm that balances Reward, where the system is learning to take the quickest path to a previous found target, and Entropy, where the system will explore new pathways based off probability equations over the Probability Distribution Map.

The team did fix multiple variables like drone height, however the research team successfully demonstrated greater success over the Horizontal Sweep/Lawnmower method by multiple metrics, like Distance To Find, Percentage of Targets Found, and Probability of Finding the Target over Distance. However the training for this was also set over a flat landscape with a height constraint, so this needs to be modified to fit a better real-world use case as opposed to a flat forest or plain before we can use it.

We hope to use our connection with Jan-Hendrick to interview him and learn as much as we can from him, as well as potentially working together on an updated equation, since he indicated following December or January

he would be finishing up some work and would be interested to further advance his research and help ours.

It is our intention as a team to begin leaning more on the expertise present within Lockheed Martin as we advance into an equation and mathematics-heavy portion of the LLM Training and Drone Command and Control implementation. We also must do our utmost to find other experts on existing Drone Swarm research to aid us in optimizing the final product, as it would be foolish to not use the many advancements in Drone Search-and-Rescue within even the last 5 or 2 years.

Drone Distance and Measurement

A major consideration of our setup is how our Ground Station interacts with the SOAR and SPARROW team's drones, what limitations exist, and how that shapes, limits, and enables our product design. We must begin by determining the range of movement of drones, in order to find out what mobility is required for the ground station, informing choices of hardware, power supply, and system resilience and reliability in the elements.

Drone Swarm Movement must be tied to transmittable distances to the Ground Station under the agreed upon Minimum Viable Product and can be impacted by a number of factors, such as signal strength over distance, This limitation is providing a scalable limit on Search & Rescue operations when they may occur at a large scale, in mountainous or otherwise transmission-hostile environments, or a combination of the two.

This limitation makes it the case that the placement of the ground Station must be carefully thought out and implemented to maximize drone range in likely search areas. If possible, a mobile ground station would greatly help the ability for this to be less of an issue, and more of a feature in areas where ground troops or aid workers can traverse. This does pose a major issue if drones are being used in a large hazardous area, larger than

the reliable transmission distance, and if the ground station crew cannot traverse close enough.

Because of the need for a mobile Ground Station, the Ground Station would need to run offline, with access to communication with the drones as well as GPS. This can be achieved with a laptop, likely a Windows Operating machine to work seamlessly with the ArduPilot Mission Planner and Ollama systems, and since our team is far more experienced with

Analysis of the terrain and transmutability of signals from the Ground Station to Drones through some sort of sensor or Lidar of some kind would be helpful in determining the limits of Drone Swarm range. We must determine both range from Ground Station, and well as drone-to-drone internal range limitations. In testing phases, using the test drone at first, but hopefully later using the SOAR and SPARROW custom drones in concert, we must ascertain drone-to-ground-Station range, and drone-to-drone range.

Additionally, Drone range will be limited by power consumption during a Search & Rescue mission. Since these drones are going to be designed for continual re-use, they need to return to the Ground Station by the time the Drone dies. In our testing phases, it will be incredibly important to work with the SOAR and SPARROW teams to test the lifespan on the drone batteries in the midst of operations, and measuring maximum safe range of drone operations based upon average drone speed, drone battery life, drone battery drainage rate, and distance calculate when a drone is required to return to the Ground Station, potentially for recharging.

Drone Swarm AI Operating Information

So, establishing how far the drones can reliably function in the testing phase will be an important piece of data to go along with our open-source Drone Swarm AI for any prospective users. In potential dual use cases, like the military, police, FEMA, or any other potential user with established

procedures in place, these specific limitations and end-of-use cases can help the drone swarm AI more readily fit into existing architectures of Search-and-Rescue and be deployed with less hassle on the part of the user, reducing their need to independently test.

This and any other important metadata to include for prospective Drone Swarm AI users would go in a #read-me file on GitHub or wherever else we post the final open-source product, making it easy for users to find and work off of. This other operating information provided alongside the system should include how many parameters the model is trained on, both in the Computer Vision component and the LLM Drone Swarm Movement portion, as well as model data size and training dataset size.

Other operating information can include what methods we used to make, train and fine tune the model, such as reinforcement learning, where the AI Engineers reward the system for good responses, and punish it for undesired responses. We can also include the tech stack involved in the User Interface, making it clear what programs and platforms we've used, allowing greater open source customization from others in the future. One of the most important components of a good open source publication is transparency of process and methods used in the creation and implementation of the product. We can also include information on the drone structure and designs that we used to test.

In addition to a #read-me instructional file, by the end we should produce clear metrics and tests for the Drone Swarm AI, included with the open source project. This can clearly demonstrate the performance of our product, giving an easy understanding for future researchers to determine whether their improvements actually beat out the original model. Visual graphs and models being included to demonstrate these would be incredibly

helpful for user understanding, and for easier comparison testing for future research efforts.

The metrics for testing and future comparison testing should at least include maximum search area in a given time, power consumption, Computer Vision processing time, LLM processing time, Ground Station to Drone signal latency at distance, performance in clear weather vs. Inclement weather (if possible), among others.

Cybersecurity of the Drone Swarm

It may also be prudent to share training datasets we utilized, providing both a way for others to fine-tune the model more easily and expand upon our training in future work, as well as adding transparency to the process of how we made the model. This can also help in the cybersecurity realm, as a clear process of what data made it into our model, when it was introduced, and how can help identify dataset poisoning, model poisoning, sources of prompt injection to the model, or other cyber attacks specific to AI and the training process.

A clear ontology of what data is used and how can also help our own team later on should we work on this from an open-source freelance angle later on, expanding upon our own work. It also makes this more safe and secure for Lockheed Martin to reliably use, as well as the US Government or other organizations that have strict cybersecurity standards as standard procedure.

The defense of the Drone system has multiple identifiable weak points and flaws. For one, the current proposed overall system from the MVP has initial on-board processing on every drone, then sending flagged images to the ground station to further analyze, verify, and then command and control the drones from. Each drone will have installed on-board an NVIDIA Jetson, which increases the cost and unique value of each drone.

Providing Nvidia Jetsons regularly as a requirement for more drones limits the ability to cheaply make mass-produced compatible drones, but the cost is low enough to still enable Search-and-Rescue operations given a small-to-medium budget. If drones are lost, destroyed, or otherwise incapacitated, the supply chain for new drones is bottlenecked by Nvidia stock and supply.

The Ground Station itself is required for the drones to operate properly, so should the Ground Station fail in any significant way, the drones will just hover in place until they run out of battery, or follow a failsafe procedure if we successfully add a functionality for the drones to return to base on loss of connection to the Ground Station.

The drone swarm and ground system communication also have no encryption in the existing plan, meaning in an adversarial use case, information with labelled objects from the on-board Computer Vision processing could be intercepted by an electronic adversary. The drones could also be susceptible to false coordinates sent from an adversarial device using MissionPlanner to subvert the drone swarm.

Most of all, the Drone Swarm uses GPS to navigate, which can be unreliable in stormy and inclement weather or in mountainous areas where signals are blocked. This is the most inherent way for the Drone Swarm to meet failure and potentially lose a drone or fail a mission, and it is unavoidable in the current scope of the project to rectify that vulnerability. The ground station itself should be hopefully running offline, simply using WiFi or other signals to communicate with the drone swarm but not needing to connect to cell towers or other infrastructure like a mobile phone could have to.

Drone Recharging and Replenishment

When operating in the field with the final drones, it would be recommended to have spare drone power banks, or batteries, on hand to recharge spent drones, to reduce the amount of time drones spend on the ground not searching for victims potentially still in danger during the Search and Rescue operation. Having spare drone blades for the rotating chopper portions by the Ground Station alongside the power banks would also be advised for drone repair and replenishment being achieved in an efficient amount of time.

Dependent on the number of resources, it could even be possible that spare drones that finish charging in cycles during “off-time”, when other drone swarms take off to continue the search in cycles. Additionally, the Ground Station, if mobile, must be provided a steady power supply, even if it’s a chargeable battery connected to a laptop, in order to maintain the Drone Swarm’s ability to communicate and scan.

Different drones may have different power requirements, dependent on the makeup of each drone. As we’ve been informed by Joe Rivera, our sponsor, as well as our sister SOAR and SPARROW interdisciplinary teams working on the drones, not every drone might have the same equipment. One drone may have Lidar while another may have infrared camera. If one drone has an NVIDIA Jetson Nano onboard taking care of the YOLO model computation it’s power may drain much faster than that of ones just running a normal camera. This needs to be anticipated and planned for in the hardware setup and charging cycle, as a core component of a computing drone being operational for far less time than other drones could freeze the drone swarm’s ability to efficiently and rapidly cover a Search-and-Rescue area. The potential plan of swapping drones and recharging is a likely solution to this potential logistics and power issue in the drone swarm hardware.

The use of the Drone Swarm in Wild and Urban areas

Since our Drone Swarm would hopefully be able to operate in vastly different areas, containing many different types of objects and identifiable associations for the Computer Vision portion of the model to make, it may require a multitude of various datasets or even versions of the same model to achieve that multi-environment aim. Multiple versions of the model, one for heavy urban, one for suburbs, one for wilderness, one for rural, or just one for populated and one for unpopulated, may be necessary.

This is because of the potential different aims of the Search-and-Rescue, and the different needs of the various applications of Search-and-Rescue. In a wilderness trying to find a lost hiker, it is useful to ignore trees, focus on signs of humans, cars, buildings, smoke, and other hallmarks of civilization. The LLM should be able to identify writing, signs, rocks spelling out "SOS", or other clear cries for help.

Any identification of these signs of the lost hiker would be reported to the ground station with the flagged image, reprocessed by the more powerful Computer Vision portion of our team's model and perhaps shown to a human operator for confirmation, and then process how the drone swarm should move to either confirm a find or search the area.

For our MVP, this is the minimum we need to achieve, this wilderness application of Search-and-Rescue. However if one wanted to search for survivors in a flooded North Carolina suburb, such as following Hurricane Helene, they would want to use heat maps of likely areas based off of buildings, roads, high ground, and civilization, but it would not want to flag every image of an identified building or car to the ground station. This would be far too much information overload, and not be helpful to the overall Search-and-Rescue effort. However, it would want to still identify smoke,

since it may come from an active campfire, people, tents, and other immediate signs of personal presence in the area.

In a heavily urban area, such as during a disaster relief/Search-and-Rescue use case, the LLM and training would have to take into account urban street plans, be able to recognize writing, signs asking for help, or other human visual signals. The presence of buildings and cars would be ignored, and drones would navigate both above streets and above roofs in order to get all angles, searching alleyways and side streets from the top-down, instead of having the camera potentially at an angle in a wilderness use-case. In almost any case it is likely that top-down camera coverage is effective, however any other angle is far less effective in a heavily built-up urban area, such as Brooklyn or Chicago.

However, this demonstrates the need for fundamentally different versions of the Computer Vision portion of the model, so that it identifies the right objects and connects images correctly to the LLM. This also shows the need for the LLM to respond differently in different environments and use cases, following different priorities, patterns, and search algorithms dependent upon the area, previously identified signs of people or people themselves, and other system input.

Potential extended applications of the AI Drone Swarm

This brings up the question of: How easily could this Drone Swarm apply to other scenarios and use cases with a difference in training data and LLM-directed Swarm movement algorithms? There are a berth of organizations and use cases that could utilize an AI-driven Drone Swarm, especially for surveillance, warfare, and coordinated drone efforts outside of Search-and-Rescue.

In the case of surveillance, almost nothing would need to change from the different versions of the models for populated and unpopulated areas,

especially if perhaps guarding and patrolling a secure area, for instance Eglin Air Force Base, which contains a large public wilderness area open to the public, in addition to a great deal of private military ground. To implement it, simply use the nonpopulated area model, search the area regularly for any intruders or suspicious activity, and charge the drones regularly. More specific, single-person identifying surveillance using facial recognition or similar technology would require much more work and entirely separate models, but simply identifying if there are people in an area and what they are doing is a direct application of the Drone Swarm AI.

Additionally, there could be a great deal of dual use in modern drone warfare, especially in the context of the Ukraine conflict where drone warfare is advancing rapidly. In addition to Search-and-Rescue for soldiers who are MIA or civilians, as well as surveillance of military points, a swarm of automated drones that can identify a target to a ground station over a large area at speed is valuable for several reasons.

This potential dual use in a very real and important environment is part of why a clear ontology of data, where we got it, and what tech is involved is very important for secure use of the open source AI drone Swarm in an environment that has active Electronic Warfare in the area. One drawback of the current iteration of the Drone Swarm is that it depends on GPS and MissionPlanner, and famously GPS has been unreliable when blocked by Electronic Warfare in Ukraine and other hostile zones where Search-and-Rescue or other drone operations may be conducted.

Impact in the Search-and-Rescue Community

The Search-and-Rescue community requires the ability to more rapidly, safely, and preferably inexpensively use technology to advance Search-and-Rescue operations. The desire for our product in the MVP, designing relatively cheap drone swarms in an open-source manner that successfully

utilize AI to at least partially automate the drone Search-and-Rescue component, helps in multiple tangible ways.

Currently, according to Jan-Hendrik's research introduction, Search-and-Rescue teams use two human operators for every drone in the air to successfully fly and monitor the video from the drones. Including our AI LLM greatly reduces the need for multiple humans, potentially cutting the manpower needs to run a drone swarm in half. Secondly, the drone swarm working in concert instead of all being flown by individual human operators even more so reduces manpower requirements. Thirdly, building the AI system open-source allows anyone to access the AI, ensuring a rapid acceptance and implementation process for any Search-and-Rescue teams that wish to use it.

Building the Drone Swarm with a budget of \$6300 (the combined budget of our team, and the SOAR and SPARROW team budgets) for 4 drones total, as well as a fully functioning ground station, allows for mass scalability, ease of access, and replaceability. This could allow even normal civilians to afford an unarmed drone swarm with the ability to conduct search and rescue, and puts a mass production of these effectively in the hands of every Federal or State entity in the US, and likely elsewhere. \$100,000 could buy a State Trooper force just under 16 setups matched to ours, allowing for them to find a missing child. Additionally, at larger scales some parts become cheaper, making the supply chain to develop, distribute, and make effective Search-and-Rescue Drone Swarms even cheaper and accessible.

Budget

So far, our group has not used the allotted \$300 budget provided to us by Lockheed Martin and Joe Rivera. We have not had use for it, as any and all training for models that may be compute-intensive in the future could be

done on GPUs present in the Senior Design Lab. The end result required, that of a laptop-based ground station with at most an NVIDIA Jetson Orin running the AI, also emphasizes the requirement for the AI system to run locally and offline on the Ground Station. This restricts model choices to lighter weight models with less parameters, such as Llama models with 7 billion parameters instead of 70 billion parameters.

As for how we might actually use our project budget, it's possible that outfitting our test drone given to use by Dr. Leinecker could be potentially useful in running early tests, but that possibility seems less useful compared to using the actual drones from the SOAR and SPARROW teams. We may get a LiDAR sensor, but that would be it at most. We may use it for training credits should be need to specialize and fine-tune the model using image datasets from drones, beyond what normal school GPUs will be able to do locally on base LLM models and accompanying scripts and prompt engineering.

The use of the budget in the future is far more likely to end up as an extension of the budget for our sister SOAR and SPARROW teams should they run out of their \$3000 budget allotment each and require more without our needing the funds, either for building the drones, software needed for their designs, or more expensive sensors/equipment needed for the SPARROW effort.

Stretch Goals

This project, due to the open-source requirement, lends itself to improvement over time, even without our involvement. However, should we complete our MVP in full early, there are clear stretch goals we may be able to pursue. The Wikipedia for Search-and-Rescue defines multiple areas of the field, including Ground(Lowland), Mountain, Cave, Urban Search and Rescue(USAR), Combat Search and Rescue, and Maritime.

I believe it is not likely for us to be able to add Cave, despite drones being a very good way to safely traverse tight caverns where humans might not be able to go, as that is far beyond the scope of this project. However, should we be able to find Urban, Mountainous, and Ocean datasets that we can train the drone AI on, we could add multiple model versions for different landscapes. The Ocean, for instance, has very different object detection priorities than USAR. To add the variable environments properly, we would also need to calibrate the search pattern priorities, as Urban areas require very different search patterns due to infrastructure.

To combat-proof the Drone Swarm, we could work on three things: Drone Replaceability, GPS, and Encryption. Encrypting the signals between the drones and Ground Station would help protect information the drone relays from being intercepted by adversaries in the Electronic Warfare space. Drone Replaceability could be achieved by having preset designations for drones in a swarm formation, and assigned new drones deployed to replace destroyed or disable drones being assigned to the same designation.

Finally, GPS is the hardest to replace, but could be potentially possible. In Ukraine, due to a lack of GPS because of signal jamming, they have used an AI-assisted camera view called Eagle Eyes, comparing live video from the drone to previously taken photos and video, and determining location from that calculation. If Eagle Eyes was implemented instead of the MissionPlanner software, using the Ground Station as the main computational center, the drones could successfully operate and become far less detectable.

Ollama

Ollama is an open-source AI LLM platform, that functions with Python or Java libraries to run LLMs locally on personal, private devices. It is a Community-driven project founded by two Canadian researchers; Michael

Chiang and Jeffrey Morgan. Ollama provides multiple distinct advantages for our use case and sponsor due to its open source nature. It is not controlled by foreign companies in countries like China or Russia that may be able to access private servers of private companies, protecting US-based sponsors like ours, Lockheed Martin.

The consistent maintenance and updates to Ollama allow for it to be robust against cybersecurity threats, new vulnerabilities, and fresh zero-days against the AI models that can be supported on the platform. The greatest advantage from a security standpoint is that Ollama runs entirely on your private device, and does not send LLM chat information, keystrokes, or any kind of monitoring to external servers, because there are no external servers, in direct contrast to ChatGPT, Gemini, or other private model providers.

Outside of security concerns, there are multiple advantages to our use case. The biggest advantage for the system's security is also the greatest advantage for its usability, mobility and effectiveness: the entire LLM is run locally. The Control & Movement LLM will be able to run offline on the Ground Station, ensuring there is no limit to the environment or a need for Wi-Fi signals due to the LLM itself. Remote Search-and-Rescue efforts in mountainous areas with unreliable signals will be enabled by this system design.

Ollama – Llama 3.2 Vision

In the recent November 6th update to Ollama, support for Llama 3.2 Vision has been added to the framework. Most notably, this includes improved and accurate Image Q&A for image inputs into Llama models in Ollama. As LLaVa, Microsoft's Vision-Assistant model and our LLM of choice, runs on Llama, the newer versions of Llama and LLaVa in Ollama will have

upgraded support for identifying objects in an image natively. This enables our process and workflow of identifying Search-and-Rescue victims directly.

This update also involves Optical Character Recognition, essentially the LLM being able to read written language intuitively. This can allow for accurate recognition of Search-and-Rescue signals, such as “SOS” or “HELP” in large letters, written directions to a last location given by a Search-and-Rescue victim, or other potential clues to a location that can be automatically identified, flagged, and acted upon by the Control LLM.

While not something that is currently identifiable in our workflow, the 3.2 Vision feature that allows for identification of data in models and graphs could be potentially useful if any automatically generated data on the Search-and-Rescue area could indicate more likely locations for a victim to be. This could allow for an additional method of inputting data into the LLM to improve the Search-and-Rescue effort, should relevant graphs and models be made during an operation.

LlaVa and Ollama

Ollama has support for LlaVa models up to LlaVa 1.6, also called LlaVa-NeXT, which includes more resolutions and 4x more pixels, which greatly improves the accuracy and useability of our pixel count algorithms to infer distance from the drone to an identified point using altitude, camera angle, and pixel count. It also has better Image inference and better world reasoning for images in different scenarios, improving the model’s ability to identify Search-and-Rescue targets in a variety of environments. LlaVa’s upgrade to better versions of Llama has enabled this improvement, as well as Open-Source improvements by the LlaVa research team.

First Prototype Installation

To begin the process of creating a prototype Control and Movement LLM, I first got the base models to operationally run locally offline and provide relevant coordinates in an LLM output. I implemented Ollama onto my Windows machine laptop, which is similar to laptop system the SOAR team will use for the ground station. It is an open-source download from ollama.com, that can either be run in its own space or on the Windows terminal.

From there, I used the run <model> command to initiate a download for llama 3.2-7b, which uses 7 billion parameters, making it a lighter model more optimized for a personal computer. I then used a similar command to “run llama” and download libraries for LLaVa afterwards. It then operates a fully functioning LLM within the Windows Terminal. This basic implementation is the foundation we will use for the Control & Movement LLaVa LLM at the core of our project.

Initial tests were promising, as basic prompt engineering to the base LLaVa model could generate new coordinates to move a hypothetical drone to given a starting coordinate set and directions. It unfortunately slightly hallucinated and misinterpreted commands immediately, meaning that we were correct in our assumption that the base model wasn’t specifically trained enough for the use case. I also began simply testing commands and flight paths for one drone, planning to expand to multiple drones and calculating optimal areas of camera coverage and flight plans later, first wanting to identify the ability to accurately move a drone on a coordinate plane.

For example, in response to “You are a drone hovering at 70 ft. Above a flat plain, at the coordinates 41N 31W. Move the drone 50 ft. North and provide new coordinates.” In response it gave a wordy explanation, proceeded to subtract the elevation by 50, equaling 22, which is incorrect

and unnecessary. It provided the Latitude 41.03516 and Longitude 30.98352, which while it moved the drone west in addition to North unnecessarily. Following this initial test, It successfully gave similar coordinates in the correct direction when I commanded it to "Move west 50 feet. Provide only new coordinates" in an LLM prompt, however it still provided text in addition, which would mess up the inputs to MissionPlanner directly from the LLM output.

Model Improvements

There are multiple methods required to fix the problems with the initial base model. Firstly, the response needs to just provide Coordinates, in a format readable to the ArduPilot MissionPlanner application that the Ground Station will have installed to functionally control and communicate directions to the drone swarm. This will be achieved by implementing tool calling onto the Ollama API and our model. Ollama has pre-existing support for tool calling, including support specifically for tool calling to the llama 3.2 model, that LLaVa will run on, as described on the Ollama models page.

Secondly, when we run the LLaVa model in the terminal, we can add python (or Java) scripts which will provide context, AI Guidelines and Guardrails for the LLM Output in the form of prompt engineering, as well as specific metadata fields input by a human operator. These fields would include Search-and-Rescue operation coordinate limits bound to the active search area, and as LLaVa 1.6 can read graphs and map overlays, it would be possible for the LLM to read input graphs of a new search area and recalculate a search path for it. This would require the script to be updated from inputs in the LLM prompt field given the correct python script and file setup. This would likely be achieved through a customized version of an Ollama UI plugin that allows script updating and LLM reloading with the updated script.

We also need to learn proper prompt engineering and how to effectively implement AI Guidelines and Guardrails into the model. We have researched how to properly frame prompts to the end UI, and providing instruction and simple training for human operators on how to best communicate with the LLM in the field would be a useful addition to the project. This would likely be in the form of information in the Github README file, showing suggested prompt formats.

Then, using the MITRE ATLAS framework for AI & Cybersecurity, I found guidance for guidelines and guardrail mitigations against model abuse, inaccuracy, and hallucination. The LLM Guidelines, safety controls placed onto the LLM through a Python script, append onto user inputs and use affirmative language to enforce answer standards and outputs. This step occurs between the user input and the interpretation of the input, changing the input. The LLM Guardrails, safety controls also enforced through the python script, change the output during and after it has been processed by the LLM, screening for inappropriate or out-of-bounds responses.

Utilizing both methods of LLM control in our python scripts, in addition to automated tool calling, should effectively provide a secure and reliable LLM output that is readable to MissionPlanner. This output can also be modified for other flight programs with a modified Python script, making this an effective open-source project that can be adaptable to multiple drone systems.

Native Drone Computation

Also, as of November 18th in the Ollama 0.4.2 update, Ollama now provides support for Nvidia Jetsons and can be installed onto Jetsons with the command “curl -fsSL <https://ollama.com/install.sh> | sh”. As the current plan from the SOAR and SPARROW teams is for each of the drones to have an on-board Jetson Xavier or Nano for computing, this now allows for the

possibility of no Nvidia Jetson Orin- supported Ground Station in a future design, or more likely the Ground Station simply having the ability for human operators to recall the drones or send prompts to the LLM running on the drones.

A likely implementation of this would look like a central command drone with a Jetson installed running the LLM and sending commands to others, making the others lower cost with less Nvidia Jetsons required to launch the swarm. However, especially in battlefield Search-and-Rescue use cases or in heavy weather conditions where drones may be endangered, it is useful to have redundant Jetsons on multiple drones that can transfer Command of the drone swarm seamlessly upon a drone's destruction or malfunction.

Native Drone Computation, without the need for Ground station computing, supports potentially faster compute and communication times, as the drones don't need to communicate with the ground station and send larger image or video files between it and the swarm. The native computation also allows for more secure communications between drones and human operators. Now direct coordinates are only communicated on and between the drones, not over a presumably larger distance that could expose future coordinates to enemy Red Teams.

Training and Reinforcement Learning

The Ollama-based LLaVa model will run on Llama 3.2 at time of writing, and unless there are major developments in the next 2-3 months for a Llama 3.3, this is the final tech stack for the LLM for our project release, presentation, and publishing online

With the LLM tech stack finalized, we now need to match model training to best fit our components of choice. The stages of training a LLaVa model include data preparation, pre-training, training, and testing. For data

preparation, we gather a custom dataset of top-down drone camera images, including a sufficient number of images in groups of variable environments. We need to have several forest or plains images, a dataset of urban/populated images, a dataset of ocean or bodies of water images.

Then in pre-training, the python scripts that contain model-ingrained prompt engineering, guardrails, and guidelines must be custom-fit to each environment we have separate data for. In the final product, if we achieve multiple environments, the human operator should be able to tell the model UI which environment it is operating in, so it uses a version of the model trained for that environment.

The data preparation of these datasets should involve labelling them for what the correct image associations are, to train the model to correctly recognize objects and people. We can use services like UBIAI to automatically label images for the objects contained, manually check their accuracy, and add them to the dataset the model will use.

There are several optional and variable methods to achieve the training and testing, and we believe one of our best options for the use case is reinforcement learning. Reinforcement learning involves setting up the model in a test environment, giving it test inputs, and giving it a graded reward based upon the success of its output. The system optimizes its responses to get the maximum reward the test environment will give, training the model to fit the test criteria. Each time the system gets a reward, it will update its policy or value function, which maps a state, like s to an action a .

Reinforcement learning can happen in or outside of a model environment, and due to our timeline, we will likely use our model testing interface developed by Brody to track progress, but otherwise simply run the model on Windows terminals, matching the use case conditions.

Reinforcement learning can also have pitfalls, however. If the data is not variable enough, the model will match to fit the limited data given and make false associations. It might ascertain that a birch tree means there are humans if all the photos with humans happen to have birch trees whereas others don't, for example.

To mitigate the issues with reinforcement learning, I believe it is best to use a new form of automated reinforcement learning: Reinforcement Learning with AI Feedback (RLAIF). Normally, Reinforcement Learning with Human Feedback (RLHF) is used to have human operators determine if it was a good response or not. RLAF allows a secondary AI model to analyze the output against an expected output. This is far more accurate once set up due to the automated output comparison, and because RLHF requires a lot of human feedback to be accurate. RLHF operates using a preference model, asking the human's preferences on the output after receiving it. The best version of this is ranked preference, being shown two outputs from the LLM on the same prompt and picking the better one.

RLAIF however, operates by having the secondary model generating it's own set of Ranked Preferences it will judge the original model on, which can be fine-tuned by our team to ensure it matches the priorities of identifying humans, identifying traces of civilization, and tracks. This adds associate preference scores as data labels to the responses, automating the Reinforcement Learning process.

This can be monitored by our Senior Design team by saving the data preference labels as metadata each training epoch, allowing us to determine the effectiveness of the AI judgement model. This informs us when to change or undo any training done with the AI Feedback, should the preference labels begin favoring unrealistic or incorrect patterns.

To use this Reinforcement Learning with AI Feedback for our model, we need a model that will provide that feedback. I have identified an open-source model for this purpose: starling-lm. Starling-lm is an open-source LLM on the Ollama framework that has been itself trained with Reinforcement Learning and AI Feedback. It is designed to take an LLM Chatbot's outputs and provide feedback to improve the "helpfulness" metric. The model can be run in Ollama on the same Windows Terminal as our Control LLM by using the "ollama run starling-lm" command. We can iterate multiple terminals running the models simultaneously, input the Control LLM's outputs, set up the AI Feedback model to provide a weighted reward based on its generated Ranked Preferences, and allow the automated feedback to improve the model's helpfulness, which includes accuracy and relevance of responses.

As we run multiple epochs of training the Control LLM, the automatic flow of model outputs to the starling-lm model and reward responses will shape the original Control model, successfully achieving the training step of developing the Control and Movement LLM. The training phase will overlap with the testing phase, as flaws identified in the testing phase will result in a return to more training. Testing the model already includes one of the steps of reinforcement training, automating a reward for good and bad responses. But the larger testing phase will involve manual post-epoch terminal tests, simulation tests, and eventually field tests.

Manual terminal tests after each epoch will allow us to find traditional AI metrics and results, such as model performance, and our own metrics, such as % accuracy of how many humans were correctly identified in dataset images. These can be run natively on our Windows systems and terminals with minimal effort on a regular basis throughout the training process and immediately after training, allowing us to find issues and problems without setting up a field test or simulation.

Small-scale field tests are now available to us for a single drone with a camera due to the drone donated to us by Dr. Leinecker, and can be useful in seeing how early versions of the model perform. As our sister teams SOAR and SPARROW finish their drone and ground station development, we will be able to upgrade to swarm field tests, as well as specialized tests to account for drones with different equipment and sensors. As we near the end of the course, these integrated field tests will involve fully loading the model into a fully functioning Ground Station to demonstrate the project is successful.

Simulation tests should become available to us mid-Spring semester, as our 3rd sister team is working on a simulation in Unreal Engine for a drone swarm. Once it is live and usable for our team, we will be able to quickly and efficiently load our LLM into the environment, and use it to simulate a number of versions of our LLM's prompt engineering, training, and model versions in an easy and cost-effective manner. The simulation tests, depending on timeline and the effectiveness of other groups, may allow us to test the effectiveness of different search algorithms before we have a physical drone swarm.

Transitioning from Single Drone to Drone Swarm Model

The current prototype model setup and training is intended to achieve a successful single drone taking commands and changing coordinates due to successful identification of stimuli in input images. The expansion to multiple drones taking commands and moving to synchronized positions still needs to be added. The current prototype also fails to actually consider a search algorithm or a proper flight path, but instead is focused on successfully outputting an accurate set of coordinates given an instruction.

The LLM needs to upgrade from a follower being instructed where to go, to encapsulating the entire workflow. The LLM needs to infer coordinates from image-pixel equations, access potential search algorithms, determine

the best algorithm for the situation, run the search algorithm, translate that output to sets of coordinates, assign them to individual drones, and successfully send them to the drone swarm. It currently possesses none of that new functionality.

Multiple drones also require 2 additional steps to determining drone coordinates: multi-drone area coverage algorithms and swarm movement algorithms. The LLM needs to call functions that run these automated algorithms and determine what drone paths will be the most efficient. We also want to add the ability to switch to heat maps as a stretch goal on top of this.

To implement the multi-drone area coverage algorithm, we need to adapt the pixel-distance algorithms we have developed and instead calculate what area the drone camera is currently covering, determining the coordinates the camera is viewing between based off of the drone's current coordinates and altitude. We then need to keep a running record of what areas the drone cameras have covered so far, so that the LLM doesn't retread the same ground unnecessarily.

The most accurate version of implementing this would be implementing a model of the landscape in a plugin connected to the LLM, at least in 2d, and "highlighting" or drawing over the coordinates where the drones have covered thus far. The LLM can then use this map that updates in real-time as a new temporary bounds for the Search-and-Rescue operation area, ensuring that the Control LLM will always be calculating the flight paths with the most current information.

Search Algorithm Implementation

To implement the swarm movement algorithms, we need to have several accessible python functions that contain the different deterministic and non-deterministic algorithms, and at a standard interval during the

search, perhaps every 1 or 5 minutes, run each of those algorithms to determine key metrics. These include metrics such as: total time to completion, % area coverage within 1 hour, and % area coverage within 3 hours. The different % area coverage metrics will help show how fast the algorithms can cover a majority of the area, and prioritizing general area coverage earlier in a Search-and-Rescue effort is a highly recommended tactic to find endangered people faster.

The LLM should then apply weights to these metrics, prioritizing early % area coverage and total time to completion, and select the search algorithm or technique that performs the best to continue the search with. Included in this process should be event-based algorithms for localized searches, trail following, or similar custom actions. For instance, upon identifying a trace of civilization, like a tent, a spiral localized search pattern using all drones would be initialized. Implementing these event triggers will occur in the LLM Python scripts, ensuring that when it identifies these parts in an image it will be trained to respond with the correct localized search algorithm.

Once the correct search algorithm is selected after each regular interval, the LLM needs to constantly evaluate the next drone movement coordinates for each individual drone along the search pattern. A wrinkle in the search algorithm plan is that not every drone might be the same. While we have been assured every drone will have a camera, only one drone might have LiDAR, while one other might have thermal imaging. This means, to take advantage of these specialized sensors, drones with very different equipment may have to cover the same area multiple times, in case the thermal imaging picks up something the camera does not, for instance.

Accounting for this is important, and our solution is to develop “drone roles” with different search paths and priorities. In essence, splitting the

swarm into multiple swarms and individuals. If we have 4 drones, all with cameras, one with LiDAR and one with a night vision camera, there would be one larger search algorithm for the drones with only a camera, and individual search paths for the specialized equipment later in the process.

In this scenario, since all drones have cameras, all drones would scan in one distributed search pattern to cover as much ground as possible. As the covered area of the search increases, the need for specialized drones to retread the same area increases. The drone with night vision would be a special case as well, as it only counts as a “specialized drone” at nighttime, and during the day will always be part of the larger swarm search. We will need to work with SOAR and SPARROW to determine what sensors are confirmed beyond cameras and LiDAR, what the drone specializations are, and then independently research how to best deploy those specialized drones in the swarm.

Expected End Results

For this project, we have many aspirations for how far we can apply the model, in multiple environments, in a secure manner, and with custom specialized drones. But what we are most focused on is allowing 3-4 drones to search a wilderness area guided by an LLM, have that LLM effectively communicate with the Drone Swarm through MissionPlanner, and have that successfully LLM successfully identify a human without false positives.

Beyond that, we are striving to teach the LLM to compute and use the most efficient search pattern possible. We plan to achieve an easily replicable system that is open-source and readily available for the Search-and-Rescue community to use. The intent of this project from our Lockheed Martin sponsor was to find an effective low-cost solution, and should we train our LLM well enough, it should be a safe and effective option to use in real-world Search-and-Rescue and Disaster relief scenarios. If this project in

any way helps people or saves lives, it will have been the honor of a lifetime to have done so successfully.

Ricardo's Section

Updated General Requirements of the AI agent

Once we met with our sponsor, certain requirements were changed, and new objectives were defined. While the original stated objectives remained largely unmodified, some new additions were added to better align with the project's goals. The focus of these updates was to streamline our goals and ensure that our Search and Rescue AI agent would be capable of fulfilling the needs of real-world applications.

The first key requirement was that the LLM must be able to consistently conduct operations without encountering critical errors that could cause it to crash or exist unexpectedly. Stability is important, since the AI agent would be working in real-time environments where failure is not an option. Should the stability fail, the LLM's functionality could be reduced to a point where mission failure would occur and potentially lead to the target not being found.

The second key requirement was that the LLM retain the knowledge and skills it acquired during previous training sessions, allowing it to perform the search and rescue operations confidently. This meant that the system needed a way of remembering how to effectively conduct these operations, learning from past operations and improving itself over time. This memory would ensure that the LLM didn't have to relearn everything from scratch, saving time.

The third key requirement was that the LLM needed to be able to identify specific objects at great distances. The majority of the drone's

search and rescue operation will be conducted with overhead shots of the environment, and to properly accomplish this the drone will be positioned at a high altitude. This required advanced computer vision, and the ability to detect small signals in large scenes, ensuring that the search and rescue missions could be carried out with the highest probability of succeeding.

Through research, we found that similar requirements had already been met by an LLM agent in another project, which served as a valuable reference point for our development. This prior work gave us a strong foundation, and we waimed to build upon it by following the general structure and principles used in the earlier project. With this knowledge, we were able to choose technologies that were optimized to meet the specific needs of our Search and Rescue AI agent, ensuring that our approach was effective.

The minimum viable product (MVP) for the AI agent was also discussed. These features included the ability to control three drones at once to conduct a search and rescue operation. Additionally, the AI agent needed to be able to successfully identify search-and-rescue victims using the drone's sensors. Finally, the ai agent had to be able to pilot the drones with the data given from the drones.

The LLM chosen for this project was LLava, which is appropriate for the search and rescue task.

There was also a focus on the front-end design of the application. We were pointed to MissionPlanner as a baseline of UI functionality. The interface needed to be intuitive and easy to use, providing the operator with a clear view of the system's status at all times. It needed the interface to display ongoing conversations between the operator and the LLM, allowing for seamless communication. Additionally, the interface had to show the current tasks the LLM was undertaking, such as setting waypoints or

calculating optimal paths for the drones. The front-end also had to include features for manual overrides, enabling the operator to take control of the drones if necessary. This might include forcing a drone to return home in case of an emergency or adjusting the drone's path in response to new information. Flexibility was important, as the operator needed to be able to correct problems that may arise during operations. The application also had to be capable of displaying the positions of all drones at the same time, while providing the ability to focus on any specific drone or group of drones using separate panels. This feature ensured that the operator could monitor the entire operation from a high-level perspective while also being able to focus on individual drones for more fine control.

Finally, one of the most important requirements was that all technologies and tools used in the development of the AI agent must be open source. This was a non-negotiable aspect of the project, as the result would be published for the public. Aside from these requirements, any technologies or languages that were to be utilized were fully decided by us.

Project Sequence Diagram

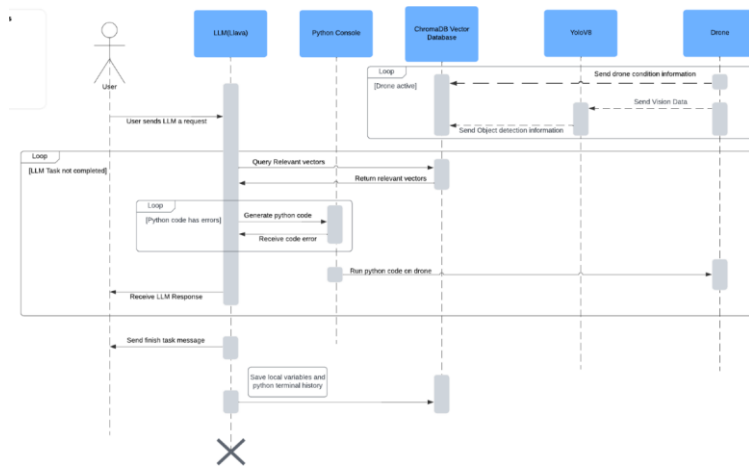


Figure 8. Sequence diagram of how the Agent will operate

The goal of our application is for the drone to be able to handle the majority of a search and rescue mission on its own. In a best-case situation, all the users would need to do is prompt the LLM to do a task, such as searching a perimeter, and the LLM will do the rest. The user could specify that the LLM should search within a circle of a certain radius, a square, or any other specific search query that the user may input. The LLM will still be able to ask the user for input, and when this happens the LLM will wait for 2 minutes for a response from the user and then if the user doesn't respond, the LLM will make an inference based on what to do. This inference wait settings should be able to be modified to take longer to wait for the user input, or to turn the feature off completely. The LLM will send the user progress updates on the LLM's current task, and what approach it's currently taking. At any time, the user can talk to the LLM again to change its current task or add any additional information that would be needed to conduct the

search operation. In the application, the user will be able to see every individual step that the LLM is taking in order to conduct the search operation, with any potential errors that it's making. By default, the LLM will only tell the user status updates on the successful operations, rather than alert to every error that occurred during the current operation. The user can ask the LLM for more detail on an operation that it has currently done, but it would hold up any tasks that the LLM was currently conducting to answer the user.

When the LLM queries ChromaDB for relevant vectors, it will look back to similar searches or training that has been done in the past and make decisions. It will also take information of the drone's current position, and any needed image data. The drone's current position data, and image data will be pinged every 5 seconds to lighten the load of all the information that the LLM will need to process. The lower throughput of data that the LLM would use should increase its ability to make decisions on the most recent data that it has. The ChromaDB database will be utilizing its own version of LLaVA in order to conduct the query, so two LLMs will be working together in order to efficiently conduct a search operation. This inner LLaVA LLM will not be the one to process any image data that it currently has, it will just return the relevant information to the search.

If the image data contains information about an interesting object, such as a person, being discovered in part of the screen, the LLM will be alerted to this. When this occurs, LLaVA will be asked to infer the meaning of the image that corresponds to the image that gave the object information. LLaVA will then ask itself whether or not this image description is important towards the current search operation. The LLM will use its tools and the object detection position data in order to determine what coordinates correspond to the detected object. Depending on how important the object that was detected is, the more the LLM will try and prioritize visiting that scene first.

If the LLM sees an object of some interest, such as a car, it will instruct the closest drone to investigate that location, while the others continue their current search. If the LLM sees a person at all, it will immediately contact human operators in order to verify that the current person is the search target. If it is, all drones will be directed to go towards that drone position and keep track of that person if they are moving. At this point, the drones will only return to their launch location if a human operator forces the AI agent to do so. If the search and rescue operation require the search of multiple objects, the human operator can ask the LLM to have a single drone follow the found person while the others try to find any other object they can.

For the LLM to be able to rapidly adapt to any situation it sees, it must be able to try and make new decisions based on its current data. To facilitate this, the agent will first try to create python code and try and run it in the console first. If the LLM responds in a way that does not follow the proper formatting, the LLM will be asked to fix it's formatting and try again. If the LLM's code produces an error when run in the console, the LLM will be alerted to this error in the code and will try again to fix it. The LLM will continuously try and follow this loop until no errors are reported, and it gets a positive response back from the console. If the LLM is stuck in a loop for 3 minutes, it will ask human operators for assistance in how to resolve its current situation. This python code will supplement the orchestration framework that it has. The python code will be more for trying to make decisions that apply to the current situation, while the tools in the orchestration framework will allow the LLM to query vectors and have pre-created functions to send commands to the drones.

Once the LLM has deemed its current task complete, it will be asked to summarize all the actions it took to conduct the search operation. This summary will be later used by the LLM to figure out what happened in any

individual snapshot that happened in the past. Any data, such as a snapshot of the current vector index, any local variables that the LLM created, and any information that was presented in the python terminal will be saved in this single snapshot. When the LLM initially starts up, it will be asked to query the latest search operation snapshot and remember any special functions or variables that it used in order to conduct the last search operation. These summaries will be composed of any functions that it used in the past, so that they are brought to immediate attention. The LLM should be able to remember how it called those functions, and what the purpose of those functions were.

When Llava is processing an image, it will take some time to calculate a corresponding caption to what the image that was generated is. Then once it generates its caption, it will also take time for LLava to make decisions based off it. The LLM for the vector database will also take some time to retrieve relevant documents. These compounds make the agent vulnerable to very rapid response changes. Human operators should take care not to rapidly change the task that the LLM is currently working on to avoid this problem.

The way that the LLM will be initially tested before the front-end is developed is through Jupyter python files. This will let us test each individual part of the process and see if any part is unable to function properly as its own component. Then, we will do a test where it's all ran together at the same time for around 10 minutes, to simulate the duration of a very short search and rescue.

Long lasting memory for the LLM and LLM coding

When it comes to deciphering meaning from a query and writing a response to it, LLMs are one of the best tools to do the job. However, due to the length of any potential search and rescue mission, it's important that the

LLM doesn't forget any past information that it has learned from the ongoing search and rescue operation. It's also important that the LLM doesn't forget any methods or ways that it used in the past to conduct previous search and rescue operations, so that it doesn't need to recreate these methods again. At its base, Llava has a token limit of 4096, which with how much data it would be processing it may not be enough to both handle the entire search and rescue operation and also be able to remember past searches and trainings. To compensate for this, we chose a Vector Database so that Llava could select information relevant to its task, while still having access to a wide selection of information. For this task to be successfully completed, we had to select three different technologies, a Vector Database, a vector embedding, and an LLM tooling technology. All of the chosen technologies were open source in accordance with the requirements of the sponsor. We chose to use Python as the language for all these tools, since all team members are comfortable with utilizing Python, and any additional machine learning libraries that we could need are likely already created in an Open-Source Python project.

When the LLM is creating python code to process its tasks, it must follow a certain format. The format that the LLM used is based on GoodAI's format that their LLM, HallM, used to process python code.

```
>>> # User: Create a python function that takes in a name and color and returns a json object
>>> person = create_person("John", "blue")
Traceback (most recent call last):
  File "<console>", line 1, in <module>
NameError: name 'create_person' is not defined
```

```
SyntaxError: unexpected character after line continuation character
>>> def create_person(name, color):
...     person = {"name": name, "color": color}
...     return person
>>> john = create_person("John", "blue")
>>> print(john)
{'name': 'John', 'color': 'blue'}
>>> done()
```

Figures 9 and 10: LLM creating python code to make a json object due to the user requesting it.

In between these two steps, Llava made 7 errors in total. When testing with other LLMs, they tended to function more consistently when attempting to complete this same task. It will be important to align the prompts in such a way that they align with Llava, since standard generic prompts which may work on other LLMs, may not work with LLava. Since Llava is mainly image focused, following prompts that ultimately lead to LLava analyzing an image would likely make it more effective at doing its required tasks. Through more training, and augmenting its memory with the vector database, the number of these errors will be reduced. It will also be able to utilize functions that it has created before, in its current state it's not able to do so. The most common error that it committed was adding a / in functions that are separated with an underscore. Create_person was turned into create_/person, until it was able to recognize this mistake.

LLamaIndex with Llava

LlamaIndex is the orchestral framework that ties together the vector database, the LLM, and any other tools that are needed for the AI assistant to properly conduct search and rescue operations. It's specialized for tasks that are about retrieval-augmented generation (RAG), which is having an LLM that takes in information from outside of its training dataset and makes decisions based off it. It simplifies the process of embedding and inserting a document into a vector database by providing an API that allows both to be set at runtime. LLamaIndex allows the inserting and updating during runtime, which is important due to continuously getting new object detection data from the drones. It allows the LLLM to perform tasks that have sequential components, such as querying for information first, and then utilizing other tools to make a decision off that information. This will allow our AI agent to first query the latest information it requires to make the current search. Then it can use its other tools to set waypoints for the

drones, or any other tasks. The tools that the Agent uses are composed of pre-defined python code that accomplishes specific tasks.

The way that LlamaIndex allows the LLM to interact with the vector database is through creating an agent with the LLM that has tools attached to it. The tool that it uses to query the vector database is the QueryEngine, which takes another LLM which will parse queries and retrieve the information that is relevant. One of the project requirements was that Llava was utilized as the LLM, so there will be two instances of LLava.

When Llama Index creates an agent through this way, this new agent requires prompts that allow it to recognize how to use its tools. By default, Llava is not able to consistently make use of the tools that are provided to it, since it will attempt to give its tools the parameters necessary to function, but in an incorrect format.

```
Thought: The current language of the user is English. I need to use a tool to help me answer the question.  
Action: multiply  
Action Input: {'input': 100}  
Observation: Error: multiply() got an unexpected keyword argument 'input'  
> Running step 07fb66e5-952b-4b09-84e4-1b8dbc361a05. Step input: None  
Thought: It seems that the "multiply" tool does not accept an input argument in the format provided. I need to use a different tool or approach to answer  
Action: canadian_budget_2023  
Action Input: {'input': ''}  
Observation: Error: unsupported operand type(s) for *: 'NoneType' and 'float'
```

Figure 11. LLMs inconsistency at properly utilizing provided tools

To make Llava properly function with LlamaIndex, prompts will need to be developed that allow it to recognize the proper format to query its tools. The prompt will have to contain information about the name of the parameters in the functions, since that's how the LLM correctly places the information, it currently has into the appropriate places in the function. After appropriate prompts have been developed, the agent will have to continuously be trained until it can consistently follow the correct format to call upon its tools.

ChromaDB

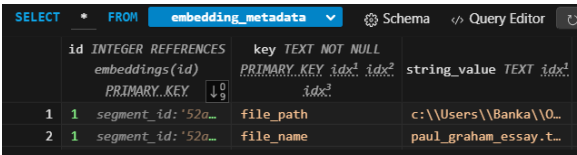
The goal of a vector database like ChromaDB is to store information vectors generated through vector embeddings, allowing documents to be stored in memory in a queryable format that doesn't need to be reprocessed each time they are accessed. This functionality is essential in enabling rapid and relevant retrieval of stored data, which is needed for a search and rescue operation.

When selecting the vector database for our LLM, there were specific requirements that we identified to best aid our LLM in their search tasks. The database needed to be open source in order to follow the sponsor requirements. The database needed to be self-hosted to allow continuous operation during the AI agent's search and rescue missions. The search and rescue operations will most commonly be done far from urban areas, so internet connection may be limited. The drones only maintain a connection to the ground station, so any external servers could result in losing important information. It could also lead to significant delays from when the drones gather the information, and the LLM responding to the collection information.

For the drone's operation, the primary format for information transmission will be the JSON format. ChromaDB is able to manage structured data well, and this JSON compatibility allows for simple integration, since the drone's data can be stored and queried without the requirement of the embedder to process it again. ChromaDB's capability to ingest, store, and rapidly return queried data significantly reduces potential communication delays. This speed helps mitigate any connection problems that may arise.

ChromaDB's API is simple to use and offers a streamlined integration with our LLM orchestration framework. ChromaDB runs on SQLite as a backend, giving it a lightweight and accessible storage solution that lets us easily see what vectors are stored in the database. This will be helpful when debugging and

testing the agent. SQLite has low overhead and is optimal for storage of information of any given mission.



The screenshot shows a SQLite query editor with the 'embedding_metadata' table selected. The table schema is displayed, showing columns: 'id' (INTEGER, PRIMARY KEY, REFERENCES embeddings(id)), 'key' (TEXT NOT NULL, PRIMARY KEY), and 'string_value' (TEXT). Below the schema, two rows of data are shown:

		segment_id: '52a...	file_path	c:\Users\Banka\O...
1	1	segment_id: '52a...	file_path	c:\Users\Banka\O...
2	1	segment_id: '52a...	file_name	paul_graham_essay.t...

Figure 12. A snippet of the ChromaDB SQLite database, with some embeddings inside.

These features make ChromaDB an ideal choice, fulfilling the need for a high-performance, resilient, and easy-to-maintain vector database system tailored to the requirements of our AI agent.

Accurate object detection and tracking at long distances

When conducting search and rescue operations, it is important for the drones to cover as much area as possible to maximize search efficiency, and drone power efficiency. The drones must operate at higher altitudes rather than remaining close to the ground in order to be able to achieve this. This has the advantage of giving the drones a wider picture of the search location but comes with the disadvantage of necessitating precise object detection of small objects. To address this drawback, we chose YoloV8 as our object detection model since its powerful image detection allows it to detect such small objects. The most optimal search conditions would be a sunny day, in an environment without many obstructions. By utilizing the ultralytics Yolo models, we were able to create a training environment to develop and test a model and see its performance. This allowed us to create a model that is more specific to the needs of our project. In addition to this, we incorporated the VisDrone dataset, which is specifically tailored for drone object detection and object tracking. This allowed us to train the model with scenarios more akin to what a drone would face in the field. One benefit of YoloV8 is that it's able to process video output and process it into object detection data, which

would be fed to the Vector database to keep the AI agent constantly informed of the search situation. This would allow the AI agent to make better decisions while operating in the field.

To further tailor YoloV8 to our search and rescue scenario, we found other datasets and gave them labels utilizing Roboflow. Roboflow gave us an easy way to annotate the input images and collaborate to spread the number of images throughout the team. Its AI features also helped cut down on the annotation process, since Roboflow itself was able to create some high-quality annotations of its own. It also had the advantage of allowing us to combine multiple different data sets of varying situations. Such as the drone being closer to the ground, being far away from the ground, and other daylight conditions that may be present in an active search and rescue mission.

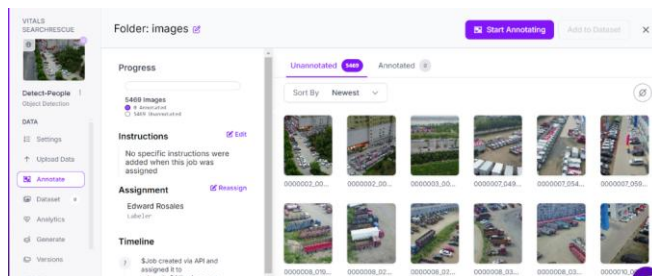


Figure 13. Images uploaded to roboflow to be annotated, or further analyzed before combining into a full dataset.

From the training, we were able to determine that YoloV8 at base was well equipped to deal with detecting small objects from far distances. In the pictures below, there are examples from the training showing the labels that were derived, most important being 0, a person. Additionally, YOLOV8 was also able to recognize groups of objects as one entity, such as the label 6, which stood for a group of people. The model didn't have any noticeable hallucinations, ignoring most of anything that wasn't a label that was

specifically looked for. YOLOV8 can be trained to locate objects that would be useful also be useful for the search and rescue operation. These additional objects could consist of things like cars that are off the roads, abandoned bicycles, and potentially even things like clothing.

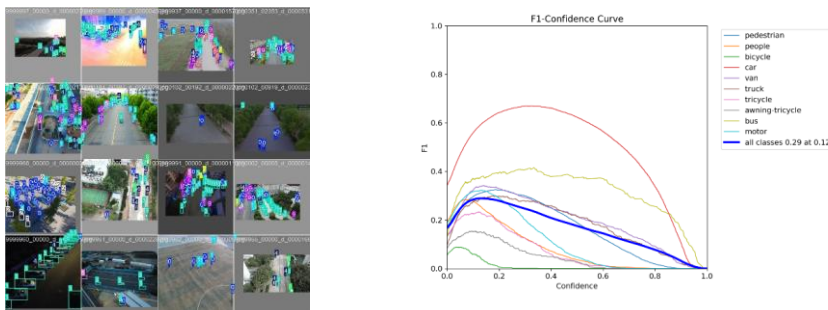


Figure 14 and 15: Left image shows the labels that YoloV8 attributed to the images, and the right image shows the F1 confidence curve of the YoloV8 testing.

According to the report that ultralytics generated when testing the dataset, it's very good at being trained to detect specific objects.

Front-End

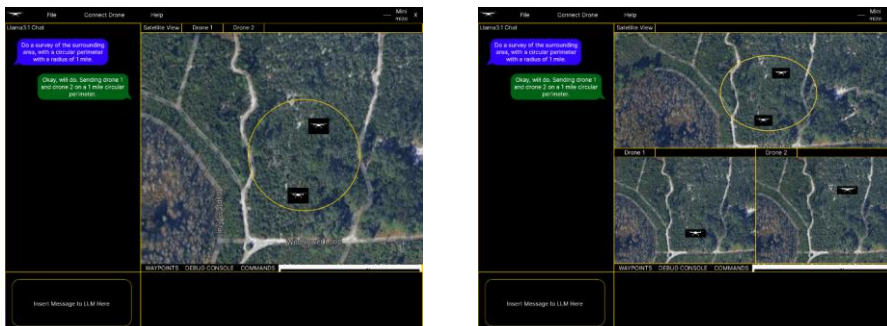


Figure 16 and 17: Example Front-End for the AI Agent. Figure 8(Right image) shows the use of multiple different drone windows for easier viewing.

For our frontend, we aimed to create an interface that was simple to utilize while providing the ability to command the drone in multiple different ways. Our design hinged on having multiple views available to an operator of a search and rescue operation at any time, allowing for a lot of flexibility in various scenarios. We laid out the groundwork for how our application would look in Figma, designing it with the flexibility to create and move panels around. This allowed us to rapidly prototype different versions of the frontend until we reached one that was both easy to use and pleasant to look at.

To enhance user experience there will be intuitive navigation features that ensure that operators can seamlessly switch between different views and functions without any confusion. The operator will have the ability to assign commands to the LLM using the text box on the left and see the LLM's response after their requests have been processed. Additionally, the operator can view the waypoints the LLM has assigned in the waypoints tab. The next tab available is the Debug Console, which will show the true output of the LLM's processing, rather than just the result that the LLM would regularly show the operator. It will show each step, such as gathering information from the database, utilizing the information from that database, and any other intermediate steps that the LLM deems necessary to complete its task. Finally, the commands tab will be utilized by an operator to force the LLM to stop what it's doing and execute a specific task. An example of this would be calling off the current search operation and returning all the drone's home.

Through the file section, a human operator will be able to upload a specific pre-trained LLM model to use in the current search and rescue operation. Allowing the human operators to swap out to LLMs with different training

datasets will allow them to select LLMs that are better suited to certain commands, and operations. It will also allow operators to train the LLM, use it in a mission, export the model that was utilized at that point, and then re-upload the model to train it further.

Drone Failsafe

We have specified specific conditions in which the drones could be put into peril and would need to be returned.

- The drone is low on battery
- The drone has lost connection to the ground station
- Worsening weather conditions
- GPS failure
- Camera/Object detection failure
- Ground station has shut down
- Drone has delays reaching the waypoints
- Unfavorable amounts of sunlight

If a drone is low on battery, what actions the LLM will take with that drone depends on the situation. If a human operator deems the drone to be of high importance to the mission, they can override the safety put in place to keep the drone going for as long as possible, even with the possibility of drone loss. In other cases, if the drone is too far away from the ground station when it has low battery, it will be instructed to gently land down on the ground and turn off all equipment other than its GPS locator so that human operators may later retrieve it. If the drone is close enough to the ground station, it will be directed to shut off any non-critical equipment, such as the drone's camera, and return to where it launched from. The drone's battery charge will also be monitored, and if the battery draw is decreasing at a quicker than normal rate, then the drone will be instructed to land at its current location.

When a drone has lost connection to the ground station, the ground station will continuously ping the drone to try and find it again. If the drone is missing for more than 3 minutes, the human operators will be alerted and be told the drone's last known location, and general condition. If the drone reconnects after this time, the default behavior for the LLM is to send the drone back to its launching location. If the drone is continuously regaining and dropping connection but each disconnect lasts less than 3 minutes, the LLM will also send the drone back to its launching location. If the drone is moving out of range, and losing connection, the LLM will direct the drone to go to the last location that it had the strongest connection within the last 5 minutes.

The AI agent's response to worsening weather depends on which weather conditions are occurring. If it's lightly raining, the drones will not be ordered to return and will keep searching. If heavy rain is predicted, the LLM should ask the human operator whether or not it should stop its current search task. If instructed not to stop the search task, the drones will keep trying to continue the search, although this could lead to a drone being lost. If the environment is filled with fog, the LLM will instruct the other drones to follow the drone that is equipped with a LIDAR sensor, sense the LIDAR sensor could see better through the fog than the regular camera system. During highly windy weather, the drone will be instructed to land at its current location and wait for a human operator to retrieve it to avoid the drone being sent off course. During night, or otherwise dark locations, the LLM will tell the other drones to more closely follow the drone that is equipped with the LIDAR sensor and resume the search mainly with the LIDAR drone. The drone should not be deployed for search and rescue operations in highly snowy conditions, since camera output, lidar output, and flying ability will be heavily affected.

When a drone's GPS system malfunctions, but the drone is still in good condition otherwise, the drone will be instructed to hover in place. If the GPS signal doesn't come back within 1 minute, the LLM will instruct the drone to land down at its current location. If the drone's GPS signal is malfunctioning and only giving GPS information that is not accurate and describing that the drone hasn't moved at all even though it has, the LLM will send the drone back to its launch location. This is so that the drone doesn't stray farther away from its last known location and accidentally goes out of range due to not knowing for certain what its current location is.

When a camera malfunction occurs, the drone will be instructed to immediately return to its launch location since it can no longer continue the search. If more than 5 minutes goes by without new object detection data being sent to the LLM at the ground station, the LLM will ask the human operator whether or not the search should be continued by that drone. By default, the LLM will send the drone back to its launch location.

When the ground station is shutting down, if any tasks are still being executed by the drones it will instruct them to return to the launch location. The ground station should be able to execute this, even due to sudden shutdowns to prevent the drones from hovering indefinitely until they run out of battery. The ground station should also attempt to save whatever checkpoint information that it has made in the current search operation. If the ground station is turned back on, then the ground station will load the newest checkpoint that it has created. If the LLM can detect the drones, and what its last current object was, it can stop the drones from retreating to the launch location and instead continue the search and rescue operation. The LLM should ask a human operator if the last search and rescue operation is resumed before proceeding to do so.

When the LLM assigns a waypoint to a drone, it does it by calculating an upper time limit for reaching that destination. This time limit makes sure that the drone is efficiently getting to its destination. If the drone takes longer to reach the destination that was set as the predetermined duration, it triggers an alert for human operators. At this point, the human operator are prompted to decide the appropriate course of action, which might involve assigning the drone another task to attempt to fix it, investigating other issues, or calling the drone back to the ground station.

The drone's primary method of identifying its target relies on its onboard camera, which plays a pivotal role in detecting and analyzing the environment. However, since the camera relies on color information as light decreases, such as when the time gets closer to night, the camera's ability to capture and process reliable data is significantly reduced. When this happens, the drone's ability to deliver accurate and usable Search and Rescue information to the Ground Station is heavily diminished. At this point, if the drone has not successfully located its target, the drone should return to the ground station. Continuing the Search and Rescue operation under low light conditions would give progressively worse information. The drone returning to the ground station ensures that the drone is safe.

Drone malfunction Anomaly detection

To effectively handle scenarios involving drone malfunctions, it is necessary for the drone to detect and identify that there's an issue as soon as possible. A quick response allows for the drone to take action that will minimize any potential harm that could come to it. Relying on basic status analysis can result in false positives, where small lapses in operation are incorrectly flagged as malfunctions. It may also fail to detect true negatives, where a gradual decline in performance goes unnoticed until it leads to a catastrophic failure. Such failures would occur without warning.

Anomalies in a drone's functionality are composed of deviations from its expected operational performance, including sudden or small changes in efficiency, or accuracy. These anomalies can endanger the drone.

To enhance the detection of fault anomalies, advanced methods such as deep learning can process it. This algorithm type is good at analyzing patterns in sensor data and finding anomalies.

Dataset

Due to not having access to the finished drone, flight data cannot be created into a dataset to monitor faults specific to that drone. To compensate for this, the Biomisa Arducopter Sensory Critique (BASiC) Dataset contains information about sensor failures, such as "GPS, remote control, accelerometer, gyroscope, compass, and barometer." This data was collected through flying the drone for hours at a time and outputting the drone flight information to a .bin file. The drone used to record the data was a quadcopter, the same type as the finished drone. The dataset has several types of failures, such as GPS failures, RC failures, accelerometer failures, and others which would be useful in detecting anomalies in the drone's equipment. It also has some flights where no errors occurred. Once the finished drones are available, drone flights will be conducted to capture data that is specific to those drones. To visualize the data in the .bin file, Mission Planner was utilized.

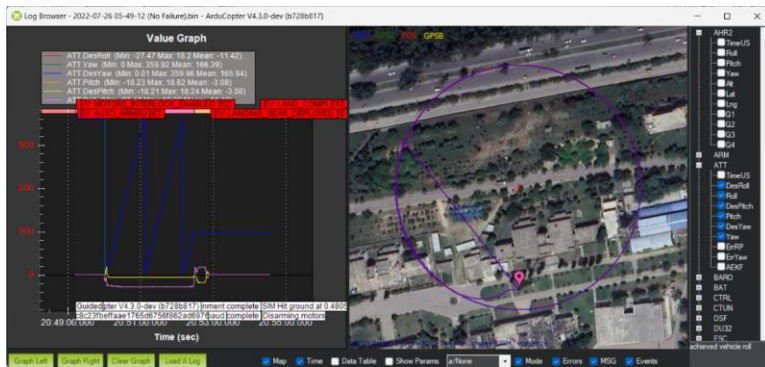


Figure 18 Mission Planner showing the desired values (roll, pitch, yaw), actual values (roll, pitch, yaw), and the corresponding flight path. This was a no error flight.

Anomaly Detection Toolkit (ADTK)

The approach taken was to first isolate the ATT category of message, which contains crucial information on the yaw, pitch, and roll of the drone. This data, tied to the drone's movements, was taken to create a basic dataset. Then, taking the TimeUS field that each message in the .bin, the messages were aligned and combined to form rows. These rows included several columns with key data points such as yaw, battery voltage, and the desired roll and yaw values. By comparing the actual roll and yaw values with the desired ones, anomalies could be identified. Specifically, the difference was calculated by subtracting the desired values from the actual ones, such as $\text{yaw} - \text{desired yaw}$, providing an indicator of how much deviation occurred. This method was critical for detecting large shifts between the desired values, which could signal issues with the drone.

One drawback to taking the difference as the data to run anomaly detection on was that due to slight delays between the time a desired value, such as yaw, was set and when the desired value was achieved by the drone. These delays sometimes cause spikes in the data that weren't true anomalies, but the result of the system's latency. Despite this, the majority of the data,

especially when the drone was operating under normal conditions, showed oscillations around zero. This showed that the system was mostly stable. To properly detect anomalies an additional field was added, called Datetime. This field stored the actual date and time of the flight, linking each message to the corresponding moment in time when it was recorded.

To standardize the data and ensure consistent seasonality, the dataset was resampled every 500 milliseconds. Any missing data points were handled with a combination of forward fills and mean values to maintain the seasonality of the time series. With the data structured, the ADTK was used to utilize algorithms designed for detecting anomalies. Since the data was multivariate, anomaly detection was applied to each type separately, ensuring that issues specific to each variable could be detected by themselves. Several different anomaly detection approaches were tested to determine which method best suited the task of fault detection, given the complexity and variability of the drone during its operation.

During testing, the LevelShiftAD version of the ADTK detector, which analyses the median of two sliding windows, proved to be the most effective. It consistently identified significant spikes in the data, accurately capturing moments when the drone experienced substantial deviations from expected behavior. At the same time, it was able to filter out some of the noise, such as the ones that occur during regular flights, and ensured that mostly only appropriate anomalies were flagged. This made it great at detecting faults that may have otherwise gone unnoticed.



Figures 19 and 20. LevelShiftAD under Gyro failure conditions, looking for anomalies in Yaw Difference. The right image shows the drone deviating from the directed path.

The anomalies being detected against were high spikes in difference between the Yaw and the Desired Yaw. Each red marker represents an anomaly that was detected. When utilized in a dataset where there were no errors present in the flight, LevelShiftAD tended to only identify an anomaly where there were periods of rapid change.



Figures 21 and 22. LevelShiftAD under a No error flight, looking for anomalies in Yaw Difference. The image on the right shows the drone properly flying the selected path.

To combat LevelShiftAD sometimes categorizing sudden rapid changes as anomaly, the anomaly detection will be run on multiple distinct categories of data at once. This will help prevent detecting false anomalies in the cases

where one category has a spike, but another remained consistent, indicating that a fault had not yet occurred. Supplementing the drone's fault detections with LevelShiftAD will make it so that if a drone's equipment is failing during flight, human operators will be notified sooner. The drone's data sent to the ground station used for fault detection will be analyzed on an individual drone basis, since each drone could face different conditions.

Potential complications of fault anomaly detection

When the anomaly detection system is run on the live drone, the data transmission to the ground station will not be at consistent, continuous intervals. This consistency can cause problems, since lapses in the transmitted data could lead to some anomalies being unnoticed due to the gap between data points. These anomalies could be critical and lead to situations where the drone is presumed to be safe but in danger.

To minimize the impact of the inconsistent data transmission, the anomaly detection will have to be optimized towards being able to recognize anomalies even during periods where data transmission is spread out. Interpolating between the values of the transmissions could be utilized to try and make the information provided to the anomaly detection more consistent.

Another complication is that the system that the drones use has various levels of importance in keeping the drone safe. In a situation where the GPS system is malfunctioning, the drone can continue flying and decide on which action to take based on its data from its other sensors. In this case, values such as yaw, roll, and pitch may still be close to their desired outputs. However, if the fault is caused by the rotors, the yaw, roll, pitch parameters are likely to not fit the desired values. The GPS may continue to function properly, even as the drone's flying ability is compromised. This difference in importance of system failures makes it important to weigh

certain fault anomalies higher than others. Faults detected in critical systems such as rotors, power, or communication will be immediately addressed if detected, while ignoring other less problematic anomalies. If multiple lower priority fault anomalies are detected at the same time, a response should still be done since it's likely that the drone's systems will collectively continue to decline.

Scenery Drone Anomaly detection

When it comes to search and rescue missions, the goal is to find the target in the shortest amount of time possible. There are situations where the drones may not identify the target in their vision, but there are anomalies present that would indicate the presence of the target. Such anomalies could be cases where clothes are spotted on the ground, an active campfire, and other cases where objects that have not been defined in object detection would lead to a quicker search. In cases such as this, the success of the operation can hinge entirely on detecting these signs. It is important to include anomaly detection in the drone to have the search and rescue be as comprehensive as possible.

If the drone was able to successfully identify the target while investigating, the drone will ignore any further anomalies that it detects and will continue following the target. If the drones lose track of the target, they will prioritize searching the immediate area instead of looking into any anomalies that they detect. Once the amount of time has passed wherein a target could have covered a large distance, the drones will start responding to any found anomalies again. If possible, drones should attempt to return to the closest anomaly that has already been discovered, with a preference to those that are closer to their current location.

Dataset

For a drone to be able to properly identify when an anomaly has occurred, it must be trained on data that will be like the conditions that the drone will face in search and rescue missions. This will be mountainous areas and large forests.

Mountainous regions tend to have a lot of flat open space and often have consistent rock colorations. These factors would make any notable anomalies stand out more. Weather conditions can impact anomaly detection, whether it be through fog, rain, or snow. If possible, the dataset should include samples of such conditions.

Forests are important to build a proper dataset around since the look of any given forest can heavily vary. The visuals of a forest vary significantly depending on where they are located, the species living in it, and the current season. Some forests are composed of lush green trees, while others have trees that feature colors such as yellow and orange. Forests also look different when it's snowing, which can further complicate detecting which objects are anomalies, and which objects are not. To finely tune ASTNet's capabilities to the drone search and rescue operation task, a custom dataset will need to be created that has sections of videos where there are no anomalies, and sections where there are frames containing notable anomalies.

Challenges in acquiring dataset

A challenge in acquiring such a dataset is that regulations of flying drones tend to be strict in most forested areas of the United States. Simulations of forests are not able to be utilized for this purpose due to not being able to closely reflect a forest in the real world. Of the datasets that are publicly available, most are still images rather than a drone's visual flyover. These datasets also tend to represent urban environments,

compared to the rural environments that the drones will face during their search and rescue operations.

To be able to surmount this challenge, there are short videos online that already contain video footage of a drone flying in forested and mountainous regions. Enough of these videos would have to be collected and have their anomalous frames annotated to form a proper dataset. With this approach, the main challenge that would remain would be placing anomalies in videos that do not already contain anomalies.



Figure 24 Frame of a video containing flyover footage of a drone. Made by SALAR, <https://www.pexels.com/video/aerial-view-of-serene-forest-river-in-spring-29453322/>

For the example shown in Figure 3, the structure located on the bottom right would make the frames containing it anomalies. During search and rescue operations, any structures that are secluded could house the target the drones are looking for.

Attention-based Residual Autoencoder for Video Anomaly Detection (ASTNet)

To effectively identify anomalies that may arise during a drone's operation, it is important to develop an efficient anomaly detection system. This system will allow the drones to perform the search and rescue operations with greater efficiency, without the limitations that come with being unable to directly identify and track the target. This ability is especially important since search and rescue missions tend to be unpredictable and

take place in challenging environments. These missions can require rapid responses, which leads to traditional detection methods not functioning as well. Since searches can be conducted in various ways, the ability to detect unexpected events in real-time can make the difference between locating and not the target.

ASTNet, although originally designed and primarily used for video surveillance applications, proves to be highly versatile, allowing it to function in real-time environments like search and rescue operations. The system's adaptability lets it function effectively in many different situations, including ones that necessitate looking at complex and unpredictable environments.

ASTNet's approach to anomaly detection is different from other traditional methods, relying on video encoders rather than conventional machine learning models or neural network techniques. While the traditional models do well at detecting expected specific objects or events, they do not function as well at detecting unexpected objects or events. ASTNet's implementation can work with these traditional models to cover this blind spot in their functionality.

ASTNet is good at recognizing anomalies by predicting what the next frame in a video should look like, based on what its dataset has determined to be "normal" operations. This prediction-driven approach lets it detect sudden movements, unexpected objects, or changes in the scene that may not be typical for the environments that they are present in. ASTNet's ability to anticipate the next video frame lets it have higher sensitivity to anomalies that may otherwise go unnoticed, such as objects appearing in places where they should not, or movements that are unusual.

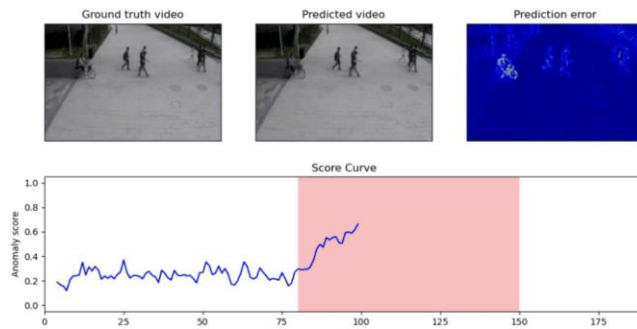


Figure 25 ASTNet detecting a bike as an anomaly, credit <https://github.com/vt-le/astnet>

Potential complications of scenery Anomaly detection

When a drone detects anomalies, its main goal is not to find a specific target but to identify anything in its environment that deviates from what is considered normal within its dataset. This allows the drone to quickly detect sudden changes in the environment, allowing it to rapidly respond. However, this approach also increases the likelihood of false positive anomalies, where normal events are accidentally flagged as anomalies.

One situation that may lead to false positives occurs when a ground team is present. The search team may accidentally create anomalies by leaving behind tools designed to aid in the scanning or navigation process. These tools, which may be essential for the search operation, could be seen as unusual, prompting the drone to flag them as anomalies. Markers placed to guide the ground team's movements or communicate important locations could also lead to false positives.

Another scenario is when detected anomalies are caused due to situations unrelated to the current search and rescue operations. These scenarios could be caused by hikers, wildlife, or other external factors. Although these would be considered true anomalies, they have the potential to disrupt the drone's search by taking its attention away from the current mission. If a drone

investigates these anomalies, it may waste time and resources investigating unrelated events, slowing down the search. This slowdown could lead to the drone having to return to the ground station to recharge before it's able to find its target.

To prevent the false anomalies from veering the drones off their mission, there are many approaches that will be taken to minimize their impact.

1. Single Responsibility Anomalies: In cases where a drone detects an anomaly, it will be solely responsible for investigating that anomaly. This prevents multiple drones from becoming unnecessarily diverted by the same anomaly, enabling them to continue their search operations. If the anomaly were of significance to the search and rescue operation, other drones could then go to the current anomaly.
2. Human operator final decisions: For more significant anomalies that could impact the progress of the search, the drones will send a message to the ground station to request a decision on the next course of action. Human operators will help filter out false positives that may have been flagged by the algorithm.
3. Prioritizing Object Detection: When a drone detects an anomaly but identifies the search target, it will prioritize the target over the anomaly. This ensures that the drone always focuses on finding the target, rather than being sidetracked by potentially unrelated anomalies. If the drone loses track of the target, it will prioritize anomalies that are close to the target's last known location, increasing the chances of finding the target again. However, if enough time has passed and it is possible that the target has moved a significant distance away, the drone will shift its focus away from anomalies near the target's last known position and expand its search.
4. Ground team coordination: To conduct the search most effectively, the drones and the ground team need clear communication. By sharing

what areas have already been searched, accidental anomalies can be minimized. The drones can also be taught to ignore signals or markers that the ground-team commonly uses to mark locations.

Edward

Computer Vision

Introduction:

Computer vision is a critical technology in the development of autonomous drone systems, particularly in applications such as search and rescue operations. The integration of this technology allows drones to interpret visual data from the environment, recognize objects, avoid obstacles, and make decisions in real-time. It is through computer vision that drones achieve a higher degree of autonomy, enabling them to navigate complex environments without direct human intervention. This process is heavily reliant on a combination of hardware (such as cameras and sensors) and software (algorithms designed to process and interpret visual data), both of which are central to the functionality of the drone swarm in this project.

For this project, the drones are equipped with an array of sensors, including infrared (IR) and electro-optical (EO) cameras, as well as LiDAR sensors. These sensors are vital for providing the data necessary for the AI agent to process the surrounding environment. IR cameras capture heat signatures, allowing the system to detect living beings or sources of heat, while EO cameras capture standard visual data in the visible light spectrum. LiDAR offers precise depth perception by measuring the distance to objects in the environment using laser pulses, making it a valuable tool for navigating cluttered or unstructured terrains. This combination of visual and depth-based data ensures that the drones can operate effectively in a variety of

search and rescue scenarios, enhancing both object detection and obstacle avoidance capabilities. This combination enables the drones to operate effectively in both daytime and nighttime scenarios, as well as in conditions where visibility is otherwise impaired, such as during fog, smoke, or other obstructions commonly encountered in search and rescue operations.



Figure 26: Infrared Image

Infrared (IR) Cameras

Infrared cameras capture thermal radiation emitted by objects. Unlike RGB cameras that capture visible light, IR cameras detect heat signatures and present them as images with varying intensities based on the temperature of the objects. These cameras typically produce monochromatic images where warmer objects appear brighter, making them distinguishable from cooler surroundings. IR data consists of a single channel, which poses challenges when integrating with RGB-trained object detection models.

Applications in Search and Rescue

IR cameras are especially useful in scenarios where visibility is limited, such as nighttime operations or environments filled with fog, smoke, or dust. By detecting heat signatures, the system can identify living beings, such as survivors trapped under debris, or locate sources of heat, like a vehicle or campfire. This capability makes IR cameras invaluable in detecting life forms in low-visibility situations, ensuring no survivor goes unnoticed.

Electro-Optical (EO) Cameras

EO cameras operate in the visible light spectrum, much like standard RGB cameras, capturing detailed imagery of the environment. They provide visual data that matches human perception, allowing the drones to detect and classify objects accurately during daylight conditions.

Applications in Search and Rescue

EO cameras are used for identifying and tracking objects that are visually distinguishable, such as vehicles, clothing, tents, or structural damage. They are particularly effective in daylight operations, where visibility is unobstructed. Additionally, EO imagery provides visual confirmation for objects detected by other sensors, such as verifying the presence of a person identified by an IR camera. This synergy between EO and IR sensors helps improve detection accuracy.



Figure 27: LiDAR Image

LiDAR Sensors

LiDAR (Light Detection and Ranging) sensors use laser pulses to measure the distance to objects, creating a detailed 3D map of the environment. The sensor emits light pulses that bounce off objects and return to the sensor, calculating the time taken for the light to travel. This data forms a point cloud, which provides highly accurate depth information, making LiDAR particularly useful for obstacle detection and terrain mapping.

Applications in Search and Rescue

LiDAR offers critical capabilities in environments where precise spatial awareness is necessary. It enables the drones to detect obstacles such as fallen trees or debris, making it easier to navigate in cluttered environments. Additionally, LiDAR is invaluable in terrain mapping, allowing drones to map areas like collapsed buildings or forests in 3D. This information is crucial for generating safe flight paths and assisting ground responders in planning their rescue operations.

While IR, EO, and LiDAR cameras provide critical enhancements, much of the focus in this project is on RGB-based object detection using the YOLO model. RGB cameras allow drones to detect and classify objects using the same type of visual data humans perceive. IR and EO cameras, though essential for specific contexts, particularly in low visibility environments, complement the RGB data rather than replace it. As the project progresses, further fine-tuning of the models for IR and EO data may be considered, but the current emphasis remains on the performance of the YOLOv5 model for RGB-based detection tasks.

YOLO Models:

Given the variety of sensor data being processed, choosing the right object detection model becomes crucial. YOLOv5 has been selected as the primary model for object detection. While more advanced models like YOLOv8 offer improved accuracy, YOLOv5 strikes a balance between computational efficiency and performance, making it ideal for environments with limited computing power. The model's ability to provide reliable results without the heavy computational demands of its successors, such as YOLOv7 and

YOLOv8, makes it a practical choice for the current project.

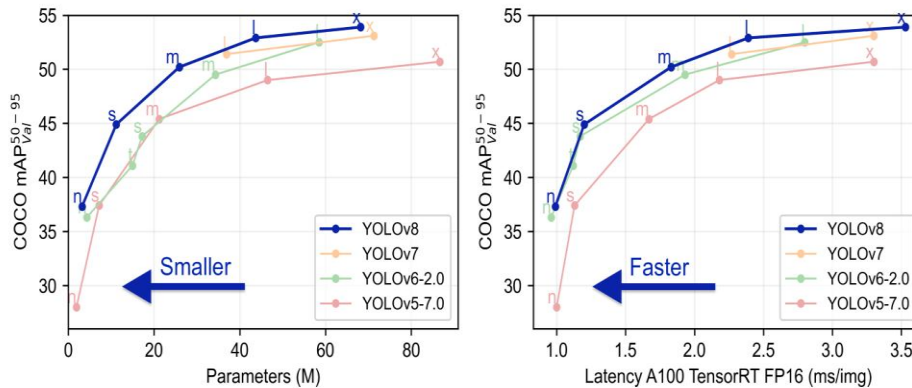


Figure 28: YOLO Comparison Charts

Looking at the comparison charts, *Figure 17*, YOLOv8 demonstrates superior performance in terms of mean Average Precision (mAP), with values reaching closer to 50-55% across various parameter settings. It is, however, more computationally intensive, as reflected by its higher number of parameters and latency requirements. For systems not limited by hardware, YOLOv8 could potentially offer substantial accuracy improvements, making it an appealing option in cases where computational resources are abundant.

On the other hand, YOLOv5, while achieving slightly lower mAP (around 35-40% on average), has a significantly smaller parameter size, which directly translates to faster processing times and lower latency. This characteristic allows the model to handle real-time object detection tasks effectively in resource-constrained environments, such as those typically encountered in field operations where drones may have limited onboard processing capabilities.

Given these trade-offs, YOLOv5 was chosen for its balance of computational efficiency and detection accuracy. The decision also provides room for future upgrades: once higher computing power becomes available, transitioning to a more advanced model like YOLOv8 or higher can be done to enhance the accuracy and capability of the system without worrying about the additional computational overhead.

Object Detection

One of the core aspects of computer vision is object detection, which allows the drones to identify specific objects or persons within their environment. Object detection involves recognizing instances of objects from a particular class (such as vehicles, people, or debris) within an image or video stream. In autonomous systems, this capability must be performed in real time, as delays in processing could result in incorrect navigation or mission failure. Various algorithms and machine learning models are applied in these systems to ensure that object detection is both fast and accurate, even in dynamic or unpredictable environments.

The software side of this system primarily involves the use of advanced object detection algorithms trained on extensive datasets. One prominent dataset used for training such models is the Common Objects in Context (COCO) dataset, which contains millions of images labeled with annotations for object detection tasks. Models trained on the COCO dataset are highly adept at recognizing a wide variety of objects in real-world environments. These pre-trained models can be further fine-tuned with additional data to improve their performance in specific applications, such as search and rescue.

Challenges With Computer Vision

LiDAR

The integration of LiDAR also presents new opportunities and challenges. LiDAR data, though extremely accurate for depth perception, requires specialized algorithms to interpret and integrate with visual data effectively. Future work will explore sensor fusion techniques to combine LiDAR and camera-based data, enhancing detection accuracy by providing both shape and distance information. This fusion will be especially useful for detecting obstacles and improving navigation in cluttered environments.



Figure 29: YOLO and Aerial View Image

The drones leverage object detection algorithms like the YOLO model, specifically YOLOv5. YOLO is known for its speed and accuracy, as it processes the entire image in a single pass through the neural network, unlike other models that scan the image multiple times. However, while YOLOv5 has been trained extensively on ground-level images, such as those typically found in urban settings, aerial-view object detection presents a unique set of challenges. When testing the model, it was discovered that it struggled with identifying objects from above like in *Figure 18*. The angle,

altitude, and perspective of the drone's cameras significantly differ from typical ground-based views, which can affect the accuracy of object detection. Therefore, additional training of the YOLO model with aerial imagery is required to optimize its performance for detecting objects from various heights and angles in real-time during search and rescue missions. This retraining allows the model to better understand the shapes, sizes, and perspectives of objects as seen from above, significantly improving detection accuracy in real-world scenarios.

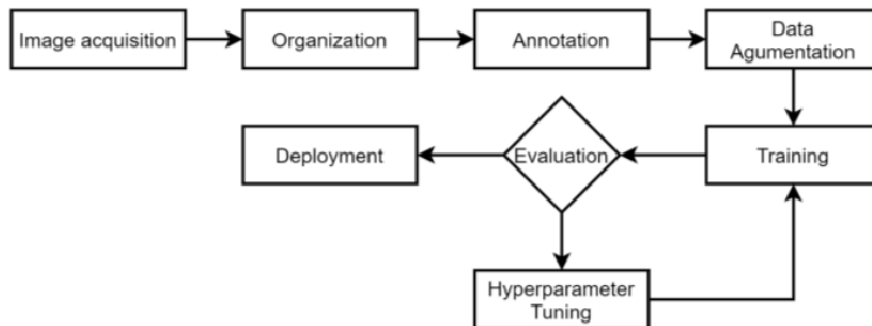


Figure 30: YOLO Training Flowchart

To address the challenges of using YOLOv5 for aerial object detection, the solution lies in the systematic process of training the model, like in *Figure 19*, with a custom dataset that is tailored specifically to the unique characteristics of aerial imagery. This process begins with image acquisition, where aerial images from various altitudes, angles, and environments are collected to form the foundation of the dataset. Following this, the images undergo organization, ensuring that they are categorized and arranged for optimal processing during model training.

The next step in the pipeline is annotation, where objects in each image are manually labeled to provide the YOLO model with the necessary ground truth for training. These annotations are crucial, as they define the objects the model is expected to detect during real-world drone operations. After annotation, the data is enhanced through data augmentation techniques. This process introduces variations to the dataset, such as rotations, zooms, and lighting adjustments, to simulate different real-world conditions that drones might encounter in various environments. Augmentation helps to ensure that the model generalizes well to unseen data and improves detection accuracy across a broader range of scenarios.

Once the dataset is prepared, the YOLO model is ready for training. During this phase, the model is fine-tuned on the aerial dataset, optimizing its weights to accurately detect objects from unconventional angles and perspectives typical of drone-based imagery. After the initial training, the model undergoes evaluation, where its performance is assessed on a validation set to determine its accuracy and speed. If the evaluation reveals areas where the model's performance can be improved, hyperparameter tuning is employed. This involves adjusting parameters such as learning rates, batch sizes, or the number of training epochs to enhance the model's ability to learn from the dataset. Once the model achieves the desired level of accuracy, it can be deployed for use in drone operations, where it performs real-time object detection during search and rescue missions. This iterative process, as depicted in the flow chart, ensures that the YOLO model is thoroughly refined and optimized for aerial object detection tasks. By carefully following these steps, the system can handle the complexities of aerial imagery, ultimately improving object detection accuracy in real-world scenarios.

Use of YOLO on Non-RGB Sensors

On top of the previous challenges, this project introduces additional complexities due to the integration of infrared (IR) cameras alongside standard RGB sensors. Object detection in infrared imagery poses unique challenges, primarily due to the distinct format and characteristics of IR data. Unlike RGB images, which are composed of three channels corresponding to red, green, and blue light, infrared images typically consist of a single channel that represents heat signatures. These heat signatures are crucial for detecting objects that may be invisible or obscured in the visible light spectrum, such as during nighttime or in conditions with heavy smoke, fog, or other visual obstructions. However, the challenge arises when attempting to apply RGB-trained object detection models, like YOLOv5, to this single-channel IR data, as these models are built to process three-channel inputs.

Fortunately, there is a straightforward and widely adopted solution to this issue: replicating the single-channel IR data across three channels to mimic an RGB format. This method involves duplicating the infrared data into the red, green, and blue channels of an image, effectively creating a pseudo-RGB image that can be fed into models originally designed for RGB object detection. By doing this, the neural networks trained on RGB images can still be utilized without significant modification to their architectures. While the underlying data remains infrared-based, the three-channel replication allows the model to process the imagery as if it were a standard RGB input, unlocking the potential of pre-trained object detection algorithms without the need for extensive retraining.

However, while this method offers a practical solution, it is not without its limitations. Replicating the infrared data into three channels can sometimes result in a loss of nuanced information, as the network may not fully exploit the unique characteristics of thermal imagery. Additionally, subtle variations

in heat signatures might be overlooked when processed in this format, which could reduce the overall precision of detection in certain edge cases.

Nonetheless, this technique remains highly effective in most scenarios, especially in environments where low-light conditions or visual impairments, like smoke or heavy foliage, make heat detection the most reliable form of object identification. It allows the drone swarm to maintain robust object detection capabilities while minimizing the need for complex, specialized models, thereby ensuring computational efficiency.

As the project advances, more advanced approaches could be explored to handle infrared data more natively. For example, developing custom models or fine-tuning pre-trained networks specifically for IR data could further enhance detection accuracy. Alternatively, multimodal fusion techniques, where both RGB and infrared data are used together, may provide even more comprehensive environmental awareness, combining the strengths of both visible and heat-based object detection. Nonetheless, the current approach of adapting the infrared input for RGB-based models provides a strong foundation for the drones' object detection in a wide variety of challenging environments.

Integration of AI and Decision-Making

The real-time nature of the object detection process is paramount. As the drones traverse through disaster-stricken areas, they must quickly and accurately identify objects that may be of interest, such as individuals in need of rescue, dangerous debris, or structural damage. To achieve this, the object detection model must process video feeds at high frame rates, allowing for smooth, uninterrupted interpretation of the drone's surroundings. This capability is especially important in time-sensitive missions, where delays in identification could result in the loss of life or further damage to the environment.

The use of computer vision in this project also extends beyond simple object detection. Drones must be able to interpret their surroundings in a more complex manner, identifying paths, determining the safest routes, and avoiding obstacles such as trees, buildings, or debris. To achieve this, the AI agent within each drone processes vast amounts of visual data and applies it to decision-making algorithms. By continuously analyzing the drone's environment, the AI agent can autonomously navigate the terrain and adapt to new obstacles as they appear.

Collaboration between hardware and software components is essential for the successful operation of this system. The sensors gather raw data, which is then processed by the AI algorithms to make informed decisions. For instance, an IR sensor may detect a heat signature indicative of a human, prompting the drone to direct its EO camera to the area for visual confirmation. Once confirmed, the object detection model identifies the human and relays the information back to the swarm's central processing unit, which then coordinates a response.

YOLO in the Drone Swarm

Each drone in the swarm is equipped with the YOLO object detection model, enabling it to independently identify and flag objects of interest such as people, vehicles, buildings, tents, and clothing. Once detected, these objects are tagged with relevant metadata, including GPS coordinates and timestamps, and transmitted to a ground station for further processing. This decentralized detection system allows the swarm to cover more ground efficiently, with every drone acting as both a sensor and data provider, ensuring no critical information is overlooked.

The ground station is equipped with a Large Language Model (LLM) to analyze and synthesize the information provided by multiple drones. As the

drones continuously scan the environment, the LLM integrates the collected data to form a comprehensive situational overview. For example, when multiple drones detect the same object or cluster of objects—like tents near discarded clothing—the LLM can infer the presence of survivors and suggest an optimized search route or recommend deploying more drones to that area.

The YOLO model's ability to perform real-time object detection ensures that the drones can provide immediate updates, which are essential in time-sensitive rescue operations. The fast transmission of detected objects to the ground station allows the LLM to generate accurate, dynamic insights as the environment changes. As new data streams in, the LLM helps prioritize flagged objects based on urgency—such as identifying people or potential hazards—and continuously refines the search strategy based on the latest findings.

This framework ensures seamless communication between the drones and the ground station, where the LLM functions as a mission coordinator. The LLM uses the aggregated data to direct drones toward high-priority zones or alert ground responders to specific points of interest. For example, if drones detect a building collapse, the LLM can direct the swarm to focus on that area while advising ground responders on safe access routes identified by the drones.

By leveraging YOLO's object detection capabilities and a real-time communication network, the swarm remains agile and responsive to new developments. This integration of computer vision and advanced data management ensures that each drone contributes meaningfully to the mission, enhancing the overall effectiveness of the search and rescue operation.

Results

The integration of computer vision technologies is central to the autonomy and effectiveness of the drone swarm in this project. By leveraging sensors such as RGB, infrared (IR), and electro-optical (EO) cameras, along with the YOLOv5 object detection model, the drones can operate independently in diverse and dynamic environments. These technologies enable the identification and classification of critical objects, such as people, vehicles, and structures, ensuring that each drone contributes meaningfully to search and rescue missions. The primary focus on RGB-based object detection, supplemented by IR and EO sensors, ensures that the swarm can maintain operational efficiency in both ideal and challenging conditions.

The decision to implement YOLOv5 reflects a careful balance between computational efficiency and detection accuracy, meeting the constraints of limited onboard hardware while ensuring real-time performance. As each drone flags objects of interest, a ground station equipped with a Large Language Model (LLM) synthesizes the data, providing coordinated and adaptive mission strategies. This seamless communication between the drones and the ground station enables effective collaboration, allowing the swarm to respond dynamically to new information as the mission unfolds.

Looking ahead, future development will explore the full potential of IR, EO, and LiDAR cameras to complement the RGB-based detection system. While the current approach adapts infrared data for RGB-trained models by replicating single-channel IR data across three channels, future work may focus on developing custom models that natively process IR and EO imagery. Advanced models such as YOLO-TIR or specialized convolutional neural networks (CNNs) for thermal imagery could be explored to better capture the unique characteristics of heat signatures. Additionally, multimodal fusion techniques that combine RGB, IR, and EO data will offer a

more comprehensive understanding of the environment, reducing false positives and improving detection accuracy under difficult conditions.

The computer vision system's modular design provides flexibility for these future enhancements. As the project progresses, iterative improvements to both the hardware and algorithms will ensure that the swarm remains adaptive and capable of meeting the evolving demands of search and rescue operations. With these advancements, the drone swarm will not only achieve higher precision and efficiency but also unlock new possibilities for operating in extreme environments, ultimately enhancing the impact of autonomous rescue missions.

Computer Vision to LLM

Introduction

Integrating YOLO's object detection capabilities with a Large Language Model (LLM) such as LLAMA represents a crucial step in enabling the autonomous drone swarm to interpret its environment effectively and make informed decisions. This integration demands a seamless transfer of data between YOLO, which identifies objects in images, and the LLM, which contextualizes and analyzes that data to guide decision-making. The process involves not only transferring raw outputs but also ensuring that the data is formatted and structured in a way that LLAMA can process effectively. This section outlines the challenges, considerations, and potential methods for designing and implementing this data transfer.

At the core of this integration is the need to translate YOLO's raw outputs—object labels, bounding box coordinates, and confidence scores—into a format that can be understood by LLAMA. Unlike YOLO, which outputs numerical and categorical data optimized for machine interpretation, LLMs process natural language or structured input to generate contextual insights.

This difference in data representation necessitates a translation layer that bridges the gap between computer vision and natural language processing. Furthermore, the system must ensure that this translation happens efficiently and in real-time, enabling the drone swarm to react to dynamic environments without delay.

The first consideration is the format of the data. YOLO's output must be encapsulated in a structure that preserves its integrity while being flexible enough for the LLM to interpret. Formats such as JSON, plain text, or XML are commonly used, each with its advantages and trade-offs. JSON is particularly suited for this task due to its ability to represent hierarchical and structured data while maintaining compatibility with most programming languages and LLM APIs. For simpler applications, plain text may suffice, especially if the task requires basic descriptions of detected objects. However, for complex scenarios that involve multiple objects and metadata, structured formats like JSON or XML provide a robust solution.

Another critical aspect is the mechanism by which the data is transmitted from YOLO to the LLM. Various approaches, including file-based communication, APIs, WebSockets, and message queues, can facilitate this transfer. For real-time applications, WebSockets or APIs provide low-latency communication, ensuring that YOLO outputs are delivered to the LLM as soon as they are generated. On the other hand, file-based communication, while simpler to implement, may introduce latency and is more suited for batch processing. Regardless of the method chosen, the priority is to maintain the speed and reliability required for the drones to operate autonomously in dynamic and potentially life-critical situations.

This section will explore these aspects in greater detail, focusing on the optimal data format and transfer mechanisms for the project. It will also address challenges such as latency, data synchronization, and the scalability

of the solution as the drone swarm grows in size and complexity. By effectively bridging YOLO and the LLM, the system can achieve a higher degree of autonomy, enabling the drones to operate collaboratively and intelligently in search and rescue missions.

Data Format Design

The choice of data format is foundational to the successful integration of YOLO and the LLM. It defines how the object detection outputs from YOLO such as object labels, bounding boxes, and confidence scores are structured and conveyed to the LLM for further processing. Several options, including JSON, plain text, and XML, are available, each with unique advantages and limitations.

JSON is particularly well-suited for this project due to its flexibility and widespread adoption in data exchange. Its ability to represent nested structures makes it ideal for scenarios where multiple objects, each with associated metadata, must be transmitted simultaneously. For example, a single JSON file can encapsulate details such as frame number, timestamp, GPS location, and an array of detected objects with their respective attributes. This format ensures clarity and consistency, making it easier for the LLM to parse and analyze the data. Furthermore, JSON's lightweight syntax reduces overhead in transmission, which is critical for real-time applications.

In contrast, although plain text offers simplicity and human readability, which can be advantageous during development and debugging. A textual representation of YOLO outputs, such as "Frame 12: Detected person at () with 91% confidence," allows for quick comprehension without requiring specialized tools. However, this simplicity comes at the cost of flexibility, as plain text lacks the ability to represent complex relationships or nested data efficiently.

XML, while similar to JSON in terms of structure, is more verbose and less commonly used in modern applications. Its extensive tagging system can increase file sizes and processing overhead, making it less suitable for real-time systems with bandwidth constraints. Nonetheless, XML may still be considered in environments that prioritize compatibility with older systems or tools.

The choice between these formats depends on the specific needs of the system. JSON's balance of flexibility, clarity, and compatibility makes it the preferred choice for this project. By encoding YOLO outputs in JSON, the system ensures that the LLM receives the data in a structured, interpretable manner, enabling it to generate meaningful insights and decisions efficiently.

Data Transfer

Beyond selecting a suitable data format, the mechanism by which YOLO's outputs are transmitted to LLAMA plays a critical role in maintaining the system's responsiveness and reliability. Several options exist, each with unique strengths and trade-offs, depending on the operational requirements of the drone swarm.

File-Based Communication

One of the simplest methods involves writing YOLO outputs to files, such as JSON or text files, which the LLM reads periodically. This approach is easy to implement and debug, as the files can be inspected directly to verify the data. However, the file I/O operations introduce latency, making this method unsuitable for real-time applications. It is best suited for scenarios where periodic updates or batch processing are sufficient.

APIs

Using REST or GraphQL APIs provides a more dynamic and flexible mechanism for data transfer. In this approach, YOLO sends detection data to an API endpoint hosted by the LLM or a middleware service. APIs allow for

low-latency communication and can manage dynamic input sizes, making them well-suited for real-time operations. Additionally, APIs can incorporate authentication and error-handling mechanisms to ensure secure and reliable data exchange. However, setting up and maintaining API infrastructure requires additional development effort.

WebSockets

For scenarios that demand continuous and bidirectional data exchange, WebSockets offer an ideal solution. Unlike APIs, which follow a request-response model, WebSockets establish a persistent connection between YOLO and the LLM. This allows for real-time streaming of detections, ensuring minimal latency in processing. For example, each YOLO detection can trigger an immediate update to the LLM, which can then provide near-instant feedback or adjustments to the drones. While WebSockets are more complex to implement than APIs, their efficiency makes them indispensable for highly dynamic systems.

Message Queues

In systems where multiple drones are feeding YOLO outputs to a central LLM, message queues like RabbitMQ, Kafka, or Redis provide a scalable solution. These systems decouple the producer (YOLO) and consumer (LLM), allowing for asynchronous data transfer. Message queues are particularly advantageous in handling high data volumes, ensuring that all messages are processed without overwhelming the LLM. This approach also allows for prioritizing critical detections, such as those involving human survivors, over less urgent data.

Shared Memory

This option only became a possibility during the time of this writing, as Ollama recently introduced support for Nvidia Jetsons, which form the core system architecture for the drones and the ground station. For onboard

systems where YOLO and the LLM operate on the same hardware, shared memory presents the fastest data transfer mechanism. This approach eliminates the need for intermediate files or network communication by allowing YOLO outputs to be stored directly in a shared memory space accessible to the LLM. The result is near-instantaneous data exchange, making shared memory ideal for real-time processing in resource-constrained environments.

However, this method has its limitations. It is restricted to tightly coupled systems where YOLO and the LLM coexist on the same device. While it offers unparalleled speed and efficiency, it lacks the flexibility of other methods like WebSockets or message queues, which are better suited for distributed systems. Despite this, the recent support for Nvidia Jetsons makes shared memory an attractive option for onboard processing in this project, particularly for scenarios where low-latency decision-making is critical.

Prompt Engineering

Prompt engineering is the next crucial step to integrating the YOLO model with the LLM, as it defines how object detection outputs are presented for interpretation. The prompt serves as the bridge between the structured numerical data generated by YOLO and the natural language capabilities of the LLM, guiding it to produce actionable insights relevant to the drone swarm's objectives. This process requires careful design to ensure clarity, relevance, and scalability, enabling the LLM to make effective decisions in real-time scenarios.

At the heart of prompt engineering is the challenge of contextualizing YOLO outputs for the LLM. The LLM processes data in a text-based or structured format, which contrasts with YOLO's output of bounding box coordinates, confidence scores, and class labels. For instance, YOLO might detect a "person" at coordinates (34, 45) with 91% confidence. While this raw data is

informative, it must be reformatted into a meaningful description for the LLM to generate valuable recommendations. A well-crafted prompt might look like: "YOLO has detected a person at coordinates (34, 45) with 91% confidence. Based on this information, recommend the next action for the drone swarm." This simple yet specific format provides the LLM with both the data and the context it needs to respond effectively.

The structure of a prompt plays a critical role in its success. Prompts typically consist of three main components: context, data summary, and instructions. The context introduces the scenario, setting the stage for the LLM's interpretation. For example, the context might state, "The drone swarm is conducting a search and rescue mission in a disaster zone." This informs the LLM of the overarching goal and frames its subsequent responses. Next, the data summary translates YOLO's outputs into a concise and structured format. An example would be: "Detected objects include: 1 person at (34, 45) with 91% confidence and 2 vehicles at (150, 75) and (300, 400) with 87% and 82% confidence, respectively." Finally, the instructions clarify the task the LLM is expected to perform, such as, "Based on this data, suggest the next optimal flight path or adjustments to the swarm's strategy."

Static prompts, while useful during the initial stages of development, are insufficient for real-time operations. In dynamic environments, the system requires prompts that adapt to changing conditions and mission priorities. Dynamic prompt generation is essential, as it allows the system to programmatically create prompts based on incoming data. For example, a drone detecting multiple objects in a densely populated area would require a prompt emphasizing prioritization: "Focus on the person detected at (34, 45) with 91% confidence and suggest a path to approach while avoiding the vehicle at (150, 75)." This adaptability ensures that the LLM remains responsive to the immediate needs of the mission.

Despite its benefits, prompt engineering comes with challenges. One significant issue is data overload. When multiple drones simultaneously feed their YOLO outputs into the system, the sheer volume of data can overwhelm the LLM, leading to slower responses or reduced accuracy. To address this, the prompts must prioritize critical information, such as detections of people or potential hazards, while deferring less urgent data. Ambiguity in prompt design is another common challenge. Vague instructions or poorly structured data can lead to irrelevant or incorrect responses from the LLM. For instance, failing to specify the spatial relationships between objects might result in the LLM generating a non-optimal flight path. Additionally, there is a trade-off between providing detailed information and maintaining the LLM's processing speed, particularly in time-sensitive missions.

Iterative testing and refinement are essential to overcoming these challenges. By conducting experiments with different prompt formats and analyzing the LLM's outputs, weaknesses can be identified, and improvements can be quicker to implement. Real-world simulations are particularly valuable, as they reveal how the system performs under dynamic and unpredictable conditions. This iterative process ensures that the prompts consistently elicit accurate and actionable responses from the LLM, enhancing the overall efficiency of the drone swarm.

In conclusion, prompt engineering is a dynamic and iterative process that bridges the gap between YOLO and the LLM. By designing prompts that are clear, contextual, and adaptive, the system can ensure effective communication between these components, enabling the drone swarm to operate autonomously and intelligently in complex search and rescue missions as the project progresses, further refinements in prompt engineering will play a pivotal role in optimizing the integration and performance of the LLM within the system.

Workflow Summary

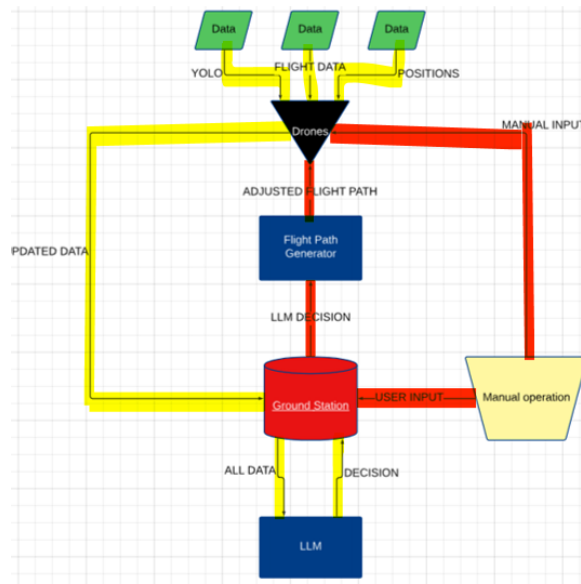


Figure 31: General overview of the workflow of the project.

The integration of YOLO and the LLM within the drone swarm hinges on a well-orchestrated workflow encompassing data format, data transfer, and prompt engineering (highlighted in yellow in *Figure 31*). These stages collectively enable the seamless transformation of raw object detection outputs into actionable insights that drive real-time decision-making.

The data format stage ensures that YOLO's outputs, such as object labels, bounding box coordinates, and confidence scores, are structured into a standardized representation like JSON or plain text. This step establishes a consistent foundation for further processing by organizing raw detections into a comprehensible and reliable format. The structured data captures essential information while preserving its integrity for downstream use.

Data transfer mechanisms then serve as the critical link between YOLO and the LLM, ensuring that formatted outputs reach the appropriate system component in an efficient and timely manner. Whether through WebSockets for real-time streaming, APIs for dynamic communication, or shared memory for onboard systems, the transfer mechanism is designed to handle the constraints and demands of a distributed drone swarm. This stage emphasizes reliability, scalability, and low latency, allowing the system to remain responsive during time-sensitive missions.

Finally, prompt engineering bridges the gap between structured machine outputs and the LLM's natural language processing capabilities. By contextualizing YOLO's outputs into clear, actionable prompts, this stage transforms numerical data into inputs that the LLM can interpret and reason about. Dynamic prompts adapt to evolving mission conditions, ensuring the LLM receives up-to-date and relevant information for generating insights or adjusting swarm behavior.

Together, these stages form a cohesive pipeline that transforms object detection data into meaningful action. The workflow ties the precision of computer vision with the reasoning power of the LLM, enabling the drone swarm to collaborate, adapt, and respond dynamically in complex environments. This integration not only enhances the swarm's autonomy but also lays a strong foundation for further advancements in real-time communication, data interpretation, and mission coordination. By iteratively refining each stage, the system can continue to improve, ensuring greater effectiveness in search and rescue operations.

Swarm Communication

Introduction

Effective communication between drones in a swarm is the backbone of any collaborative autonomous system, especially in complex operations such as

search and rescue. In such missions, drones must share data, coordinate movements, and adjust their strategies in real time to ensure seamless coverage of the area and efficient use of resources. Communication in a drone swarm is not merely about transmitting information but also about enabling collective decision-making, maintaining situational awareness, and responding dynamically to new developments.

This project's swarm of drones is designed to operate in challenging environments, where obstacles, limited connectivity, and bandwidth constraints can hinder communication. Each drone, equipped with unique sensors like LiDAR, infrared (IR) cameras, or electro-optical (EO) cameras, plays a specific role in gathering and interpreting data. These drones must collaborate to provide a comprehensive understanding of the environment, transmitting their findings to the ground station and receiving updated instructions based on the combined outputs of the swarm.

To achieve this level of coordination, the communication system must manage a range of tasks. These include sharing object detection data from the YOLO model, synchronizing positional information to avoid collisions, and receiving commands from the ground station informed by the LLM. The system must balance real-time responsiveness with robustness, ensuring that bundles of data get sorted by priority, and that communication remains operational even in the face of network disruptions.

This section explores the forms of communication necessary to enable this level of coordination, examining how drones share information among themselves and with the ground station. It will also address the challenges inherent in maintaining reliable communication in dynamic and potentially hostile environments, as well as the technological solutions that can overcome these barriers.

Communication Protocols

Communication protocols are fundamental to enabling the seamless exchange of information within the drone swarm and between the drones and the ground station. These protocols define how data is transmitted, prioritized, and synchronized, ensuring that the system can function efficiently in real-time scenarios. For the drone swarm to operate cohesively, the communication protocols must support various tasks such as sharing YOLO detection outputs, coordinating flight paths, and maintaining positional awareness, all while adapting to environmental and operational challenges.

Inter-drone communication is vital for fostering collaboration within the swarm. By sharing data such as positional coordinates, detected objects, and status updates, drones can dynamically adjust their operations to cover ground more effectively and avoid collisions. Protocols such as Mobile Ad Hoc Networks (MANETs) allow drones to communicate directly without relying on a fixed infrastructure, creating a decentralized system that can adapt as the drones move. In situations requiring high data transfer rates, Wi-Fi Direct offers peer-to-peer connectivity, enabling drones to exchange critical information quickly. For environments with lower power requirements, Bluetooth Mesh provides a scalable and energy-efficient alternative. Additionally, custom radio frequency (RF) protocols can be employed to ensure robust communication in areas with limited network coverage, such as remote or disaster-stricken regions.

The connection between the drones and the ground station is equally important, as the ground station serves as the central hub for processing and decision-making. Cellular networks, such as 4G or 5G, provide reliable long-range communication, with 5G offering low-latency and high-bandwidth capabilities that are particularly suited for transmitting YOLO data and sensor outputs in real time. For localized operations, a dedicated Wi-Fi network can facilitate fast and dependable data transfer to the ground station. In scenarios where low-bandwidth communication is sufficient,

protocols like LoRa (Long Range) offer an efficient option for transmitting telemetry or status updates. Satellite communication may also be considered for operations in remote areas, although its high latency and cost make it less practical for real-time applications.

Efficient communication requires careful prioritization of data to ensure that mission-critical information gets transmitted without delay. For instance, detections of individuals or potential hazards should get prioritized over routine telemetry updates. Synchronization across the swarm is equally critical, as time-stamping and sequencing messages allow for consistent and coordinated responses among drones. This ensures that the swarm can function as a unified entity, even when operating in dynamic or unpredictable environments.

Fault tolerance is another essential consideration in designing communication protocols. Environmental factors, hardware failures, or network interruptions can disrupt data transmission, so redundancy mechanisms must be incorporated to ensure continuity. For example, packet retransmission or fallback to alternative protocols like LoRa can mitigate the impact of such disruptions. While reliable protocols like TCP ensure data integrity, they may introduce latency; hence, a hybrid approach combining UDP for real-time updates and TCP for critical information can strike a balance between speed and reliability.

Lastly, security is paramount in the communication framework, particularly when sensitive data, such as the location of survivors or mission-critical instructions, is transmitted. Encryption mechanisms, such as end-to-end encryption and secure key exchange, protect the data from unauthorized access or tampering. Authentication protocols further ensure that only authorized drones and ground stations can participate in the communication network, safeguarding the integrity of the swarm's operations. Due to the

time constraint on this project, encryption will likely not be implemented by our team, or at least not in a highly secured manner. This will result in it falling to another team for further enhancement.

The selection and implementation of communication protocols directly influence the system's ability to operate effectively in challenging conditions. By addressing requirements for speed, reliability, prioritization, and security, the drone swarm can achieve the real-time collaboration and adaptability necessary for successful search and rescue missions. These protocols form the backbone of the system, enabling a cohesive and intelligent operation as well as allowing for optimized implementations of search algorithms.

Data Types to Exchange and Prioritization

The success of the drone swarm in search and rescue missions depends on the effective exchange of data between drones and the ground station. The communication system must accommodate various data types, ranging from critical real-time object detections to routine telemetry updates. Ensuring that this data is transmitted and processed in an order that aligns with mission priorities is essential for achieving the swarm's goals and maintaining operational efficiency.

One of the most critical data types is the output from YOLO's object detection model. This includes object labels, bounding box coordinates, confidence scores, timestamps, and GPS coordinates. These outputs allow the drones to identify and localize objects of interest, such as people, vehicles, debris, or hazards. YOLO's outputs form the foundation for the LLM's decision-making process, enabling it to assess the environment and recommend optimal strategies for the swarm.

Positional and telemetry information is another vital data type. Each drone must continuously share its GPS coordinates, altitude, speed, and orientation with other drones and the ground station. This information ensures spatial

awareness within the swarm, enabling drones to coordinate their movements and avoid collisions. Telemetry data, such as battery levels and sensor status, is essential for resource management, allowing the system to allocate tasks dynamically based on drone availability and performance.

Environmental data, such as weather conditions, wind speed, and temperature, plays a supporting role in the swarm's operations. This data is used to adjust flight dynamics and ensure safety in adverse conditions. For example, detecting high winds may prompt drones to adjust their altitudes or temporarily halt operations in affected areas. While environmental data is less critical than object detections or positional updates, it remains a crucial factor in mission planning.

Command and control instructions from the ground station complete the data exchange loop (red highlights in *Figure 31*). These instructions, generated by the LLM, include updated flight paths, prioritized search zones, or specific tasks for individual drones. This feedback ensures that the swarm operates cohesively, with each drone contributing to the mission based on the latest information.

To manage this diverse range of data effectively, a priority queue will be required. Critical data such as detections of people, collision avoidance updates, and low battery warnings are given the highest priority and transmitted immediately. Medium-priority data, like other object detections and environmental updates, is processed as resources allow, while routine telemetry and non-critical sensor readings are deprioritized to reduce network congestion. This prioritization ensures that the system remains focused on mission-critical tasks without being overwhelmed by less urgent information.

Dynamic prioritization further enhances the system's adaptability. For instance, in areas with high drone density, collision avoidance data is

prioritized, while in regions with potential survivors, detections of people take precedence. This flexibility allows the swarm to respond effectively to changing conditions and mission requirements.

By defining clear data types and implementing a robust prioritization system, the drone swarm can achieve real-time collaboration and decision-making. The framework ensures that critical information is transmitted and processed first, enabling the swarm to operate efficiently and focus on its primary objectives in search and rescue operations. This structured approach to data exchange is integral to the project's success and scalability.

Communication Summary

The seamless operation of a drone swarm in search and rescue missions relies on a carefully designed framework for data exchange, prioritization, and communication. Effective coordination begins with the identification and structuring of key data types, including YOLO object detection outputs, positional and telemetry data, sensor readings, environmental information, and command and control instructions. Each type of data serves a specific purpose, from enhancing situational awareness to enabling real-time decision-making.

Prioritization plays a pivotal role in managing the continuous flow of information within the swarm and between the drones and the ground station. High-priority data, such as detections of people or collision avoidance updates, is transmitted and processed immediately to support mission-critical actions. Medium-priority information, including additional object detections and environmental updates, provides valuable context, while routine telemetry and non-critical data are deprioritized to optimize resource usage.

Communication protocols function as the backbone of this system, ensuring reliable, low-latency transmission of data across the swarm and to the

ground station. Whether through ad hoc wireless networks, cellular connectivity, or satellite links, the protocols must be robust enough to manage unpredictable environments while supporting redundancy and security. By implementing dynamic prioritization mechanisms and leveraging real-time data processing, the system can adapt to changing conditions and maintain operational efficiency.

Together, these elements—data type identification, prioritization, and communication protocols—form an integrated framework that enables the swarm to function as a cohesive and intelligent unit. This structure ensures that critical information is shared and acted upon effectively, allowing the drones to collaborate, adapt, and make decisions in complex and time-sensitive environments. As the project evolves, refining these mechanisms will further enhance the swarm's capabilities and ensure success in search and rescue operations.

Drone Testing Regulations and Legal Framework near UCF, Florida

Operating drones near the University of Central Florida (UCF) involves understanding a complex set of federal, state, and local regulations. These rules are designed to ensure safety, protect privacy, and encourage innovation while maintaining accountability for drone operators. For UCF, a hub for research and innovation, these regulations are particularly important because of unique location of UCF close to several government entities.

The UCF area is uniquely positioned as an ideal location for drone testing. Its proximity to Orlando International Airport introduces controlled airspace considerations, and its huge student population and residential buildings make safety and privacy crucial. Meanwhile, UCF's reputation as a leading research university opens opportunities for groundbreaking developments in

drone technology, particularly in artificial intelligence, environmental monitoring, and disaster recovery.

Drone technology plays a critical role in modern research and commercial operations. In Florida, drones are used to monitor wildlife in the wetlands, survey storm damage after hurricanes, and assist in disaster recovery efforts. However, testing drones in a populated and regulated environment like UCF demands strict adherence to federal and state laws. Neglecting these laws can lead to legal penalties, difficulties in research, or damage to an operator's reputation. On the other hand, following these rules not only ensures public trust but also allows for innovation to thrive in a controlled and safe manner.

This section outlines the framework that governs drone testing in Florida, with a focus on the UCF area. By understanding and complying with these regulations, operators can conduct their research and testing safely and legally while contributing to advancements in drone technology.

Federal Regulations: FAA Part 107

The Federal Aviation Administration (FAA) oversees the operation of small unmanned aircraft systems (sUAS) under Part 107. This framework applies to both recreational and commercial drone use, including testing near UCF. Understanding these regulations is essential for safe and compliant operations.

- **Drone Registration:** Every drone weighing more than 0.55 pounds (250g) must be registered with the FAA. Registration ensures accountability, requiring operators to display the registration number on the drone. The process is straightforward: operators register their drones online via the FAA DroneZone platform, paying

a small fee. Failure to register can result in fines and potential confiscation of the drone.

- **Visual Line of Sight (VLOS):** Operators are required to keep their drone within direct visual contact throughout the flight. Advanced testing, such as flying beyond the operator's line of sight (BVLOS), requires a waiver from the FAA. These waivers are particularly important for operations like search and rescue missions, which often involve testing drones in a larger or blocked areas.
- **Altitude and Airspace Restrictions:** Drones cannot fly above 400 feet above ground level (AGL). Additionally, operators near controlled airspace, such as the area surrounding Orlando International Airport, must use the Low Altitude Authorization and Notification Capability (LAANC) system to obtain real-time flight permissions. For example, testing a drone within 5 miles of the airport would require prior authorization through LAANC.
- **Flights Over People:** Under Part 107, drones cannot fly over non-participating individuals unless the operator obtains a special waiver. This rule is particularly relevant near UCF, where outdoor events and crowded areas are common. Operators must carefully plan their flight paths to avoid unintentional violations.
- **Night Operations:** Although Part 107 limits drone flights to daylight hours, twilight operations (30 minutes before sunrise or after sunset) are allowed if the drone is equipped with anti-collision lights. Full night operations require an additional waiver, which involves showing how the operator will maintain safety during reduced visibility.
- **Waivers and Exemptions:** FAA waivers allow operators to bypass specific Part 107 restrictions for advanced testing scenarios. For instance, a researcher testing autonomous drones for SAR missions

may apply for waivers to operate at night, fly BVLOS, or carry external payloads.

Compliance with these federal rules ensures safety and enables operators to conduct innovative testing within legal boundaries.

State Regulations (Florida):

Florida has implemented additional state-specific regulations to address privacy concerns, public safety, and the protection of critical infrastructure. Florida's House Bill 1027 pre-empts local governments from regulating drones, meaning only the state legislature can enact laws governing drone use. However, cities and counties may impose ordinances related to public safety, harassment, or privacy violations.

Key statewide regulations include:

- **Critical Infrastructure:** Drones are prohibited from flying over critical infrastructure such as power plants, water treatment facilities, and communication towers without prior permission. Testing near these areas requires special permits and coordination with authorities to avoid violations.
- **Privacy Concerns:** Florida's Senate Bill 766 prohibits drones from capturing images or videos of individuals on private property where they have a reasonable expectation of privacy. This law applies even when the drone is flying over public airspace. For testing conducted near residential areas or private property around UCF, operators must ensure they are compliant with privacy regulations.
- **State Parks and Forests:** The Florida Administrative Code restricts drone flights in state parks, forests, and other managed lands. Testing conducted in these areas requires special authorization from the

Department of Agriculture, which evaluates the potential impact on wildlife, forest resources, and public safety.

Local Regulations (Orlando):

Although Florida's state laws pre-empt local drone regulations, some municipalities still impose additional restrictions related to public safety and privacy. Near UCF, the following local rules apply:

- **City of Orlando Ordinance:** This ordinance restricts drone flights within 500 feet of city-owned parks, schools, and large venues like the Camping World Stadium and Lake Eola Park. Testing near popular locations that draw large crowds requires special permits. Since UCF often hosts events that gather large groups, operators must plan carefully to avoid violating this ordinance.
- **UCF Campus Drone Policy:** UCF enforces strict rules for drone flights on campus. Unauthorized drone flights are prohibited to protect the safety and privacy of students and staff. Operators must seek approval from UCF's administration before conducting any tests on campus grounds. This process typically involves submitting a formal request, detailing the purpose of the flight, safety measures, and expected duration.
- **Large Events:** Orlando frequently hosts large events, and special permits are required for drone operations during these gatherings. Operators conducting testing near UCF must consider additional safety precautions to ensure that drones can safely operate around large groups.

Special Considerations for Drone Testing

Certain types of drone testing, particularly those related to specialized missions such as search and rescue, require additional permissions and waivers:

- **Search and Rescue (SAR) Operations:** Drones used in SAR missions may qualify for exemptions under FAA's Part 107 regulations. However, operators must still meet basic safety standards, including maintaining visual line of sight or obtaining waivers for BVLOS operations. SAR testing often requires the flexibility of real-world conditions, and operators should ensure they are compliant with all necessary legal requirements.
- **Research and Development Waivers:** For advanced research projects involving artificial intelligence (AI) or machine learning, operators may apply for research and development (R&D) waivers. These waivers provide additional flexibility in testing environments, allowing for the development of cutting-edge technologies such as swarm intelligence, autonomous navigation systems, and real-time object detection.

Drone testing near UCF, Florida, requires strict adherence to FAA regulations, state laws, and local ordinances. Operators must ensure that they follow the required steps to obtain waivers, permissions, and special use authorizations where necessary. By complying with these legal guidelines, drone operations can be conducted safely and within the bounds of the law. For specialized testing, such as search and rescue missions or AI-based research, the necessary exemptions and permits must be sought well in advance to avoid delays.

Drone Swarms for Rescue Missions

Drone swarms are revolutionizing search and rescue (SAR) missions by providing enhanced capabilities that traditional methods cannot match. These swarms can cover large areas quickly, enabling rescuers to locate survivors in otherwise inaccessible or dangerous environments. Equipped with advanced sensors and cameras, drone swarms can detect heat signatures, capture real-time video, and provide situational awareness, all while navigating complex terrains autonomously. In addition to search capabilities, drones can also deliver essential supplies, such as food, water, and medical kits, directly to stranded survivors, bridging the gap until ground teams can reach them. This combination of rapid coverage, real-time data sharing, and the ability to perform tasks autonomously makes drone swarms an invaluable tool for enhancing the effectiveness and speed of rescue operations.

Technical Challenges and Solutions in Drone Swarm Operations

Deploying drone swarms for rescue missions introduces a set of technical challenges that differ significantly from single-drone operations. The complexity of coordinating multiple drones in unpredictable and hazardous environments demands innovative solutions for communication, navigation, power management, and environmental resilience. Addressing these challenges is essential to ensure the effectiveness of drone swarms in disaster scenarios where human intervention is limited or impossible.

- **Real-Time Communication and Coordination:** Effective communication between drones is critical to the success of swarm operations. In a rescue mission, drones must exchange real-time data about their surroundings, detected survivors, and obstacles. One of the biggest hurdles is reducing communication latency to prevent delays that could compromise the mission. Mesh networking is commonly used, where each drone serves as a node in the network,

enabling the swarm to maintain low-latency communication even if some drones lose connectivity. This decentralized approach enhances the reliability of the swarm in environments where traditional communication systems, such as Wi-Fi or cellular networks, might be disrupted.

For instance, during search and rescue missions in remote earthquake zones, drones equipped with mesh networking can create ad-hoc communication networks that relay data back to rescue teams, even in areas where infrastructure is severely damaged. However, challenges remain, such as managing bandwidth when many drones are transmitting simultaneously. Future advancements, like 5G-enabled drones, could further minimize latency and support higher data volumes, making communication even more robust.

- **Autonomous Navigation:** Search and rescue missions often take place in challenging environments, such as collapsed buildings, dense forests, or flooded regions. Autonomous navigation is key to ensuring that drones can operate effectively without constant human intervention. Technologies like LIDAR and stereo vision sensors are instrumental in mapping surroundings in real time, allowing drones to avoid obstacles and identify pathways to survivors. Algorithms like Simultaneous Localization and Mapping (SLAM) enable drones to continuously update their understanding of the environment as they move, ensuring accurate navigation even in dynamic conditions.

For example, drones deployed in post-disaster urban environments use SLAM algorithms to navigate through debris-strewn streets and collapsed structures. These algorithms also allow drones to collaborate with each other, creating a shared map of the area to optimize search coverage. However, advancements are still needed to improve the

precision of these systems, particularly in low-visibility conditions or environments with high dust or smoke levels.

- **Battery Life and Range:** One of the most significant technical barriers in rescue missions is ensuring that drones have sufficient battery life to cover large search areas. Many rescue operations demand long-duration flights, which strain the capacity of current battery technologies. Solutions include the use of hybrid drones that combine electric and gas-powered systems, extending flight times while maintaining maneuverability. Another approach involves establishing field battery swap stations, where drones can autonomously land, replace depleted batteries, and resume operations with minimal downtime.

For instance, in large-scale search efforts, such as those following a hurricane, strategically placed battery swap stations enable drones to operate continuously over wide areas without requiring human intervention. Additionally, research into wireless charging technologies, where drones can recharge mid-flight using electromagnetic fields, could revolutionize rescue operations by eliminating the need for frequent battery replacements.

- **Environmental Factors:** Rescue missions often take place in extreme weather conditions, such as high winds, heavy rain, or extreme temperatures. Drones used in these scenarios must be designed to withstand harsh environments while maintaining stability and functionality. Weatherproofing materials, enhanced propeller stability, and onboard temperature regulation systems are critical to ensuring operational reliability.

Testing drones in simulated extreme environments has proven invaluable in identifying vulnerabilities. For example, drones tested for wildfire operations are exposed to high temperatures and dense smoke to evaluate their performance under such conditions. Additionally, advancements in aerodynamic designs and reinforced materials help drones maintain flight stability in high winds, making them more effective in challenging environments like coastal hurricane zones.

By addressing these challenges through technological innovation and rigorous testing, drone swarms can become reliable tools for search and rescue operations. Their ability to adapt to unpredictable environments and operate autonomously makes them invaluable assets in disaster-stricken areas where speed and efficiency are critical.

Drones in Search and Rescue (SAR) Missions

Drone swarms are transforming the landscape of search and rescue (SAR) missions by having range of capabilities that significantly improves operational efficiency. These capabilities extend from fast area coverage to delivering supplies, making drone swarms very beneficial in modern SAR efforts.

- **Wide-Area Search Capabilities:** Drone swarms can be rapidly deployed to scan large areas, particularly in inaccessible regions where ground-based rescue teams would struggle. In situations like forested terrains, mountainous areas, or post-disaster urban environments, drones equipped with high-resolution cameras and infrared (IR) sensors can detect heat signatures from survivors who are trapped or stranded. The collective power of the swarm allows for faster area

coverage than traditional rescue methods, drastically reducing the time it takes to locate survivors, in critical situations.

- **Real-Time Situational Awareness and Mapping:** Another significant advantage of drone swarms is their ability to provide real-time situational awareness. During a mission, drones can relay live video feeds and sensor data back to a central command station, where authorities can make informed decisions about resource deployment. Drone swarms can also be used to create 3D maps of the affected area. These maps give ground teams critical information about terrain, structural damage, and potential hazards, guiding them in making safe and efficient decisions during the rescue effort.
- **Supply Delivery to Inaccessible Areas:** In some rescue scenarios, survivors may be trapped in areas that are unreachable by ground teams. Drone swarms equipped with payload delivery mechanisms can transport essential supplies such as water, food, or medical kits directly to survivors, providing immediate aid while rescue teams work to reach them. These payload drones can autonomously fly to GPS coordinates, drop supplies, and return for additional missions, ensuring a continuous supply flow until rescue teams arrive.
- **Day and Night Operations:** Drone swarms excel in both day and night operations due to their ability to leverage different sensors for different conditions. During the day, drones equipped with standard cameras and environmental sensors can search for survivors. At night, drones with thermal imaging cameras can detect heat signatures, allowing rescue teams to locate survivors in darkness or through smoke during fire-related incidents.

These capabilities make drone swarms an invaluable tool in search and rescue missions, increasing the likelihood of locating and saving survivors in challenging environments.

Safety and Reliability in Drone Swarm Rescue Operations

Safety and reliability are crucial when deploying drone swarms in life-saving rescue operations. Rescue teams need to trust that drones will perform as expected without causing harm to survivors, rescuers, or the environment. Several key mechanisms ensure safety and reliability in swarm operations.

- **Redundant Systems and Fail-Safes:** One of the most important features of drone swarms is the implementation of redundant systems. This means that critical components such as communication systems, power sources, and navigation sensors have backups in place. If a drone loses communication with the swarm or experiences a mechanical failure, it should have fail-safe protocols that allow it to either return to base or perform a controlled landing. This minimizes the risk of drones falling into hazardous areas or causing accidents.
- **Obstacle Avoidance and Collision Prevention:** In environments filled with debris, collapsed buildings, or other obstacles, drones must be able to navigate without colliding with objects or each other. Modern drones are equipped with collision avoidance systems that use a combination of LIDAR, ultrasonic sensors, and computer vision to detect and avoid obstacles. These systems continuously scan the environment and adjust the drone's flight path in real-time, ensuring safe navigation through complex terrains.
- **Emergency Protocols for Communication Loss:** Communication is vital in swarm operations, but there is always a risk of signal loss due to environmental interference or equipment failure. Drones used in rescue operations should have pre-programmed emergency protocols that activate when communication is lost. These protocols

may include returning to the last known safe location, completing predefined tasks autonomously, or hovering in place until reconnected with the central command.

- **Operational Redundancy in Swarms:** In addition to individual drone fail-safes, the swarm should have operational redundancy. If one or more drones in the swarm fail, the remaining drones should be able to dynamically reassign tasks to continue the mission. For example, if a mapping drone goes offline, another drone in the swarm should automatically take over its role without disrupting the overall mission flow.

By ensuring these safety measures are in place, drone swarms can operate with minimal risk, providing reliable assistance during critical rescue operations.

Real-World Applications

The use of drone swarms in real-world rescue operations has demonstrated their immense potential to save lives and improve the efficiency of disaster response efforts. These drones have been deployed in diverse scenarios ranging from urban search and rescue to environmental disasters, proving their adaptability and reliability. Below are case studies showcasing the successful deployment of drone swarms:

- **2017 Mexico City Earthquake:** After a 7.1 magnitude earthquake devastated Mexico City, drone swarms were deployed to assist in the search for survivors among collapsed buildings. These drones provided real-time aerial views of areas inaccessible to ground teams, helping rescue workers pinpoint locations where people were trapped. Their ability to navigate narrow alleyways and debris-strewn zones enabled quicker rescues and a more efficient response.

- **Floods in Germany (2021):** During the catastrophic floods in Germany, swarms of drones were used to search for survivors stranded by rising water levels. These drones mapped flooded areas, provided damage assessments, and located individuals in need of evacuation. They also monitored critical infrastructure such as dams and bridges, helping authorities prioritize repairs and prevent further damage.
- **California Wildfires (2020):** The recent surge of wildfires in California has seen drone swarms play a crucial role in monitoring fire progression, assessing damage, and locating people and animals trapped by the flames. During the 2020 wildfires, drones equipped with thermal cameras were able to see through dense smoke, assisting firefighters in identifying hotspots and navigating dangerous terrain. In some cases, drones also delivered water and medical supplies to civilians cut off by the fires.
- **Australia Bushfires (2020):** Drone swarms played a huge role in wildlife rescue efforts and monitoring fire damage during Australia's catastrophic bushfires. They provided high-resolution imagery of affected areas and identified animals in need of rescue, aiding conservation teams in their efforts to protect endangered species.
- **Beirut Explosion (2020):** Following the devastating explosion in Beirut, drone swarms were deployed to map the affected areas, search for survivors trapped under rubble, and assess structural damage. Their ability to operate in areas deemed unsafe for human responders expedited relief efforts and improved safety for ground teams.
- **Turkey-Syria Earthquake (2023):** In response to the large-scale destruction caused by the earthquake in Turkey and Syria, drones equipped with infrared cameras were used to locate survivors under collapsed buildings. Their swift deployment and real-time data

collection played a vital role in coordinating rescue operations across vast affected regions.

- **COVID-19 Pandemic:** During the COVID-19 pandemic, drones were utilized to deliver medical supplies, monitor public gatherings, and enforce social distancing measures. Drone swarms were particularly effective in densely populated urban areas, reducing the need for human intervention and limiting the risk of virus transmission among first responders.
- **Glacier Monitoring in Greenland:** Drone swarms have been deployed to monitor melting glaciers in Greenland, providing critical data on the effects of climate change. Their ability to operate in harsh weather conditions and cover large areas efficiently has advanced environmental research efforts.
- **Anti-Poaching Efforts in Africa:** In protected wildlife reserves, drone swarms have been used to reduce poaching and monitor endangered species. By covering vast areas in real-time, they have enhanced the efficiency of anti-poaching teams and reduced illegal activity.

These real-world applications highlight the effectiveness of drone swarms in rescue missions, particularly in disaster-stricken areas where speed and coordination are essential for saving lives.

Future Trends in Drone Swarm Technology

The future of drone swarm technology for rescue missions is bright, with continuous advancements being made to improve the capabilities, autonomy, and efficiency of these systems. The following trends will likely shape the next generation of drone swarms used in rescue operations.

- **Self-Healing Drones:** A significant challenge in drone swarm operations is hardware damage caused by collisions, environmental factors, or wear during missions. In critical situations like search-and-rescue (SAR) missions, even minor damage can disrupt operations. Self-healing drones offer a solution, using self-repairing materials such as polymers that respond to heat or ultraviolet light. These materials allow drones to automatically seal small cracks or abrasions during flight, minimizing downtime.

Modular designs are also being developed, enabling drones to replace damaged parts mid-mission using components carried by other swarm members. However, these technologies can increase drone weight and complexity, which may impact flight efficiency. Additionally, adapting self-repair materials to extreme environments remains a challenge. Despite these obstacles, the potential to improve resilience and reduce reliance on human intervention makes self-healing drones a promising future development.

- **Global Standards for Swarm Operations:** As drone swarms are deployed for various purposes, inconsistent airspace rules, communication protocols, and data privacy laws create challenges, especially in multinational operations. Global standards could help resolve these issues by creating unified frameworks for airspace management, communication, and ethical data use.

Organizations like International Aviation Civil Organization (ICAO) or the United Nations could lead efforts to develop international regulatory guidelines, making global operations more efficient. However, reconciling national regulations, ensuring data security, and managing costs are significant obstacles. The development of universal

standards is essential for the ethical deployment and full potential of drone swarm technology.

- **Quantum Communication:** Traditional communication methods, such as radio frequencies and Wi-Fi, face limitations in drone swarm operations, especially as the swarms grow larger. Quantum communication offers a solution by enabling secure, instantaneous data transmission through quantum entanglement, which could eliminate delays and improve synchronization.

Though promising, quantum communication is still in its early stages, with challenges like maintaining quantum states in dynamic environments and the high cost of implementation. Despite these hurdles, as the technology matures, it could revolutionize drone swarm operations, particularly in high-security missions.

- **AI-Powered Search Algorithms:** One of the most promising advancements in drone swarm technology is the development of AI-powered algorithms that enable drones to autonomously search for survivors. These algorithms allow drones to process and analyze large datasets in real-time, detect patterns, and prioritize search areas based on factors such as terrain, weather, and survivor movement patterns.

For example, drones equipped with AI can assess terrain data and use machine learning models to predict the most likely locations of survivors. By continuously learning and adjusting to the situation, these algorithms make drone swarms more efficient and capable of operating in complex, evolving environments. This autonomous capability significantly reduces the need for human intervention, making it invaluable in disaster scenarios where time is critical.

- **Integration with Ground Robotics:** As drone swarm technology continues to evolve, future rescue missions will likely see greater integration with ground-based robots. While drones excel at providing aerial views and real-time data, ground robots are better suited to navigating through debris and confined spaces to directly reach survivors. Combining both aerial and ground technologies will create a more comprehensive and effective rescue solution.

For example, during a search in a collapsed building, drones could provide an overhead map of the area, identifying potential survivors. Ground robots could then be deployed to move through the wreckage, offering a closer look and assisting in physical rescue operations. This integrated approach leverages the strengths of both technologies, improving the speed and success of rescue missions.

The future of drone swarm technology lies in overcoming current limitations with innovations like self-healing drones, global regulatory standards, and AI-powered algorithms. Additionally, the integration of drones with ground robotics and quantum communication promises to enhance coordination, security, and efficiency in large-scale missions. Together, these advancements will unlock the full potential of drone swarms, revolutionizing rescue operations and enabling them to address complex global challenges.

Ethical Implications of Using Drones in Rescue Missions

The increasing use of drone swarms in search and rescue operations presents new ethical questions that must be carefully considered by operators, governments, and rescue organizations. These ethical concerns are especially significant when balancing the need for quick action with the rights of individuals affected by a disaster.

- **Privacy Concerns:** One of the primary ethical concerns with drone usage in rescue missions is privacy. Drones, particularly those equipped with high-definition cameras and infrared sensors, can capture images and data in areas where individuals have a reasonable expectation of privacy. During a rescue mission, a drone might inadvertently fly over private homes or property, capturing images or video of people in vulnerable situations. This could include individuals who are not involved in the rescue mission but whose privacy is still important. Operators must be mindful of this, ensuring that drone footage is used exclusively for mission purposes and handled with care to prevent unnecessary usage.
- **Balancing Privacy and Public Safety:** In emergency situations, the priority is often placed on saving lives, and the invasion of privacy may be justified if it helps locate missing individuals or assess damage in a timely manner. However, there should still be protocols in place to protect sensitive data gathered during these operations. For example, any personal footage that is not relevant to the mission should be discarded or anonymized, and operators should ensure compliance with privacy laws. Balancing privacy with public safety requires a clear ethical framework, and transparency from rescue organizations is essential to maintain public trust.
- **Public Perception:** Public perception of drones plays a crucial role in how they are deployed during rescue missions. While the public may recognize the life-saving potential of drone technology, there is also concern about surveillance, particularly in urban areas where drone use is more visible. Addressing these concerns requires clear communication about the purpose of drone operations, the scope of their use, and how the data collected will be handled. Operators should aim to create public awareness campaigns to educate communities on

the importance of drones in emergency scenarios, building trust and cooperation in times of crisis.

Data Management in Drone Swarm Rescue Missions

Drone swarms are invaluable in rescue missions because of their ability to quickly gather large volumes of data. This data is essential for assessing the situation, identifying survivors, and helping rescue teams make informed decisions. However, managing and processing this data in real time presents significant challenges.

- **Volume of Data:** During a rescue mission, each drone in a swarm collects massive amounts of data, ranging from high-resolution video feeds to environmental sensor data. When multiple drones are involved, the data can quickly become overwhelming. For example, in a large-scale disaster such as a hurricane or earthquake, hundreds of drones may be deployed to scan vast areas, collecting terabytes of information in the process. This includes footage of debris, thermal images of trapped individuals, and topographical data to map out the affected area. Managing this data effectively is critical to ensure that it is to point in the short time frame required for a rescue mission.
- **Real-Time Data Processing:** One of the major challenges of drone data collection is processing the information in real time. Rescue teams need up-to-date information to make decisions on where to deploy resources, and delays in data processing can be costly. To address this, drone operators often rely on edge computing, where data is processed on the drone itself or at a nearby local server before being transmitted back to the command center. This reduces the time needed to analyze the data and ensures that rescue teams can act on the most current information. Machine learning algorithms are also used to filter out irrelevant data, such as background noise or non-

critical images, ensuring that only the most important data reaches rescue coordinators.

- **Data Security:** With such large amounts of sensitive information being transmitted and processed, ensuring data security is very important. During a disaster, personal and geographical information is collected, which, if mishandled, could pose privacy risks or be misused by malicious actors. Drone operators must implement strong encryption protocols to safeguard this data, both during transmission and when stored. Furthermore, access to the data should be restricted to authorized personnel to prevent breaches or unauthorized usage.

Regulatory Challenges in Civilian Use

As drone swarm technology becomes more advanced, its potential for civilian rescue missions grows. However, the scaling of this technology for widespread civilian use presents several regulatory hurdles that must be overcome to ensure the safe and effective deployment of drones in large-scale emergencies.

- **Current Regulatory Landscape:** In most countries, drone regulations are still evolving, with many laws focused on individual drone usage rather than large-scale drone swarms. In the U.S., for example, the Federal Aviation Administration (FAA) requires drones used in rescue missions to adhere to strict guidelines, such as maintaining a visual line of sight and operating below a certain altitude. These rules are essential for safety but can hinder the full potential of drone swarms in rescue missions, where multiple drones may need to operate autonomously across a large area. As drone swarms become more capable, regulatory bodies will need to update these laws to accommodate the unique challenges posed by

swarm operations, such as BVLOS operations and autonomous navigation.

- **Challenges of Scaling Up:** Scaling drone swarm technology from small-scale operations to widespread civilian use introduces several logistical and regulatory challenges. One of the primary concerns is airspace management. In a large-scale disaster, hundreds of drones may be deployed simultaneously, creating a potential conflict with other aircraft such as helicopters or planes. To manage this, regulators will need to establish Unmanned Aircraft Systems Traffic Management (UTM) systems that can track and coordinate drones in real-time, ensuring that they do not interfere with manned aircraft or each other.
- **Coordination with Emergency Services:** Another regulatory challenge is integrating drone swarms with existing emergency services. Many countries have well-established emergency response protocols, such as 911 systems, and introducing drone swarms into this infrastructure requires careful planning. This includes training emergency responders to work alongside drone operators and ensuring that drones can seamlessly communicate with existing command and control systems. Developing standardized protocols for drone usage in emergencies will be essential for ensuring that drones are operated safely, rather than complicate, traditional rescue efforts.

Humanitarian and Environmental Impact

The deployment of drone swarms in rescue missions has a significant impact not only on the speed and efficiency of humanitarian efforts but also on the environment. These impacts must be considered when evaluating the overall benefits of drone technology in disaster response.

- **Humanitarian Impact:** Drone swarms have revolutionized how humanitarian aid is delivered in disaster zones. In the past, relief efforts often faced delays due to the inaccessibility of affected areas. However, drones can bypass blocked roads, damaged infrastructure, and dangerous conditions to deliver food, water, medical supplies, and other essentials directly to survivors. This capability is particularly useful in rural or mountainous regions where traditional aid delivery methods are slow or impossible. Additionally, drones can provide real-time information on the needs of affected populations, enabling aid organizations to tailor their responses to specific requirements, thereby maximizing the efficiency of their operations.
- **Environmental Monitoring During Rescue Missions:** In addition to their direct role in search and rescue operations, drone swarms are invaluable for monitoring environmental changes during disasters. For example, during floods, drones can monitor water levels in real-time, helping authorities predict further flooding and evacuate at-risk populations. In the aftermath of wildfires, drones can track smoke and air quality levels, providing data that is critical for protecting both rescue workers and local residents. Additionally, drone swarms can be used to map out environmental damage, such as oil spills or deforestation, allowing for a quicker and more targeted response.
- **Sustainability of Drone Operations:** While the humanitarian benefits of drones are clear, there are also concerns about the sustainability of drone swarm technology. The production, deployment, and operation of large drone fleets require energy, materials, and resources that have an environmental impact. Many drones rely on lithium-ion batteries, which have a limited lifespan and must be disposed of properly to avoid environmental contamination. Additionally, the energy required to charge and maintain these fleets

can contribute to carbon emissions. To mitigate these impacts, researchers are exploring the development of more energy-efficient drones and alternative power sources, such as solar-powered drones, which could reduce the environmental footprint of drone operations.

Scalability of Drone Swarms in Search-and-Rescue Missions

The growing complexity of modern search and rescue (SAR) missions has amplified the need for scalable and efficient drone swarm technologies. Although drone swarms have proven effective in small-scale operations, transitioning to larger-scale missions introduces a host of challenges. These challenges center around coordination, communication, energy management, and data processing. When addressed properly, however, scalability unlocks immense potential in terms of efficiency, reliability, and coverage, making drone swarms invaluable for SAR applications. This section explores the advantages of scalable systems and the technical hurdles they must overcome, alongside the innovative solutions enabling their growth. It also explores the importance of training for operators using drone swarms, especially for critical moments of disaster, where every second makes a huge difference.

Benefits of Scalability in Drone Swarms

- **Flexibility and Dynamic Task Allocation:** Search and Rescue (SAR) missions are unpredictable, often demanding swift adjustments to plans as unforeseen challenges arise. For instance, encountering new obstacles, shifting victim locations, or evolving environmental conditions can require immediate change in efforts. Scalable drone swarms thrive in these situations due to their ability to dynamically reassign tasks and modify flight paths in real-time, ensuring mission continuity without delays or disruptions. This exceptional flexibility is

achieved through the integration of advanced algorithms that empower the drones to operate autonomously. These algorithms enable drones to analyze incoming data from sensors, identify priority changes, and adapt their roles and trajectories accordingly. For example, if a drone detects a heat signature indicating a potential survivor, it can prioritize that area and redirect nearby drones for additional support, all while the rest of the swarm continues its assigned tasks. This decentralized approach to decision-making not only enhances efficiency but also allows the swarm to respond effectively to rapidly changing scenarios, maximizing the chances of mission success in unpredictable environments.

- **Increased Efficiency and Speed:** Scalable systems significantly improve operational speed by allowing for task parallelization. In SAR missions, where time sensitivity is crucial, the ability to execute multiple tasks simultaneously, such as scanning, victim detection, and obstacle avoidance, reduces the time required to complete a mission. This parallel processing not only enhances speed but also ensures that resources are optimally utilized, maximizing the swarm's overall effectiveness.

Challenges of Scaling Drone Swarms

- **Coordination and Communication Difficulties:** With an increasing number of drones, coordination becomes exponentially more complex. Large-scale swarms require constant communication to exchange vital data, including positional updates, environmental readings, and task statuses. This increased communication load can overwhelm centralized systems, leading to delays or failures in real-time operations.

To address this, decentralized communication models are essential. By

enabling drones to exchange data with their nearest neighbors rather than a central hub, the system reduces bottlenecks and ensures smoother operations. Implementing mesh networking, as suggested earlier, further strengthens communication resilience, as drones can relay information through multiple pathways, minimizing the impact of connection failures. Such decentralized frameworks are not only scalable but also inherently robust, making them ideal for large-scale missions. This will be extremely useful when there are multiple drones being deployed in large numbers.

- **Task Allocation and Load Balancing:** Efficient task distribution is a critical challenge in larger swarms. Uneven allocation can lead to some drones being overburdened while others remain underutilized, reducing the overall efficiency of the system. This imbalance can result in areas being over-searched while others are left unexplored. Modern task allocation techniques, such as particle swarm optimization (PSO) or auction-based algorithms, offer a solution. PSO enables drones to optimize their movements and task distribution based on localized data, ensuring that each drone contributes meaningfully to the mission. Similarly, auction-based methods allow drones to bid for tasks based on their proximity, battery levels, and current workload. These intelligent algorithms dynamically balance the workload, ensuring that all areas are adequately covered and no resources are wasted.

The scalability of drone swarms for SAR missions is both a challenge and an opportunity. While larger swarms offer unparalleled advantages in terms of coverage, redundancy, flexibility, and efficiency, they also require innovative solutions to address coordination, energy management, and data processing challenges. By leveraging decentralized communication, dynamic task allocation, energy-efficient strategies, and edge computing, these challenges

can be effectively mitigated. As drone swarm technology continues to evolve, addressing these scalability challenges will be essential to unlocking their full potential for future rescue operations, disaster relief efforts, and other critical applications.

Workforce Development for Scalable Drone Swarms

As drone swarm technology grows in complexity and capability, search-and-rescue (SAR) missions increasingly depend on skilled operators and maintenance teams. Managing large-scale operations requires not only technical expertise but also the ability to adapt to dynamic environments. Specialized training programs are critical to preparing teams for these challenges, ensuring they can handle the demands of scalable drone swarms effectively and safely.

Training Operators for Complex Systems

Operators are at the heart of drone swarm missions. Unlike single-drone systems, managing a swarm involves overseeing the behavior of multiple drones working in coordination. Effective training equips operators with the skills to manage these systems while responding quickly to mission-specific challenges.

- **Building a Strong Foundation:** Training for operators should start with the basics, such as understanding how drone swarms communicate, navigate, and allocate tasks autonomously. Operators need to grasp these concepts thoroughly to oversee operations confidently and anticipate potential problems.
- **Hands-On Simulations:** Real-world scenarios can be difficult to replicate, so simulation-based training plays a vital role. Simulations provide operators with the chance to practice in controlled

environments that mimic realistic challenges, such as communication failures or sudden changes in terrain. For example, operators might practice adjusting swarm behavior in response to an unexpected obstacle or prioritizing tasks when resources are stretched thin. These experiences prepare them for high-pressure situations where quick thinking is essential.

- **Developing Decision-Making Skills:** While drones in a swarm operate with a high degree of autonomy, operators still need to make key strategic decisions. Training should focus on helping them analyze data from drones in real time, adjust mission objectives, and step in when systems encounter issues. This includes identifying when to trust the system to operate independently and when human intervention is necessary.
- **Crisis Management:** In SAR missions, unexpected challenges are inevitable. Training should prepare operators for these moments by teaching them how to handle emergencies effectively. For instance, if several drones in the swarm lose connectivity, operators must know how to redistribute tasks and ensure the mission continues without major disruptions.

Training Maintenance Teams for Technical Excellence

Behind every successful drone swarm mission is a skilled maintenance team that ensures the system operates smoothly. As drones become more advanced, maintenance involves much more than repairing hardware. Teams must be equipped to handle everything from software updates to energy management.

- **Technical Skills:** Maintenance teams need a deep understanding of drone mechanics, including propulsion systems, sensors, and

communication modules. They also need to know how to diagnose and resolve issues quickly to minimize downtime during missions.

- **Keeping Software Updated:** As drone technology evolves, firmware and software updates are essential for maintaining compatibility and performance. Maintenance teams must know how to install these updates without interrupting ongoing operations. They also need to be comfortable with fine-tuning algorithms to improve efficiency or adapt to specific mission needs.
- **Proactive Maintenance:** Training should also focus on predictive maintenance techniques. By using data from previous missions, teams can identify patterns that signal potential problems and address them before they lead to failures. This proactive approach can save valuable time and resources during critical operations.

Ensuring Safety and Compliance

SAR missions often take place in complex environments with strict legal and safety requirements. Both operators and maintenance teams need to be trained in these areas to avoid complications during missions.

- **Understanding Regulations:** Drone swarm operations are subject to various airspace and privacy laws, which can vary depending on the location. Teams must be familiar with these regulations to ensure compliance during missions.
- **Implementing Safety Protocols:** Safety is a top priority in any operation. Teams should be trained to conduct thorough pre-flight checks, assess risks, and follow emergency protocols. For example, if a drone in the swarm experiences a mechanical failure, operators and technicians must know how to address the issue without compromising the rest of the mission.

Ongoing Education and Certification

Training isn't a one-time process. As drone swarm technology continues to evolve, so must the skills of the teams operate and maintaining these systems.

- **Certification Programs:** Certification ensures that team members meet industry standards and are prepared for the challenges of scalable drone swarm operations. Programs can range from basic operation and maintenance to advanced swarm management.
- **Continuous Learning:** Staying updated on the latest developments in drone technology is essential. Workshops, online courses, and manufacturer-provided training programs can help teams keep pace with advancements and refine their skills over time.

In summary, scaling drone swarms for SAR missions requires well-trained operators and maintenance teams. By investing in specialized training programs, organizations can build a workforce capable of managing complex systems and responding effectively to the unique challenges these technologies present. This commitment to training ensures that drone swarms can reach their full potential, ultimately saving lives and improving mission outcomes.

Broader Implications of Scalability

The scalability of drone swarms has transformative potential that extends beyond their technical applications in search-and-rescue (SAR) missions. This capability introduces new possibilities across diverse sectors, including societal well-being, environmental monitoring, economic development, and global policy. However, realizing these benefits requires addressing challenges such as privacy concerns, regulatory frameworks, and the ethical

use of technology. This section examines these broader implications and highlights the pathways to integrating scalable drone swarms into modern systems.

- **Societal Impact:** Scalable drone swarms have the potential to revolutionize disaster response, public safety, and humanitarian aid, especially in areas where traditional infrastructure is lacking or inaccessible. In large-scale natural disasters such as earthquakes or hurricanes, drone swarms can be rapidly deployed to survey damage, locate survivors, and deliver essential supplies. Their ability to cover vast territories quickly reduces response times and ensures equitable distribution of aid.

In urban environments, swarms can enhance public safety by monitoring large gatherings or supporting law enforcement in emergency situations. For example, during public events, drones can provide live data on crowd density or detect potential hazards. While these advancements offer immense societal benefits, they also raise ethical concerns about surveillance and privacy. Public trust can be fostered through transparent communication, clear regulations, and restrictions on data usage.

- **Environmental Monitoring and Conservation:** Beyond SAR missions, scalable drone swarms have significant applications in environmental protection and conservation. Their ability to operate across expansive areas makes them ideal for tasks such as monitoring wildlife populations, detecting illegal activities like poaching or deforestation, and assessing the health of ecosystems. For example, in Africa, drones are already used to deter poaching in protected wildlife reserves, and scaling these operations could greatly enhance conservation efforts.

In the context of climate change, drone swarms can gather real-time data on glacier melting, rising sea levels, and shifting weather patterns. This data is invaluable for informing global policies and guiding climate mitigation strategies. Additionally, scalable swarms can be deployed for reforestation projects, where drones equipped with seed-dispersing mechanisms can plant trees efficiently over large deforested areas, significantly accelerating restoration efforts.

However, these applications require consideration of environmental costs, such as energy consumption and e-waste. Employing renewable energy sources like solar-powered drones and ensuring recyclable materials in drone construction are critical steps toward sustainable scalability.

- **Economic Benefits and Challenges:** The scalability of drone swarms introduces economic opportunities across industries while presenting unique challenges. In SAR missions, for example, drone swarms can replace costlier traditional methods like helicopters, reducing operational expenses and minimizing risks to human responders. The efficiency of drones allows for faster, more targeted operations, maximizing value for both public and private sectors.

Industries such as agriculture, logistics, and infrastructure inspection are poised to benefit significantly. Scalable swarms can monitor crop health, optimize irrigation, inspect bridges and pipelines, and even support last-mile delivery services. For instance, in agriculture, drones equipped with multispectral sensors can analyze crop health on a large scale, enabling precision farming practices that increase yields while reducing waste.

Despite these benefits, scaling operations comes with financial hurdles. Initial investments in drone hardware, charging infrastructure, and maintenance are substantial. Public-private partnerships and government grants can mitigate these costs, enabling organizations to adopt scalable technologies. Additionally, workforce development programs are essential to train operators, engineers, and data analysts, ensuring that economic benefits extend to job creation.

The scalability of drone swarms offers unparalleled opportunities to address pressing global challenges, from disaster response to environmental conservation and economic development. By carefully addressing societal, ethical, and regulatory challenges, scalable swarms can be integrated into systems that benefit humanity as a whole. Through collaboration between governments, private organizations, and international bodies, this technology can become a cornerstone of future innovation, driving progress in ways that are efficient, equitable, and sustainable.

Advanced Algorithms for Drone Swarm Operations

Drone swarm technology has seen rapid advancements, with various algorithms designed to optimize operations in complex scenarios like search-and-rescue (SAR) missions. These algorithms differ in their approach, strengths, weaknesses, and the datasets they rely on. Below is a detailed exploration of ten such algorithms, focusing on their practical applications, challenges, and the types of data they utilize.

- **Particle Swarm Optimization (PSO)** is an algorithm that models drones as particles within a solution space, mimicking the natural behavior of birds or fish in coordinated movements. Each drone adjusts its position based on its own experience and the performance of its neighbors, aiming for an optimal balance between exploration

and exploitation. PSO is particularly effective in static environments where obstacle locations and terrain conditions remain stable. This approach relies on datasets such as GPS coordinates, preloaded topographical maps, and obstacle locations, enabling drones to efficiently divide and explore the search area.

However, PSO faces significant limitations in dynamic environments. It struggles to adapt to real-time changes, such as shifting victim locations or unexpected obstacles, as it heavily depends on static data. Additionally, the algorithm's tendency toward premature convergence, where drones settle on suboptimal solutions, can hinder mission effectiveness. While highly efficient for stable scenarios, PSO requires enhancements, such as hybridization with more adaptive algorithms, to handle the complexities of SAR missions.

- **Ant Colony Optimization (ACO)** draws inspiration from the behavior of ants, which leave pheromone trails to guide others toward food sources. In drone swarms, virtual pheromones are used to mark explored areas, guiding other drones to unexplored or high-priority regions. ACO shines in environments with dense obstacles, such as urban landscapes or forests, where efficient coverage is essential. This algorithm dynamically updates routes based on data from real-time sensors like LIDAR and infrared cameras, allowing drones to adapt to changing conditions.

While ACO's strength lies in its adaptability and effective area coverage, its computational demands grow significantly as swarm size increases. Continuous pheromone updates require substantial processing power, posing scalability challenges for larger missions. Additionally, adverse conditions like poor visibility can reduce the reliability of sensor data, impacting the algorithm's effectiveness.

Despite these limitations, ACO remains a robust solution for mid-sized swarms operating in challenging terrains.

- **Auction-Based Task Allocation** operates through a dynamic bidding system, where drones compete for tasks based on their proximity, battery levels, and workload. Tasks are assigned to the highest bidder, ensuring an efficient distribution of resources. This method is particularly suited for SAR missions that require real-time adjustments, such as shifting priorities when new victims are detected or obstacles emerge. Auction-based systems leverage datasets including GPS coordinates, battery status, and environmental data to evaluate and assign tasks.

However, the algorithm's dependence on continuous communication among drones can be a bottleneck in areas with weak connectivity. Additionally, the bidding process, while effective, can drain battery life during prolonged operations. These challenges highlight the importance of robust communication infrastructure and energy management strategies. Despite its demands, auction-based allocation stands out for its adaptability and resource efficiency in dynamic SAR scenarios.

- **Voronoi-Based Partitioning** divides the search area into zones based on the locations of drones, ensuring systematic and non-overlapping coverage. Each drone is assigned a distinct region, simplifying coordination and optimizing exploration in static environments like open fields or flood zones. The algorithm uses datasets such as GPS coordinates for drone positioning and predefined boundaries for area segmentation. By minimizing redundant coverage, Voronoi partitioning allows for efficient use of swarm resources.

However, its rigid structure becomes a drawback in dynamic conditions where zone boundaries must be recalibrated frequently. Adjusting partitions in response to new obstacles or victim detections can be computationally intensive, delaying mission progress. Moreover, Voronoi partitioning is less effective in cluttered or irregular environments, where predefined zones may not align with operational needs. While well-suited for stable scenarios, the algorithm requires flexibility enhancements for broader applicability.

- **Deep Reinforcement Learning (DRL)** combines the predictive capabilities of deep learning with the adaptability of reinforcement learning, enabling drones to learn optimal strategies through simulations. DRL is particularly powerful in unpredictable environments, such as disaster zones with unstable structures or shifting debris. This approach utilizes diverse datasets, including LIDAR scans, thermal imaging for detecting heat signatures, and high-fidelity simulated environments for training. By learning from trial and error, drones equipped with DRL can make autonomous decisions and adapt to novel challenges.

Despite its advanced capabilities, DRL comes with high computational costs. Training the algorithm requires extensive datasets and significant processing power, and real-time decision-making can strain onboard resources, especially in large swarms. Additionally, the quality of training data plays a critical role in its effectiveness; poorly simulated environments can limit its real-world applicability. While resource-intensive, DRL's adaptability and autonomy make it a leading choice for SAR missions with unpredictable conditions.

- **Distributed Markov Decision Processes** enable drones to make independent decisions while collectively contributing to the overall

mission objective. This algorithm is particularly effective in environments with incomplete or uncertain information, such as disaster zones with poor visibility. Dec-MDPs rely on probabilistic models, sensor inputs, and GPS data to estimate outcomes and prioritize tasks. By decentralizing decision-making, this approach reduces reliance on a central controller, increasing resilience and autonomy.

The primary challenge with Dec-MDPs lies in their computational complexity. Real-time updates and probabilistic calculations demand substantial processing power, which can limit scalability in large-scale operations. Additionally, the algorithm's performance is heavily dependent on the accuracy of sensor data, making it vulnerable in adverse conditions. Despite these hurdles, Dec-MDPs excel in handling uncertainty, offering a robust solution for SAR missions in challenging and unpredictable environments.

- **Bee Colony Optimization (BCO)** is inspired by the foraging behavior of honeybees, where drones act as scouts or workers to explore and exploit search areas effectively. In the context of drone swarm operations, BCO is particularly advantageous for SAR missions that require adaptive exploration in unknown terrains. Drones acting as "scouts" identify promising areas, while "worker" drones optimize coverage based on the data provided. This algorithm relies on datasets such as real-time sensor data (e.g., thermal imaging, LIDAR) and positional coordinates to navigate through dynamic environments.

BCO's strength lies in its adaptability to changing scenarios and its ability to balance exploration and exploitation. This makes it highly effective for environments with limited prior knowledge, such as disaster zones or heavily forested regions. However, the algorithm's

effectiveness is dependent on the quality of sensor data and the communication network. In cases of poor visibility or signal interference, the reliability of the algorithm decreases significantly. Additionally, BCO can become computationally expensive as the swarm size grows, requiring significant resources to process and adapt routes in real time. Despite these limitations, BCO remains a valuable algorithm for mid-sized drone swarms in adaptive exploration tasks.

- **Flocking** algorithms mimic the behavior of bird flocks and fish schools, where individual drones maintain alignment, cohesion, and separation to ensure smooth collective movement. These algorithms are primarily used for formation control and obstacle avoidance in drone swarms, making them ideal for missions requiring coordinated group maneuvers. Flocking algorithms typically utilize datasets such as positional data, velocity vectors, and real-time obstacle detection inputs to guide drone behavior.

The simplicity of flocking algorithms is their greatest strength, enabling computational efficiency and ease of implementation. They are particularly effective in ensuring collision-free navigation and maintaining formation in relatively stable environments. However, flocking algorithms face challenges in highly dynamic or cluttered environments, where rapid changes in the surroundings demand more complex decision-making. Additionally, the lack of a global optimization mechanism can result in suboptimal performance in tasks requiring precise target detection or resource allocation. While flocking algorithms are limited in adaptability, they excel in scenarios requiring efficient group coordination and robust obstacle avoidance.

- **Genetic Algorithms (GAs)** mimic the principles of natural selection to tackle optimization challenges, making them highly effective for

drone swarm tasks like route planning and resource allocation. In this approach, potential solutions are represented as "chromosomes," and their effectiveness is assessed based on specific objectives, such as reducing energy consumption or maximizing area coverage.

One of the key strengths of GAs is their ability to handle complex, multi-objective problems in non-linear, expansive solution spaces. This capability makes them particularly useful for operations where drones need to balance efficient exploration with resource conservation. However, the computational intensity of GAs can be a drawback, as they require significant processing power and time to converge to optimal solutions, which may be a limitation in missions with tight time constraints. GAs use datasets like GPS coordinates, terrain maps, and battery status to continuously improve solutions. These inputs allow the algorithm to adapt and refine drone operations dynamically. While resource-intensive, GAs are particularly valuable in scenarios like search-and-rescue missions, where task priorities and environmental conditions can shift rapidly, requiring a high degree of flexibility.

- **Firefly Algorithm (FA)** is inspired by the flashing behavior of fireflies, where the brightness of a firefly attracts others, mimicking natural swarm intelligence. In drone swarm operations, FA is particularly effective for tasks like area exploration, obstacle avoidance, and target detection. Each drone in the swarm is treated as a firefly, with its "brightness" determined by a fitness function that evaluates mission objectives, such as proximity to a target or efficiency of a flight path. Drones are attracted to brighter fireflies (better solutions) and adjust their positions accordingly. This iterative process helps the swarm converge on optimal solutions while dynamically adapting to changes in the environment.

FA excels in handling non-linear, multi-modal optimization problems, making it ideal for environments with complex terrain or irregular obstacles. Its ability to quickly adapt and converge ensures effective coverage and coordination in search-and-rescue missions. However, FA can face challenges with scalability, as larger swarms require higher computational resources to manage interactions effectively. Additionally, the algorithm's performance can degrade if the fitness function is not well-designed or if data inputs—like thermal imaging or LIDAR scans—are inaccurate. Despite these challenges, FA offers robust adaptability and scalability for mid-sized drone swarms, particularly in scenarios where environmental conditions and mission objectives are highly dynamic. Its natural mimicry and simplicity make it a valuable addition to the arsenal of swarm algorithms.

The algorithms outlined above represent diverse approaches to optimizing drone swarm operations, each with its own strengths, weaknesses, and data requirements. From GPS and LIDAR inputs to thermal imaging and simulated training data, these algorithms demonstrate the critical role of datasets in enhancing scalability, adaptability, and efficiency. By carefully selecting the most appropriate algorithm for specific mission objectives and environmental conditions, SAR operations can achieve new levels of effectiveness and reliability.

Brody

Testing

The final agent will be required to complete its tasks in an effective and deterministic manner. Since the task is relying on artificial intelligence for multiple aspects of the final product, it is necessary to ensure that the

models are outputting accurate and useful data. This will require extensive tweaking training and testing of our custom and off-the shelf models.

LLM Testing

Use of an open source LLM for certain aspects of the AI agent was a requirement set by the project sponsor. It was decided by the team that an LLM would be useful for creating a more detailed description of what was seen by a drone at a detected Point of Interest. This would enable the creation a better user experience as in the final product there could be a mission map with markers and a quick description of what is at that given point. This also leads to the second task for the LLM, decision making. As the overseer drone detects points of interest and generates a description of what the detection was the LLM will consider all these points and determine which ones should be investigated further by the inspector drones to attempt to make a positive ID on the target.

Conventionally most people interact with LLMs remotely through web interfaces or API's. This is advantageous for most users as they can run prompts on the most capable models which can contain hundreds of billions of parameters on extremely powerful machines owned by large companies that otherwise cannot be run efficiently on most personal machines. Since the application for this project is designed to aid in search and rescue operations it is safe to assume that it will often be running out in the field where the possibility for internet connection is unlikely. This means that the LLM will have to run on the machine locally and this has some advantages and disadvantages. First, there is the advantage that the concern regarding latency due to a poor internet connection when attempting to get a response from the LLM's API can be disregarded. The first disadvantage is that it must be assumed that the model will be running on a laptop or an Nvidia Jetson so response generation time must be considered to practically make real-

time decisions. The second disadvantage is that interfacing with and running a local model can be difficult to implement.

Ollama makes implementation of local LLM's much easier. Ollama is an open-source project that enables a user-friendly platform for running LLM's on a local machine. It serves as a library for many open-source customized LLM models. Conveniently Ollama provides different size options for a given model, this allows us to use more trimmed down versions of models to improve compute times on the teams systems. Ollama also makes it extremely easy to interface with the LLM either through a python script or through a CLI.

In the initial testing of several models, it was clear that none of the models were predictable or tuned well enough to serve our specific needs. This means that the provided models must be fine-tuned and trained to create good results for the projects use case. Ollama allows pretrained models to be pulled to the local system which can be further trained and fine-tuned then reupload to their library so any one of the team members can download it and run tests on it locally.

Python Interface

Shortly after downloading Ollama we learned how to interface with it using python. After importing the Ollama python module you can run prompts with a single function call.

```
ollama.chat(model=model_id, messages=[{'role': 'user', 'content': prompt, 'images':  
[image_path] }  
]  
)
```

Figure 32: Ollama Function Call

By simply providing the `.chat()` function with a model id of a currently installed model as well as a prompt containing images and/or a prompt string. This will output an object containing the model's response as well as a lot of useful information about the model execution.

While the Ollama interface is intuitive and easy to use, for testing purposes it would be beneficial to have a program that members can run on any model or several models quickly without having to write any code to do so, this will also ensure that the data being output by the chat function should be standardized to allow for easy analysis and comparison of model output and performance.

Test Script

Whenever a python program is going to be used by several clients utilizing a virtual environment is important to ensure compatibility across all machines. The main reason for this is that a virtual environment allows only python modules to run that are downloaded on that environment specifically, this means that any modules that have been installed globally by members previously that could possibly be the wrong version won't be able to run within the environment. Python provides the incredibly useful `venv` console command which instantly adds the virtual environment directory which can be activated with several activate commands depending on the user's platform.

The main function of this script was to drastically speed up the testing of LLMs and eliminate the likelihood of user errors when running tests on the LLM. To achieve this a command line interface was created that leverages the `inquirer` python package which allows for advanced command line such as list selects, and checkboxes. This is incredibly useful as it allows the program to give the user feedback as to what they have available to run on their system and allows them to select what they want to run.

```
[?] Select a model (Press space to select, enter to continue):
[ ] llava-llama3:8b
> [X] llava:latest
[ ] llama3.1:latest

[?] Provide a prompt: Press <enter> to launch your editor
[?] Do you want to use an image? (y/N): y

[?] Select an image:
image.png
> Picture1.jpg

llava:latest: Generating Response...
Model: llava:latest
Response:
The main color seen in this image is purple.
Total time taken: 12.6919114 seconds
[?] Rate the response from 1 to 10: 2
Output saved to: ./output/output_23.txt
```

[Figure 33] Python Test Script

```
User: Brody Moorman
Model: llava-llama3:8b
Prompt: list the un-natural objects in this image and their pixel offsets from the top left of the image.
Response: [0.13, 0.31, 0.46, 0.56]
Total time taken: 3.1839776 seconds
Score: 2
```

[Figure 34] Python Test Script Console Output

The script works by first calling the `Ollama.list()` function and obtaining a list of all the available models on that system. The user is then asked to select a model from those available on their system, they are able to select any amount of the models on their system. The script will use all of the other provided information and run it repeatedly on each selected model to enable direct comparison of model output and performance given the same parameters.

The user then must provide a prompt, when the script asks for this the user simply presses "Enter" and a temporary .txt file is generated and opened in their notepad. The txt file is instantly parsed, and its contents are loaded into memory and the file is deleted as soon as the file is closed by the user. This is an improvement than just writing to the command line for this specific use case. Since the user is going to be using similar prompts that will likely just be copy pasted with small alterations needing to be made. This process would be slow and difficult if it was being done on the command line with no mouse input, but in a text file it can be done almost instantly.

The script will then parse the images folder within the script directory for any images that have been provided by the user and allow them to select one for use within the prompt. Certain models allow for the input of several images, in the case that a user selects one of those models to run the script will prompt the user to first select an image to run on all selected models. Then will prompt an additional checkbox question where the user can select additional images to run on the multi-image model specifically.

From here the script will begin running the models it will write to the console what model its currently running then output the response and time taken to run the prompt on the model in seconds. The user is then prompted to provide a score of the response from 1-10, it is important that the user is only scoring based on the response and not the time taken to run the model. This is because those are two different metrics that should be compared against each other separately later in the model analysis. This is repeated for all selected models and outputs are written to a respective txt file for each model response that was generated.

Web Interface

While the python script does allow for the quick and easy running of models locally on a user's system and it does eliminate most of the possibility for user error, the entire point of making this script was to create uniformly formatted data that could be analyzed and compared no matter which members system the model was run on. Having only text files saved locally on their systems would not be useful or practical for creating meaningful metrics regarding a model's performance and aid the team in identifying the optimal model to use for the final application.

The initial ideas to solve this problem was to have a channel in the discord which members could post their positive results. However, it shouldn't be expected that every test that is run on that model will be

uploaded. This will lead to a bias as likely only the positive results will be posted and there would be no clear metrics on a model's overall performance.

The decision was then made to create a web interface where results could be compiled analyzed and displayed to all users. It would also create a platform that could be later expanded to include any other collaborative tools that may be needed in the future.

Front-End

The front end of the web interface was created using react and the chakraUI library for react. UI libraries greatly improve the development time of front end's by providing reusable standard components that are commonly needed for web development. This eliminates the need for the developer to create these components from scratch and only requires the developer to add the logic and functionality to the components as well as any additional styling they would like to add. This enables a single developer to create large scale multi-page web application front-ends in a matter of hours rather than days.

ChakraUI specifically uses inline styling for all their components using a system very similar to tailwind.css. This allows the developer to do all off the work for a component in a single file rather than having to jump back and forth between JSX and CSS files to style their components.

The app was going to need a login and signup page so that users could make an account so they could generate API keys so there could be some form of authentication when trying to send requests from their machines to the server. The main page of the website has a filterable list in the center of

the screen showing all of the tests that have been run on the side of the screen there

It is important to note that for many of the more complex responses from the model newline symbols and markdown are included in the response. By default these features are not rendered properly in HTML. For example, to create a new line in an output a new text block must be created. Originally, a custom algorithm was to be created to solve this issue by creating new text blocks for each paragraph, detecting a list and rendering it as a `<ul>` or `<ol>` and creating new text objects and applying appropriate styling to them for bold or underlined text. However, this problem is incredibly complex when you account for the fact that getting multiple text blocks that need to be styled to look formatted properly within their accompanying paragraphs is an incredibly complex problem. Considering this is just a testing platform and not part of the project deliverables it wouldn't be practical to spend the time to develop such an algorithm. Luckily a react package was discovered that solves this problem called "markdown-to-jsx". While the features of this package don't completely cover what is needed to completely properly format the responses the output is certainly good enough to make responses easily readable and well formatted enough for the purpose of this application.

Routing is also something that the application needed. So that links to specific tests or images a user uploaded could be generated and shared among other members. React natively doesn't have routing capabilities but with the ReactRouter library you an application can be wrapped in the router provider and the accompanying components behind routes can be defined such that those components are rendered when navigating to the specified url. Props are also able to be passed into components based on the parameters provided in the url. For example a route could be defined as followed:

```
<Route path="/test/:id" component={TestPage}/>
```

Figure 35: Route Path

If a user then navigated to "<https://your-website/test/2> " the application would render the TestPage component, within this component it would use the useParams hook provided by the react router library as shown:

```
const { id } = useParams()
```

Figure 36: Params

and use this id to fetch data from the backend to populate the necessary data for this page. Using these methods with react router it allows emulation of traditional routing within a react app which natively is built for single page applications.

Back-end

The back-end was originally going to be implemented by allowing users to upload their output txt files to the web server where the data would be saved in a database and analyzed. However, this took away from the efficiency and ease of use principle of this testing application. Expecting a user to upload their results after each test would be incredibly tedious and could lead to the same bias as seen before where only positive results may be uploaded to the server.

The better implementation would be to allow data to be sent to the web server directly from the python testing script. There is a python library named "requests" which allows the python scrip to asynchronously send HTTP requests to any web server. The request can be initialized with the following line:

```
res = requests.post('[HTTP_ENDPOINT]', json = body )
```

Figure 37: HTTP Request

The json parameter of the function accepts a standard python dictionary which will automatically be converted to the json for the request body. The function returns an object with the response headers, the response body and any error that has occurred during the exchange.

To allow this to work the backend would need to implement a REST web API this will be done using an express.js web server. The best practice when writing an express server is to create router and controller files for each route. Then export the router from each route file to the main server file. This keeps the code well separated and componentized. The routes for this server are auth, tests and upload.

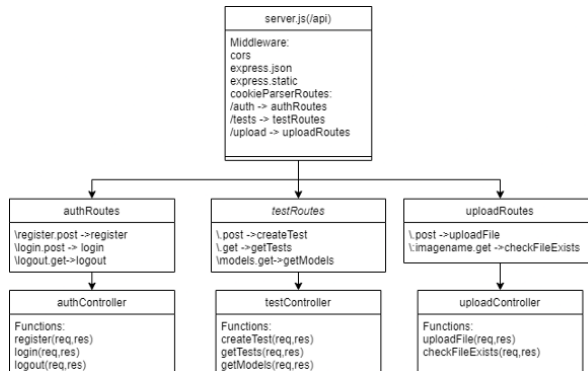
The auth route handles user signup and login, when a user signs up on the website an API key is generated for them which they put in the config file of the python script, when a user runs a test on the python script all the data from the test gets loaded into a dictionary to be sent to the server. However, if the test contained an image a request is sent to the server with the name of the image to see if it is already saved on the server, if the image is there it returns true and the data from the test is sent with the proper name of the image. If it returns false the python script sends the image to the upload route of the server the server then saves the image renames it with a unique suffix and saves its name in the database, this route then returns the new name of the image, and the python script renames the local version of the image to the matching name from the server.

The rest of the test data is sent to the server and a new test is created and stored in the database. This process is necessary since there is a high likelihood that tests will be run on different models using the same image, since it is useful to see what image was used when reviewing the model

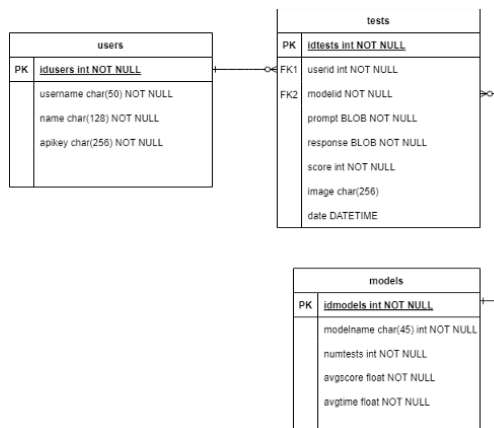
responses, the images must be stored on the server for static serving. However, if there were no checks done on existing images the server would quickly become overloaded with duplicate images. Considering the web-interface is hosted on a low-end VPS with only 50gb of storage available many duplicate high-resolution images would fill up the VPS storage quickly.

Once the test post request is made the data gets uploaded to a new row in the tests table of the MySQL database, the prompt and response fields are stored as BLOB's which allows for very long texts without the need for a lot of memory when the row is cached, the downside is that comparison queries on this data is limited and relatively slow. However, there are no queries done on these columns for this specific use case so that is not an issue. Upon a new test being posted to the server checks if the model it was run on already exists in the system. If it is in the system it reads the time taken and score of the new test and pulls the current averages for time, score, and number of tests recalculates them with the addition of the new test and then updates the metrics for that row in the models table of the database.

CORS is a middleware that is required to be used with this stack. Cors allows origins to be set from which requests to the server are permitted to be made. This is normally the address of the front end of the website. However, in this case requests need to be made from the group members local systems through the python script. There are two ways to go about this... First all of the group members IP address could be accounted for and set as origins. This would mean that members wouldn't be able to run the script if they weren't at home. The second option is to leave the cors origin as the % wildcard, this would allow anyone to make requests to the server. The downside is that malicious actors could make requests to the server. However, there is not any sensitive information on the server so the ease of implementation of this solution outweighs any real security concerns.



[Figure 38] Web-Interface Backend Request Flow



[Figure 39] Web-Interface Database Structure

Coordinate Calculations

When the AI agent begins there will be an overseer drone whose camera will likely be fixed in a straight down position. It will be necessary to

accurately estimate the global position of an object within that image for the purpose of generating coordinate waypoints for other drones to carry out their investigations of points of interest. These calculations require certain data to be known to carry out the operations. Certain sensor data from the drone will be needed from the time the picture is taken, this includes the drone's altitude, GPS position, and the compass heading of the drone. Some constant values must be known as well these include the cameras vertical FOV as well as the cameras horizontal FOV.

The following formulas were developed to carry out these calculations provided the pixel coordinates of the object in the image were successfully identified.

To begin this process, it is necessary to calculate the Ground Coverage Area (GCA) which represents the total area of ground that the image encompasses. To determine this the vertical and horizontal FOV's of the drone must be known (represented as $FOV_{vertical}$ and $FOV_{horizontal}$ respectively) as well as the altitude of the drone in meters. (represented as h). The relationship between altitude and Ground Coverage Area can be represented as:

$$GCA_{vertical} = 2h * \tan\left(\frac{FOV_{vertical}}{2}\right)$$
$$GCA_{horizontal} = 2h * \tan\left(\frac{FOV_{horizontal}}{2}\right)$$

The process will continue by determining the Ground Sample Difference (GSD) which represents the real-world distance in meters

represented by a single pixel in the image. To determine this the width and height of the image must be known (represented by $imageWidth$ and $imageHeight$ respectively). The relationship between the pixels and their real-world distance can be represented as shown:

$$GSD_{horizontal} = \frac{GCA_{horizontal}}{ImageWidth}$$
$$GSD_{vertical} = \frac{GCA_{vertical}}{ImageHeight}$$

Now assuming that the camera is facing straight down it can be inferred that the x,y pixel position of the drone if it were in the image is the direct center of the image. With that assumption the offset of the object to the drone in pixels can be represented as shown:

$$\Delta x = x_{pixel} - \frac{imageWidth}{2}$$
$$\Delta y = y_{pixel} - \frac{imageHeight}{2}$$

These pixel offsets now must be converted to real-world distances in meters, since we know the distance represented by pixels both vertically and horizontally, we can represent the offset in meters as shown:

$$d_x = \Delta x * GSD_{horizontal}$$
$$d_y = \Delta y * GSD_{vertical}$$

Now the offset in meters from the drones ground position to the images position is known. However, latitude and longitude are measures of north and east respectively. This means that these meter offsets need to be converted to north and east offsets. To do this the angle of azimuth of the drone during the time the image is taken is necessary (represented as θ). This relationship is shown below:

$$d_{north} = d_x * \sin(\theta) + d_y * \cos(\theta)$$

$$d_{east} = d_x * \cos(\theta) + d_y * \sin(\theta)$$

These offsets need to be converted to degrees of latitude and longitude. The approximate distance between degrees of latitude is 111,320m. However, it is important to remember that the distance between degrees of longitude gets smaller in distance depending on its distance from the equator. The relationships between north and east offset to latitude and longitude offset is show below:

$$\Delta lat = \frac{d_{north}}{111,320}$$

$$\Delta lon = \frac{d_{east}}{111,320 * \cos(drone\ latitude)}$$

Finally, by applying the calculated offsets to the drone's latitude and longitude at the time the image is taken (represented by lat and lon) the estimated GPS position of the object is finally known.

$$obj_{lat} = Drone_{lat} + \Delta lat$$

$$obj_{lon} = Drone_{lon} + \Delta lon$$

Caveats

The major problem with these formulas is that it is assuming that the camera is pointing straight down from the drone. This means that all pixels on either axis represent approximately the same distance in the real world. However, on our secondary scout drones it is likely that the cameras will not be pointing directly down and will be on a gimble that can adjust the cameras pitch. This leads to a distortion of distance represented by pixels in the y direction. Because the camera is pitched forwards, pixels towards the top of the image will represent a larger vertical distance than the pixels in the foreground. To empower coordinate estimation on images taken on the scout drones' formulas to predict this distortion will be necessary.

To perform these calculations the angle corresponding to the pixels vertical position relative to the center of the image must be calculated.

$$\Delta\alpha(y_{pixel}) \left(y_{pixel} - \frac{H_{image}}{2} \right) * \frac{FOV_{vertical}}{H_{image}}$$

The formula to solve this problem will be a formula to calculate the effective distance from the drone to a point on a ground given a tilt angle and a pixel row (represented as d_eff . For a given pixel at a vertical offset y_pixel , the effective slant range d_eff. to the ground at that point is represented by:

$$d_{eff}(y_{pixel}) = \frac{h}{\cos(\alpha + \Delta\alpha(y_{pixel}))}$$

While there is some distortion on the distance represented by the horizontals of the pixels it is far less apparent than in the vertical direction. For that reason, the original formulas for horizontal GSD will remain as:

$$GSD_{horizontal} = \frac{GCA_{horizontal}}{ImageWidth}$$

However, on the vertical axis each row will have a different GSD relative to its displacement from the center row of the image. This relationship can be expressed as:

$$GSD_{vertical}(y_{pixel}) = \frac{2 * d_{eff}(y_{pixel}) * \tan(\frac{FOV_{vertical}}{2})}{H_{image}}$$

Again, since the GSD on the horizontal axis isn't meaningfully changed by the pitch of the camera. The horizontal ground distance can be determined with the same formula as before:

$$d_x = \Delta x * GSD_{horizontal}$$

Since the GSD of each vertical row is different it is necessary to calculate the vertical ground displacement using a summation of each row up to that point. This relationship is represented as:

$$d_y = \sum_{i=\frac{H_{image}}{2}}^{y_{pixel}} GSD_{vertical}(i)$$

From this point the north and east offsets can be calculated the same as before with:

$$d_{north} = d_x * \sin(\theta) + d_y * \cos(\theta)$$

$$d_{east} = d_x * \cos(\theta) + d_y * \sin(\theta)$$

And the latitude longitude offsets with:

$$\Delta lat = \frac{d_{north}}{111,320}$$

$$\Delta lon = \frac{d_{east}}{111,320 * \cos(drone\ latitude)}$$

Finally, the actual coordinates of the object will also be calculated the same as in the previous method using the equations:

$$obj_{lat} = Drone_{lat} + \Delta lat$$

$$obj_{lon} = Drone_{lon} + \Delta lon$$

Code implementation

Once the overseer drones onboard object detection system running on the onboard jetson Xavier's the flags an object for analysis the drone will send the image, drone GPS position, Compass heading, and to the jetson Orin base station where the augmented LLAVA model will identify the object give it a description and return pixel coordinates of the object within the image. From here the python backend will take over.

Since python does not support method overloading as it is supported in other OOP languages different functions would be needed for the top-

down calculation and the pitched calculations. Once the coordinates are calculated they could be attached to the dictionary containing the object identification and description and added to the list to be rendered on the map at the appropriate position according to its coordinates, if it was determined by the agent that it needed further investigation it would also be added to the investigation priority queue with its calculated importance metric where upon being popped its coordinates would be sent into mission planner to be navigated to by the recently available drone.

Tool Calling

Overview

Tool Calling is a common LLM feature initially introduced by OpenAI in their GPT-4 models. It has since been adopted by many of the other front-running LLM's including Llama, Gemini, and Claude Sonnet as well as others.

The name tool-calling (sometimes referred to as function calling) can be a somewhat misleading name for the feature, as natively the LLM is not actually able to call or run functions itself. The core concept of the feature comes from the need for an LLM to be able to fetch necessary information or do calculations that it alone can't accomplish.

The way the feature works on most models is the programmer will provide the model with a "tool_set". A toolset is a list of all the functions that would be available for the model to use, the definition of a tool will be different depending on the model or library being used. However, tool definitions all generally contain the same information. A tool is defined with a name the name represents the function identifier in the code implementation, it must also contain a list of parameters each parameter should be given a name, a type, and a description of what the parameter

represents. It is also important to provide which parameters are required for the tool to run. There should also be a description of what the function does, a full implementation is not required but a brief description of what the function is meant to do and how its done is beneficial for the LLM's understanding of the tool. Finally, the type of the returned data should be defined.

```
tools=[{
  'type': 'function',
  'function': {
    'name': 'get_current_weather',
    'description': 'Get the current weather for a city',
    'parameters': {
      'type': 'object',
      'properties': {
        'city': {
          'type': 'string',
          'description': 'The name of the city',
        },
      },
      'required': ['city'],
    },
  },
},
],
```

[Figure 40] Example of tool definition as python dict

Given these tool definitions the LLM will be able to for structured outputs that would be able to fill the required fields to run the specified function using information found in the prompt or with previous knowledge. Which could then be run by an external program.

Tool Calling in Ollama

Ollama supports tool calling in supported models. Use of this feature is made incredibly simple. In python a list of tools can be created each being represented as a dict. Then the tool list is passed into the Ollama.chat() function by passing it as the "tools" parameter. Then once run with a prompt the model will decide if a tool should be called, if it is appropriate the model will populate the necessary parameters for the function and store that

information as well as the function name as an item in the 'tool_calls' list in the dictionary returned by the chat() function.

```
'tool_calls': [
  {'function': {'name': 'get_current_weather', 'arguments': {'city': 'Toronto'}}}
]
```

[Figure 41] Example tool call return from Ollama.chat()

From here the programmer could run these functions by looping through all the calls in the list.

Groq

Groq provides a model called llama3-groq-tool-use which comes in an 8 billion parameter model and a 70 billion parameter model. These models were tuned on top of llama3 and are specifically optimized for tool calling. They allow for more reliable calling of several tools for a single task and drastically improves inference speed when selecting tools when compared to the standard llama3 models.

Shortcomings of tool-calling in Ollama

While Ollama does provide a simple and native way to call tools in their available models there is one evident issue with their implementation. It doesn't provide a natural way to feed back information from the called tools back to the LLM. This in turn creates the limitation that the LLM is incapable of calling multiple tools which are dependent on returned values from one tool to inputs of another. This process is known as tool-chaining.

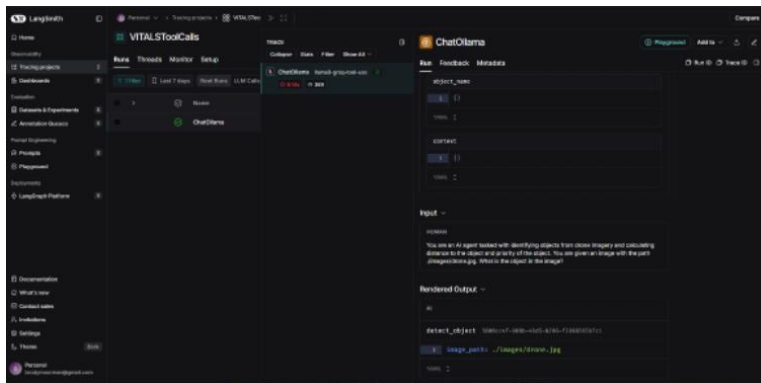
Tool chaining is an important feature for more intricate AI agents as it enables them to perform complex tasks that require multiple steps and decision-making based on intermediate results. Without tool-chaining, the capabilities of the AI are restricted to isolated operations, limiting its usefulness in scenarios like data processing pipelines, multi-step problem-solving, or dynamic decision trees. This shortcoming makes it less suitable

for applications that demand adaptability and deeper integration between tools, such as those seen in advanced automation.

LangChain

LangChain is a powerful framework designed to simplify the development of applications that integrate large language models with external tools and structured workflows. It provides a modular architecture for building AI-powered solutions that can handle complex tasks, such as retrieving and processing data, performing multi-step reasoning, and interacting with APIs or custom tools. LangChain excels in enabling tool-chaining, allowing outputs from one tool to seamlessly serve as inputs for another, thereby supporting intricate and dynamic workflows.

LangChain additionally provides a suite of development tools to aid in the processes of developing and monitoring an AI agent. These tools include LangSmith and LangGraph. LangSmith provides an in-depth analysis of calls made to an AI agent through their intuitive web interface. The interface displays each run by the agent and will display the tools the model knew it had available as well as the models understanding of what the tool was able to do, a tool trace which will display data on the tools that were called and which order they were called in, as well as important metrics such as overall latency, time to first token, as well as total tokens used. One of the best parts of LangSmith is its ability to promote collaboration. The web interface is able to be connected to with a project specific API key in addition to a user specific API key which allows any member of the team to run tool chaining tests on their own system while still being able to provide data to the rest of the team members.



[Figure 42] Langsmith Interface Example w/ call trace

LangGraph

LangGraph is another library within the LangChain ecosystem that allows for development of highly customized and adaptable AI agents. While LangChain addresses the agentic model as an object with tools attached to it, LangGraph follows a different approach by addressing the model using graphs such that it behaves as an agentic state machine. The model works by considering each node of the graph as any sort of executable function, whether that is a hardcoded callable tool such as a python function, a tool-call in which an LLM will make a formatted output to input into a function, another LLM with a specific task, or a separate model such as a vision model. The Agent will traverse the graph to eventually get to an end condition.

Implementation

Graph Definition

Initially an AgentState class must be defined, this class is necessary to maintain the Agent-oriented decision-making, in this class there is a

definition for the user input which will track the most recent message from the user, the agent output which is used by the graph to consume agent outputs, and the intermediate steps which is a list maintained during runtime throughout the traversal of the graph which will keep track of the results determined in previous steps.

```
from typing import TypedDict, Annotated, List, Union
from langchain_core.agents import AgentAction, AgentFinish
import operator

class AgentState(TypedDict):
    input: str
    agent_out: Union[AgentAction, AgentFinish, None]
    intermediate_steps: Annotated[List[tuple[AgentAction, str]], operator.add]
```

[Figure 43] AgentState class defined in Python

Every executable function or state that the agent can be in, or use must be defined as nodes in the graph. These will each be defined as their own function taking a list as an input parameter representing the current state of the agent. If the node is an intermediate node meaning that it isn't initiating or returning the final response the function will return its results to be stored in the `intermediate_steps` field of the agent's state. Otherwise, the function will return its result under the `agent_out` field of the agent's state.

```
def execute_search(state: list):
    print("> execute_search")
    action = state["agent_out"]
    tool_call = action[-1].message_log[-1].additional_kwargs["tool_calls"][-1]
    out = search_tool.invoke(
        json.loads(tool_call["function"])(["arguments"])
    )
    return {"intermediate_steps": [{"search": str(out)}]}
```

[Figure 13] Example Intermediate node definition in python

Each transition between executables is an edge of the graph. Being that most agents have access to several different executables after a specific

node many nodes will need to have conditional edges out of them which in LangGraph are referred to as “routers”.

Routers, are actually defined in the same way the nodes are defined however, they can essentially be considered if-else or switch statements to determine which node should be traversed based on the context of operations already enacted, the data which is already known by the model, and the prompt itself. Routers are especially important in complex agents such as the one necessary for this particular project as some nodes require information from other nodes to continue. Router node function definitions also take the state list as a parameter. However, they don’t directly make changes to the AgentState instance as the other nodes do. Instead, routers will return a string which represents a key which will map to the next node the graph should enter which will be defined later in graph assembly.

```
def router(state: list):
    print("> router")
    if isinstance(state["agent_out"], list):
        return state["agent_out"][-1].tool
    else:
        return "error"
```

[Figure 44] Example router definition in Python

In this example the router is checks which tool was flagged to be called by the last state, it then returns the name of the tool which was called. However, if there was no tool called it returns error meaning that the agent wasn’t able to generate the input for a applicable tool in the previous state. From here the error state could pass the state to an LLM which could determine which needed user data is missing and provide feedback to the user based on what’s missing that tripped the error. For example, in the drone AI Agent if a user asked “Search the corner of the search perimeter” the error flag would trip because a specific location was not specified. Passing the state to the error handling model it would realize that a corner

needs to be searched but it doesn't know which one. So, it may use this context to tell the user "Please, specify which corner you would like to be searched" and the graph state would be reset with the proper user input. Routers like this are important when creating complex agents to ensure that the Agent does not attempt to carry out further operations with missing or impractical parameters. It is often good practice to include error checking routers similar to this one after any call in which an LLM is responsible for the tool output to ensure that the data is correctly formatted before continuing operations.

It is also important to note that multiple router node definitions are allowed, and each conditional edge can use different router functions or the same.

In the drone command agent one of the necessary tasks is to determine a point of interest and determine its priority to be investigated by the drones. For this to work the point of interest needs to be assigned a priority metric which will determine its ranking on the priority queue for investigation by the drones. However, initially all that's given to the agent is an image from the overseer drone, and some drone meta data such as its GPS position, altitude, and heading. The Agent then needs to run the image through a vision model to identify the object and its position within the image from here a router can be used to choose whether it should first use another LLM to determine the priority of that object or if that LLM is currently busy it can run the coordinate calculation function instead. Once all required information is acquired it can reach the end condition where it sends the completed POI object to the priority queue.

Graph Construction

LangGraph provides an incredibly simple interface for creating the agentic state machines. Initially you must import the StateGraph class and END

object from the `langgraph.graph` library. `StateGraph` is the actual graph datastructure in which the nodes and edges will be stored in. `END` is a special state node which tells the Agent to stop operations as the prompt response is complete or errored out. You can initialize the `StateGraph` by passing the previously created `AgentState` to the `StateGraph` constructor.

Each node can be instantiated by calling on the `StateGraph` instance:

```
graph.add_node(node_name:str, node_definition:func)
```

It is important to note that while routers are defined and act similarly to the other nodes, they are not added to the graph in the same fashion in which the other nodes are added. Instead, they have their own unique way of being bound to an edge which will be defined later.

The entry point of the graph must also be set using:

```
graph.set_entry_point(node_name:str)
```

Unconditional edges can be set using :

```
graph.add_edge(X:str, Y:str)
```

Where `X` is the starting node of the edge and `Y` is the destination node.

Conditional edges are a bit more complicated. The `add_conditional_edge()` function provided within the `StateGraph` class requires some parameters. First, it requires a *start_key* which represents the name of the node from which the state is initially on. Next a *condition* is required, the condition is the router function previously defined. Finally, the *conditional_edge_mapping* must be provided. This expects a dictionary where each key represents a string that can be returned by the router

function and each corresponding value is the name of the node to be traversed to based on that condition. It is important that all possible returned strings from the router function are accounted for in this mapping.

```
graph.add_conditional_edges(  
    start_key="query_agent", # where in graph to start  
    condition=router, # function to determine which node is called  
    conditional_edge_mapping={  
        "search": "search",  
        "error": "error",  
        "final_answer": END  
    }  
)
```

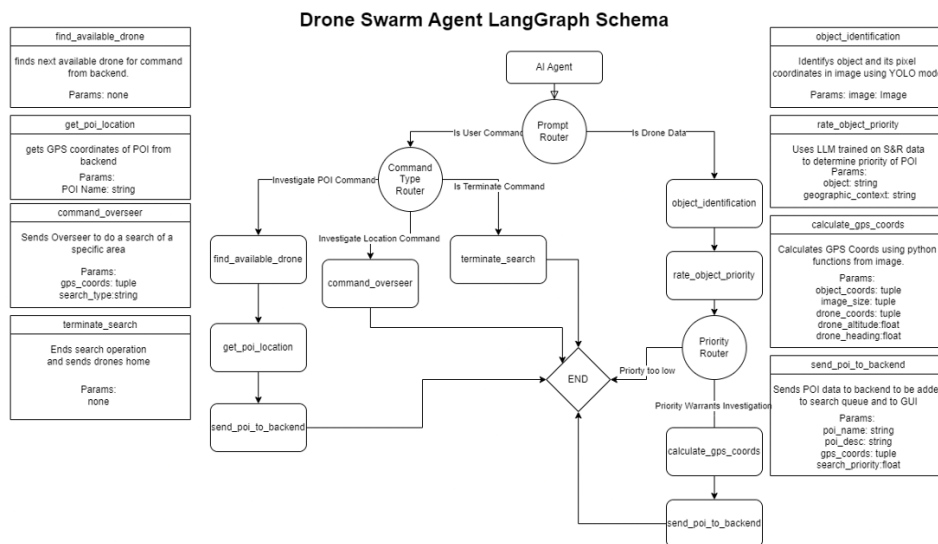
[Figure 45] Example of conditional edge initialization Python

Once all of the nodes and edges are defined and initialized within the StateGraph instance the graph must be compiled into its executable form. The StateGraph class provides an internal compile() function to do this. It should be assigned to a new variable which can then be run using the .invoke() function providing it an object with the input string and a chat history list if applicable. Otherwise just an empty chat history list should be provided.

LangGraph for Drone Agent

LangGraph plays a crucial role in powering the drone swarm AI agent as it requires the use of several different models and tools to carry out its operations. LangGraph will also enable the implementation of a “user-on-the-loop” model by allowing user input to be processed and acted on by the agent while simultaneously running its own search operations and decision making. An example of this principle would be the overseer drone searching the area and sending possible POI hits to the agent which are automatically processed, but during this time a user could write to the agent to search a specific area which the Agent would process into specific drone commands.

The LangGraph schema for the drone agent independent operations as well as on the loop user interjections can be defined with this diagram.



[Figure 46] LangGraph Schema Diagram for Drone Swarm

This diagram represents that graph that the agent will traverse on every single call to the agent whether that be an automatic call from drone image data or on a mid-operation user input. It's necessary to note that a request queue will be necessary since there will be several drones operating at the same time which would inevitably lead to backup of requests since the agent is relying on LLMs for several steps which are slow. This queue will prioritize user input over automatic agent calls to minimize latency from user input and agent reaction.

Application Backend

The backend of the application will consist of a list of all the POIs that have been identified. These will each be stored as an instance of the POI class. The POI class will need to have store information about each POI such as its location, its description, its priority rating and if it's been searched yet. As POIs are added by the AI agent they will be stored on this list which will be rendered both as a list as well as points on the interactive map which can be selected by the user to show all the available information known about the given POI.

There is also a priority queue containing the POIs that have not been searched sorted by their priority rating. As seeker drones become available, they will be assigned the top POI of the queue, and the highest priority POI will be popped from the queue. This will automatically send a command to mission planner informing it to send the available drone to the coordinates of the POI.

Once the POI is investigated relevant images from the investigation will be attached to the POI objects such that the user is able to see them on the GUI when selecting a POI. Upon human inspection of the images the user may see something in the image that the Agent missed in this case, the user would be able to re-add the POI to the search queue to be investigated again as to not miss anything important.

There also must be a list of all the drones on the operation, these will each be instances of the drone class. The drone class will have a drone_id, a drone_role, and a drone_status. This will be used to track the status of the drones as they work on the operation. As an investigation completes the drone will change its status to available which will trigger the search queue to pop and assign the top POI to that drone.

Once a drone gets assigned coordinates that GPS data will be sent to mission planner which will communicate with the onboard flight systems of the drones which are configured by the hardware teams to move to coordinates in an efficient manner without collision with other drones or obstacles.

Final Implementation

Through the development process some changes were made at the advisory of sponsors, coordinators, and the team. One of the major roadblocks throughout the development process was a lack of information regarding the specific hardware components of the drones and the inability to test on the physical drones during development.

NOTE: This is a high-level overview of our implementation for a more technical overview check the [github](#)

Application Structure

Mission State

The mission state was the primary part of our application for data storage and processing and functioned as the entry point for our application. This code is located in `app/missionState.py`. In this file we constantly update and store drone telemetry data and this is where all other modules of the application are called from such as dispatcher or GUI calls. Within this class jobs for the drones are created and coordinates are generated with the help of utility classes and the jobs are stored within the priority queue of each drone.

The drone priority queue works by each drone having an active job and a queue of jobs that are waiting to be executed. When a new job is added to a drone the priority of that job is checked and if it is greater than 3 priority points more than the active job, the active job gets pushed back to

the queue and the new job becomes active and gets sent to the dispatcher for job deployment.

Dispatcher

The dispatcher module and class functions as the point of contact to and from the drones. A connection is made to a socket (TCP by default) to a mavlink instance where packets to and from the drones are mirrored through. The dispatcher runs on two threads, a sender thread and a listener thread. The listener thread is running the function `recieve_packets`. Within this function there are several `elifs` targeting specific packets. You can easily add more packets to be checked by just adding more `elifs`.

The sender thread works by initially taking a mission sent by the mission state and stores that mission in a dictionary called `uploading_missions` indexed by the `drone_id`. Within this dictionary we store the list of waypoints, the index of the next waypoint to be sent, and a flag if the dispatcher is waiting for the drone to request a waypoint. This is necessary because the mavlink protocol requires waypoints to be sent individually. Firstly, the dispatcher notifies the drone how many waypoints are in the mission, then the drone requests the first waypoint, the dispatcher responds with that waypoint and then waits until the subsequent waypoint is requested. The flag in the `uploading_missions` dictionary is reset by the listener thread once a waypoint request is sent which allows the sender thread to send the next waypoint.

GUI

The GUI class just functions as a visual representation of what is happening in the `missionState` and also has some inputs for the user. It is made using the CustomTkinter framework. It is important to note that the GUI does not call anything directly on a users input, it instead calls a function within `missionState` which will actually perform the action.

One important thing when developing the GUI was when making subclasses of the GUI it was extremely helpful to pass a reference to the top-level instance of the actual GUI class this made it a lot easier when a button in that subclass needed to make a call to the missionState.

LLM Interactions

When interacting with the LLM it was originally planned to use a langGraph instance. However, it turned out to be overkill for the use case. Instead, just standard Ollama function calling was used to allow the LLM to interact with the application. To empower this the actual function that the LLM would call would be implemented in the missionState, however within the LangChainMain.py where the LLM functions were written there exists a list of the functions that the LLM can call. The function names must match the names of the real functions in missionState however these should just be annotated to reflect what they do such that the LLM has context, but they don't need to do any function on their own. This is because the LLM will return a list of function calls to the GUI where the LLM call was made then the GUI iterates through the list and calls the matching function name from the missionState.

Terrain Pre-Processing

To conduct the search and rescue operations, the drone must have information about the geography surrounding it, since certain features in the terrain would lead to a higher likelihood that a missing person would be there.

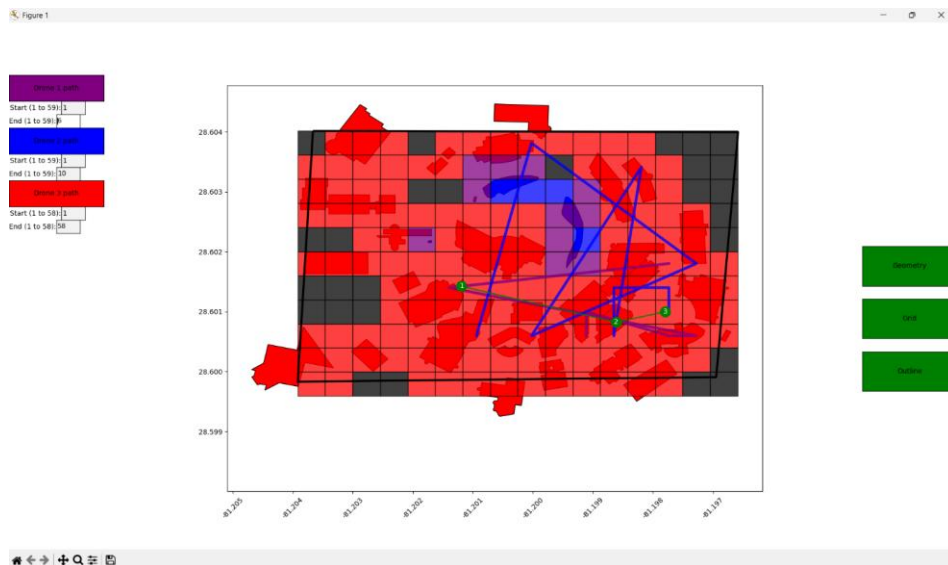
To gain the data that would be used, OSMX was used. This helped during development since it had an easy to utilize API that allowed us to quickly test and prototype different terrain processing approaches. However, in a real

search and rescue mission there's a high likelihood that internet connection won't be available. To remedy this, a PostGIS database hosted on a Docker container was implemented, so that if the app detects that there is no internet connection, it'll switch over to it.

Once gaining the data was solved, splitting the terrain was the next step. Initially, splitting the terrain into quadtrees was discussed, however, a large challenge of it was that the map rendering software that our app used didn't give us specific information about the tile dimensions, and what any given tile's longitude/latitude was.

Eventually, we decided on avoiding the quadtree and instead just focusing on the polygon that a user would designate as the search area. Using the points of the polygon, we were able to query either OSMX, or the PostGIS database to get the geographical information of that given area.

Then, we split up the search area into a grid. We were able to designate what the minimum grid length, and grid width would be. We were also able to ensure that the tiles would be a certain width long, and width high in meters.



[Figure 47] Matplotlib visual representation of drone paths, geographical features, and grid

Once the grid was split up, Matplotlib was utilized in order to visualize the grid with the features. Squares contained colors pertaining to their geographical feature, and squares with more than one feature had their colors mixed, to represent that there was more than 1 feature in that square.

Path Planning

The Path Planning for our drones is computed using the Terrain Preprocessing, using the Roads and Rivers as Priority Scores. The Algorithm creates a series of coordinates in an array that is passed to the drones through MissionPlanner, and the path planning is designed to choose nearby and high-priority grid squares first. The Priority Score is combined with grid distance to the given Drone in a Modified A Star Algorithm. This gives the coordinates at the center of each grid a score, increased by Priority, and

reduced by distance. Modifying the weights multiplied to both the Priority Score and Distance allows us to optimize the path to first go to nearby, high priority grid squares.

We also have worked on other versions of the path planning, including a prototype version that splits up the entire area into sections equal to the number of drones, and assigns each Drone to one area, then using the normal path planning algorithm to search the smaller sub-sections.

Ideas for further improvement

Due to the lack of physical drones for the presentations a simulation mode was developed for our application. This allows the user to define detection points which simulates the drone's onboard CV detecting a person and sending the image to the application for analysis. This is what the group used for all of its demos and presentations. The application is supposed to be able to receive the image from the drone in segments and then reconstruct the image which then gets sent through the CV to draw bounding boxes. The coordinates of the bounding boxes as well as the drone telemetry should then be used to estimate the position of the object. An early untested implementation of this prediction algorithm is defined in `app/Utils/coordinate_estimation.py`. Future groups should test and refine this algorithm and finally implement it to be used when generating POIs. Future groups would also need to implement the logic to receive the image data packets when a detection is made and assemble the image to be used by the application.

Bibliography and Links

Jean Berger a, a, b, Trummel, K. E., Benkoski, S., Stone, L., Hohzaki, R., Lau, H., Latombe, J.-C., Svennebring, J., Yang, S., & Rekleitis, I. (2014, June 23). *An innovative multi-agent search-and-rescue path planning approach*. Computers & Operations Research.

<https://www.sciencedirect.com/science/article/abs/pii/S0305054814001749>

https://www.researchgate.net/publication/344180309_Anticipatory_Human-Robot_Path_Planning_for_Search_and_Rescue

"FAA Part 107 Regulations for Drones." *Federal Aviation Administration*, 2024, www.faa.gov/uas/commercial_operators/part_107/. Accessed 21 Oct. 2024.

"Drone Laws in Florida: Everything You Need to Know." *Cult of Drone*, www.cultofdrone.com/drone-laws-in-florida/. Accessed 21 Oct. 2024.

"Florida Drone Regulations and State-Specific Laws." *Drone Zone FAA*, www.dronezone.faa.gov. Accessed 21 Oct. 2024.

Hind, Timothy. "Drone Swarms in Search and Rescue Missions: How Technology Is Saving Lives." *Journal of Rescue Robotics*, vol. 22, no. 2, 2023, pp. 15-29.

Smith, John. "Advancements in Autonomous Drone Swarm Technology for Search and Rescue." *International Journal of Unmanned Systems*, vol. 30, no. 1, 2022, pp. 34-47.

Thompson, Clara. "The Role of Drones in Humanitarian Aid and Disaster Relief." *Journal of Humanitarian Technology*, vol. 18, no. 3, 2022, pp. 112-129.

"Using Drones for Search and Rescue Missions." *Search and Rescue Times*, 20 July 2023, www.sartimes.com/drones-rescue-missions/. Accessed 21 Oct. 2024.

Zhang, Liwei. "Communication Systems for Drone Swarms in Disaster Management." *IEEE Transactions on Communication Systems*, vol. 56, no. 3, 2023, pp. 78-91.

Brown, Jessica. "Ethical Implications of Drone Technology in Civilian Rescue Operations." *Journal of Ethics in Technology*, vol. 19, no. 1, 2023, pp. 47-60.

Williams, Robert. "Data Management Challenges in Large-Scale Drone Operations." *Journal of Data Science*, vol. 34, no. 2, 2024, pp. 23-45.

Nguyen, Alina. "Regulatory Challenges for the Adoption of Drone Swarm Technology in Civilian Rescue." *International Journal of Drone Policy*, vol. 12, no. 4, 2023, pp. 92-110.

Miller, David. "Sustainability and Environmental Impact of Drone Technology." *Environmental Research Letters*, vol. 15, no. 6, 2023, pp. 230-250.

"How Can You Design and Implement Drone Swarm Algorithms?" *Drone Swarm Algorithms: Design and Implementation Guide*, www.linkedin.com, 25 Sept. 2023, www.linkedin.com/advice/1/how-can-you-design-implement-drone-swarm-algorithms-skills-drones.

An Investigation of Search Algorithms for Aerial ..., www.ieworldconference.org/content/WP2022/Papers/18-GDRKMCC-22.pdf. Accessed 22 Nov. 2024.

Jan-Hendrik Ewers Aerospace Sciences Research Division School of Engineering University of Glasgow j.ewers.1@research.gla.ac.uk & David Anderson Aerospace Sciences Research Division School of Engineering University of Glasgow dave.anderson@glasgow.ac.uk &D. *Deep Reinforcement Learning for Time-Critical Wilderness Search and Rescue Using Drones*, arxiv.org/html/2405.12800. Accessed 22 Nov. 2024.

"Search and Rescue." *Wikipedia*, Wikimedia Foundation, 15 Nov. 2024, en.wikipedia.org/wiki/Search_and_rescue. *

"RLHF." *LLaVA*, llava-rlhf.github.io/. Accessed 22 Nov. 2024.

"Llava." *Ollama*, ollama.com/library/llava. Accessed 22 Nov. 2024.

Ollama, ollama.com. Accessed 22 Nov. 2024.

Haotian Liu, Chunyuan Li. "Next: Improved Reasoning, OCR, and World Knowledge." *LLaVA*, 30 Jan. 2024, llava-vl.github.io/blog/2024-01-30-llava-next/.

Mitre-Atlas. "Releases · Mitre-Atlas/Atlas-Data." *GitHub*, github.com/mitre-atlas/atlas-data/releases. Accessed 22 Nov. 2024.

<https://atlas.mitre.org/mitigations/AML.M0020>

<https://atlas.mitre.org/mitigations/AML.M0021>

Young, Brett. "How to Fine-Tune Llava on a Custom Dataset." *W&B*, 27 Jan. 2024, wandb.ai/byyoung3/ml-news/reports/How-to-Fine-Tune-LLaVA-on-a-Custom-Dataset--Vmlldzo2NjUwNTc1.

"Starling-LM." *Ollama*, ollama.com/library/starling-lm. Accessed 22 Nov. 2024.

arXiv:2411.12967

arXiv:2405.12800v2

arXiv:2312.04943v1

arXiv:2405.10042v1

arXiv:2406.16370v1

arXiv:2212.00844v3

Dainis, B. (2024, September 12). *LangChain vs. LangGraph Framework Comparison*. Curotec. <https://www.curotec.com/insights/langchain-vs-langgraph-framework-comparison/>

Van Berkel, Niels. "Drone Swarms to Support Search and Rescue Operations: Opportunities and Challenges." SpringerLink, 2023. https://link.springer.com/chapter/10.1007/978-3-031-28138-9_11.

Williams, Brian. "Making Search and Rescue Drone Swarms a Reality." Virginia Tech Magazine, 2021. <https://eng.vt.edu/magazine/stories/fall-2021/drone-swarms.html>.

Feinsperger, Patrick, et al. "Drone Swarm Search Environment (DSSE)." GitHub, 2023. <https://github.com/pfeinsper/drone-swarm-search>.

Meshcheryakov, Roman, et al. "Human-Swarm Interaction in Semi-Voluntary Search and Rescue Operations." ACM Digital Library, 2023. <https://dl.acm.org/doi/fullHtml/10.1145/3569562.3569564>.

"Toward Robust and Intelligent Drone Swarm: Challenges and Future Directions." IEEE Xplore, 2020. <https://ieeexplore.ieee.org/document/9048609>.

Hale, Jonathan. "Multi-Modal and Dynamic Particle Swarm Optimization Algorithm for Drone Search & Rescue." GitHub, 2023. https://github.com/jonhale1/ME740_vision_robotics_-_planning.

Zhao, Jacky. "DroneNet: Decentralized Drone Swarm Communication." GitHub, 2023. <https://github.com/jackyzha0/DroneNet>.

Kennedy, James, and Russell C. Eberhart. "Particle Swarm Optimization." Proceedings of the IEEE International Conference on Neural Networks (ICNN), 1995, <https://ieeexplore.ieee.org/document/488968>.

Dorigo, Marco, and Luca Maria Gambardella. "Ant Colony System: A Cooperative Learning Approach to the Traveling Salesman Problem." IEEE Transactions on Evolutionary Computation, 1997, <https://ieeexplore.ieee.org/document/585892>. Dias, M. Bernardine, et al. "Market-Based Multirobot Coordination: A Survey and Analysis." Proceedings of the IEEE, 2006, <https://ieeexplore.ieee.org/document/1678027>.

Sugihara, Kunihiro, and Isaac P. Sobel. "Quantitative Analysis of the Cooperative Behavior in Multirobot Systems Using Voronoi Partitions." Springer Autonomous Robots, 1994, <https://link.springer.com/article/10.1007/BF01421588>.

Mnih, Volodymyr, et al. "Human-Level Control Through Deep Reinforcement Learning." Nature, 2015, <https://www.nature.com/articles/nature14236>.

Bernstein, Daniel S., et al. "The Complexity of Decentralized Control of Markov Decision Processes." Mathematics of Operations Research, 2002, <https://pubsonline.informs.org/doi/10.1287/moor.1030.0063>.

Trask, S., et al. "Self-Healing Polymers for Aerospace Applications." *Materials Today*, vol. 21, no. 7, 2021, pp. 743-757. <https://doi.org/10.1016/j.mattod.2021.01.003>.

Yang, Q., and Li, X. "Modular Drone Designs: Enabling On-the-Fly Repairs." *Journal of Robotics Research*, vol. 38, no. 4, 2022, pp. 511-528. <https://journals.sagepub.com/doi/10.1177/0278364921102120>.

"Establishing Global Standards for Drone Swarms: Challenges and Opportunities." *International Civil Aviation Organization Journal*, 2023. <https://www.icao.int/publications>.